
DOMAIN: AI-FIRST SERVICE DESIGN

The Agentic Service Designer

How AI Agent Teams, Service Blueprints-as-Code, and Autonomous Tooling Are Transforming Service Design from Insight to Operations

Sam Mozaffari

AUGUST 2026

The Agentic Service Designer

How AI Agent Teams, Service Blueprints-as-Code, and Autonomous Tooling
Are Transforming Service Design from Insight to Operations

Copyright © 2026 Sam Mozaffari.

First edition · August 2026 · Version 1.0.0

License

Copyright (c) 2026 Sam Mozaffari. Personal and noncommercial use: the prose of this work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0) — <https://creativecommons.org/licenses/by-nc-nd/4.0/>. You may share the unadapted work with attribution for noncommercial purposes. You may not sell this book, redistribute adapted editions, or use it primarily for commercial advantage without a separate commercial license. Commercial and organizational use (including company or team distribution, paid training, and resale on any storefront) requires a commercial license from the copyright holder — contact the author via the book repository or website. The copyright holder retains the exclusive right to sell editions via Amazon Kindle Direct Publishing (normal KDP, non-exclusive) and other retailers. Code samples in this work are licensed under the Apache License 2.0 or MIT License, as noted in context. Third-party trademarks, brand systems, templates, and assets retain their original licenses and rights.

About the Author

Sam Mozaffari

Service Designer • AI-First & Agentic Practice • Sydney, Australia

Sam Mozaffari is a practicing service designer pioneering the integration of autonomous AI agent teams into enterprise service architecture. By day, Sam designs complex omni-channel customer and employee service experiences at KFC South Pacific. As the creator and architect of *The Agentic Service Designer* platform, Sam has authored 36 comprehensive, evidence-grounded industry case studies across banking, healthcare, public services, energy, logistics, retail, and hospitality.

Sam's work focuses on the pragmatic transition from static design deliverables (post-it syntheses, PDF journey maps, and presentation decks) to executable, living service systems. His methodologies establish rigorous human-in-the-loop governance across the full autonomy spectrum—from *Suggest* to *Confirm* to *Act*—grounded in cryptographic SHA-256 evidence registers, machine-checked role contracts, and adversarial Critic validation gates.

Core Capabilities & Signature Contributions:

- **Full-Spectrum Service Design:** End-to-end journey mapping, 5-lane canonical service blueprinting, 6th-lane autonomous agent decision blueprinting, mental model alignment, and rapid multi-fidelity prototyping.
- **Agentic AI & Systems Architecture:** DeepSeek Harness Cordis plugin suite, Model Context Protocol (MCP) servers, 10-role Eve-style agent teams, SOPs-as-code pipelines, and deterministic SVG compilation engines.
- **Large-Scale Research Ops:** Transcript mining algorithms, contextual inquiry protocols, telemetry fusion, and evidence-backed behavioral persona segmentation.

Connect with the Author:

GitHub: github.com/sammozaffari

LinkedIn: linkedin.com/in/sam-mozaffari-210588a7

Project Repository: github.com/sammozaffari/the-agentic-service-designer

Written with AI Assistance & Agentic Verification

This manuscript was authored through an advanced multi-agent publishing harness. The author directed, curated, reviewed, and approved all architectural concepts, operational schemas, and analytical narratives. Independent adversarial agents conducted tripartite evaluations across theoretical canon alignment, toolchain integrity, and schema compliance.

Trademarks & Brand Disclaimers

Product names, logos, and frameworks mentioned in this work (including Claude Code, Anthropic, OpenAI, DeepSeek, Google Antigravity, Figma, Miro, Cordis, Node.js, Bun) are the property of their respective owners. Their inclusion is strictly educational and analytical and does not imply official endorsement.

All organizational case studies (Lighthouse Transit, Calder Community Bank, Wattle Grove Community Care, Northgate Health, Fernwood Council, Aurora Vento, Forge Industrial, Sable & Slate) represent composite, anonymized industry practices designed to illustrate technical and organizational mechanics without exposing confidential partner data.

Professional & Regulatory Disclaimer

The information, schemas, scripts, and frameworks contained in this book are provided for educational and organizational development purposes. Service operations involving high-stakes regulatory compliance (healthcare clinical pathways, financial lending, public safety, and critical infrastructure) require human sovereign oversight and certified professional verification prior to production execution.

This Edition

First edition · August 2026 · Version 1.0.0

Version: 1.0.0

Published: August 2026

- **Book repository:** imehr/the-agentic-service-designer
- **Release page:** [Releases \(v1.0.0\)](#)

What's new in this release

No changelog entries recorded for this edition yet.

TABLE OF CONTENTS

Contents

01	The AI-First Service Design Operating System	1	
02	The Agentic Toolchain & Harness Architecture	9	
03	Service-Dominant Logic & Value Co-Creation in the Age of AI	17	
04	The Role Contract, Evidence Registers & Deterministic Fact-Checking	25	
05	Research Ops at Scale: Mining Transcripts & Field Inquiries	37	
06	Mixed Methods & Evidence-Backed Segmentation	45	
07	Mental Models, Alignment & Opportunity Mapping	51	
08	Ecosystem Value Mapping & Stakeholder Topologies	59	
09	Living Journey Maps: From Transcripts to Sentiment Curves	69	...
10	The Master Blueprint: Five-Lane Architecture & Line of Visibility	77	
11	Agentic Service Blueprinting: The Sixth Lane & Autonomous Workflows	87	
12	Cross-Channel Orchestration & Story Mapping	99	
13	Structured Ideation & How Might We at Machine Speed	113	

14	Prototyping Services: Desktop Walkthroughs to Wizard-of-Oz	121	
15	Transformative Services, Behavior Change & Recovery Engineering	131	
16	Service Quality Management & CX Maturity Governance	141	
17	SOPs-as-Code: The AI Execution Playbook	151	
18	Frontline Enablement & Employee Experience (EX)	163	
19	The 15 Principles of Good Public Services	175	
20	The Future of Service Design: The Agentic Practice	185	
	<i>The SDAP Method & Artifact Catalog</i>	189	
	<i>CLI & MCP Tool Specification</i>	193	
	<i>The 20-Book Canonical Cross-Reference</i>	195	

01 The AI-First Service Design Operating System

From sticky notes to living, autonomous service systems

Our discipline faces a severe throughput bottleneck. We have spent the last decade perfecting the workshop, the affinity wall, and the static journey map, only to watch those artifacts rot the moment the engagement ends. It is time to replace workshop theater with an executable operating system. This chapter introduces the Service Design Agent Platform (SDAP), a rigorous, AI-driven framework that translates qualitative design methods into deterministic, machine-checked agent steps. You will learn how to orchestrate a 10-role agent team, map traditional processes to autonomous workflows, and retain human sovereignty where it matters most.

The Death of Workshop Theater

If you have spent any time in corporate innovation labs over the past ten years, you know the drill. A team of designers spends three weeks in the field, conducting two dozen contextual inquiries. They return to the studio, lock themselves in a room, and cover the walls in thousands of neon sticky notes. Executives are brought in for a "synthesis playback" and nod thoughtfully at the beautifully clustered themes. A week later, a 150-page slide deck is delivered, complete with a high-fidelity current-state journey map and a speculative future-state service blueprint.

And then, nothing happens.

This is the tragedy of workshop theater. The canonical design loop—research, ideation, prototyping, implementation—is theoretically sound. The problem lies in its unit economics and its durability. The insights are brilliant, but they are trapped in static formats. The chain of custody from a verbatim user quote to a cluster, to an insight card, to a concept, and finally to a shipped feature breaks down entirely within two weeks. When a developer asks, "Why are we building this specific status tracker?", the answer is locked in a photo of a sticky note on a designer's phone.

NOTE

My take: I have watched million-dollar engagements die on the vine because the traceability was manual. When an executive sponsor pushes back on a costly operational change, you cannot defend it with a vague appeal to "user needs." You need to instantly trace that proposed change back to 14 independent, verbatim quotes of users screaming about the exact same pain point. Traditional methods make this audit impossible.

Because qualitative synthesis demands relentless manual labor, research corpuses are routinely slashed. Twenty-four interviews produce 25 hours of audio. Transcripts stretch into hundreds of pages. No solo practitioner can code and synthesize that volume manually within a standard sprint cadence. Consequently, engagements silently cut their sample size to eight interviews, trading generative depth for timeline compliance. We end up designing services based on anecdotes rather than rigorous, triangulated data.

The "fake iteration" trap inevitably follows. Because the first loop took so long and cost so much, there is no budget for a second loop. Service design devolves into a linear, waterfall delivery plan disguised by human-centered vocabulary. Institutional amnesia takes hold; each new project starts from scratch, relying on the heroic efforts of individual practitioners rather than repeatable, systemic capabilities.

TIP

CORE PRINCIPLE: Service architecture must evolve from a craft of producing static deliverables to a discipline of engineering living operating systems. Artifacts must become executable code.

The AI-First Service Design Operating System (SD OS) was built to destroy this paradigm. By encoding every method in our practice as a strict, machine-executable Standard Operating Procedure (SOP), we dramatically alter the unit economics of the discipline. We stop throwing away our evidence. We stop guessing. We replace workshop theater with an unbroken, deterministic chain of verifiable facts.

The Service Design OS Architecture

The SD OS is not a chatbot wrapper. It is a highly structured, four-layer architecture designed to separate data generation from deterministic rendering and human governance. It operates on a fundamental premise: agents do not chat to solve problems; they write typed artifacts to a central, version-controlled store.

Figure 1.1 The Service Design OS Architecture. An agentic service practice replaces isolated handoffs with a continuous, verified feedback loop.

At the foundation sits the **Evidence Store**, implemented as a plain GitHub repository. This is the single source of truth. Every transcript, every quote, every survey response, and every telemetry log is converted into a content-addressed SHA-256 evidence row. This means that every factual claim made in any downstream artifact—be it an insight card or a journey map—must contain a cryptographic pointer back to its exact source. If a designer or an agent attempts to make a claim that cannot be traced to the evidence store, the system mechanically rejects it.

Above the store is the **Agent Production Layer**. This is where the heavy lifting of parsing, coding, clustering, and ideating occurs. The agents operate under strict SOPs-as-code. They do not freestyle. They pull from the evidence store, execute their highly constrained method, and write their output back as JSON or Markdown artifacts.

The third layer is the **Deterministic Fact-Check and Rendering Pipeline**. Large Language Models (LLMs) are notoriously bad at producing visually perfect diagrams or flawless code structures consistently. Therefore, we strip them of that responsibility. The agents only emit structured data (JSON). A deterministic compiler then takes that JSON, runs it through schema validation, and mechanically renders it into SVGs, Mermaid charts, and HTML views. If an agent hallucinates a lane in a service blueprint, the JSON schema validation fails loudly, blocking the pull request via GitHub Actions. We enforce referential integrity rigorously: evidence register &arr; artifact write &arr; acceptance rules &arr; critic verdict &arr; persist &arr; factcheck report.

Finally, at the top, sits the **Human Governance Layer**. Practitioners do not move sticky notes in isolation; they stand at the helm managing phase gates, making ethical judgments, and presenting verified findings to the client. The designer orchestrates the system, abductively reframes the brief, and negotiates the political landscape.

sop-as-code-template.yaml

```
sop_id: SD-DEF-002
method: "Collaborative affinity synthesis and insight formulation"
inputs:
  - fieldwork/coded/*.json
steps:
  1_cluster: "Group coded utterances by semantic similarity"
  2_name: "Name each cluster with an active sentence"
  3_formulate: "Emit insight cards: observation + tension + implication"
output_schema: schemas/insight-card.json
acceptance_criteria:
  - "Every evidence quote exists verbatim in fieldwork/coded/"
```

```
reviewer: critic-agent
  then_human: "Designer facilitates synthesis playback"
```

Mapping the 4D Process to the TISDD Loop and Kumar's 7 Modes

To make an AI-first system understandable and palatable to traditional organizations, we wrap its operations in the familiar Double Diamond (Discover, Define, Develop, Deliver) process. However, beneath the surface, the SD OS runs a much more aggressive, non-linear engine.

Traditional practice often treats the "This Is Service Design Doing" (TISDD) activities—Research, Ideation, Prototyping, Implementation—as a single sequential marathon. In the SD OS, these activities loop continuously within every phase. We do not wait until the Develop phase to start prototyping; we use rapid, agent-generated Wizard of Oz scaffolding to test early assumptions during Define. The machine speed of our agent team allows us to iterate three or four times in the time it traditionally takes to complete one cycle.

We also align each phase directly with Vijay Kumar's Seven Modes of the Design Innovation Process. This gives our agent roles precise cognitive boundaries for their tasks:

- **Sense Intent (Mode 1):** This is the continuous strategic frame held by the Orchestrator and the Human Designer. It is provisional and revised as the environment pushes back.
- **Know Context & Know People (Modes 2 & 3):** Executed during Discover by the Researcher agent, combining ethnographic transcripts with deep Codex-driven context scans.
- **Frame Insights (Mode 4):** The Synthesizer agent's domain during Define, moving from raw data to structural tensions and actionable insight cards.
- **Explore Concepts (Mode 5):** The Ideator agent in the Develop phase, generating hundreds of HMW (How Might We) permutations and scoring them ruthlessly.
- **Frame Solutions & Realize Offerings (Modes 6 & 7):** The Mapper, Prototyper, and Measurer roles in the Deliver phase, turning concepts into future-state blueprints and actionable, ticketed backlogs.

NOTE

My take: The most profound shift is in Mode 1. In traditional design, the brief is sacred. In an agentic system, because research and synthesis happen in days rather than months, you can afford to prove the initial brief wrong almost immediately. The ability to pivot the entire engagement's strategic intent based on 2,000 lines of mathematically clustered evidence in week two is a superpower.

The return edge from Deliver back to Discover is where the system truly becomes an operating system. Pilot metrics and live telemetry are not just reported; they are ingested directly back into the evidence store as new rows, triggering the Orchestrator to kick off a new loop automatically if

KPIs degrade.

The 10-Role Eve-Style Agent Topology & Case Studies

The Service Design Agent Platform (SDAP) utilizes an Eve-style filesystem-first agent team. Initially conceived as six core roles, the sheer complexity of continuous service orchestration expanded the topology to 10 highly specialized, distinct personas. These are not general-purpose chat windows. Each role is a dedicated system prompt bound to specific tools, executing precise SOPs, and communicating exclusively through committed JSON artifacts in the Git repository.

Figure 1.2 The 10-Role Agent Team Topology. Specialized roles with explicit boundaries prevent agent drift and preserve investigative depth.

Here is the exact division of responsibilities across the team, demonstrated through real-world applications:

1. **Orchestrator (O):** The traffic controller. It manages phase gates, dispatches the other roles, and synthesizes gate reports for the lead practitioner. It never creates design artifacts itself. In the Forge Industrial engagement, the Orchestrator managed the complex branching logic between field technicians generating telemetry and backstage pricing teams validating data.
2. **Researcher (R):** The data gatherer. It executes ethnographic processing, transcribing audio, pseudonymizing PII, and open-coding raw data into the evidence store. It also runs deep web scans for secondary context. During the Calder Community Bank project, the Researcher processed 24 full-length transcripts and 214 survey responses in less than 48 hours, a task that would traditionally take a solo designer three weeks.
3. **Librarian (L):** The archivist. It maintains method provenance, looking up canonical rules from the reference library, ensuring no agent drifts from strict methodological definitions (like the exact 5 lanes of a blueprint). The Librarian ensures that when the Mapper creates a blueprint, it adheres perfectly to standard industry schema.
4. **Synthesizer (S):** The sense-maker. It owns the data-to-insight ladder. It clusters thousands of coded utterances into themes, drafting JSON insight cards that strictly link back to verbatim evidence. For Fernwood Council, the Synthesizer pinpointed the exact tension causing failure demand: tenants called 3,100 times each month because they lacked digital confirmation of their repair tickets, rather than slow staff response.
5. **Mapper (M):** The cartographer. It drafts current-state and future-state journey maps and service blueprints. It is mathematically barred from adding touchpoints that are not supported by accepted insight cards. It mapped the cascading breakdown of passenger trust for a regional airline by linking touchpoints directly to the Synthesizer's emotional severity scores.
6. **Ideator (I):** The divergent engine. It runs HMW (How Might We) generation and morphological matrices, creating massive catalogs of concepts and immediately scoring them against the NUF

(New, Useful, Feasible) framework. At Calder Bank, the Ideator generated 118 distinct concepts to solve onboarding abandonment, preventing the team from settling on the first obvious idea.

7. **Prototyper (P2):** The builder. It translates concept catalogs into test plans, assembling Wizard of Oz scaffolding, and generating highly specific mock data mapped to behavioral personas. When Fernwood Council needed to test an SMS notification system, the Prototyper generated persona-specific mock repair data that allowed a human "Wizard" to manually test the service flow with real tenants for three weeks before a single line of production code was written.
8. **Measurer (ME):** The auditor. It sits in the live pilot, tracking KPIs, triangulating telemetry against leading/lagging indicators, and auditing operational performance data. It proved that removing Average Handling Time as a metric for frontline agents actually reduced total call volume by resolving root issues.
9. **Presenter (PR):** The communicator. It translates complex JSON and Markdown into executive-friendly pilot decision memos, weekly reports, and workshop playback materials. The Presenter ensures that stakeholders receive crisp, evidence-backed narratives rather than raw data dumps.
10. **Critic (C):** The adversarial conscience. The most critical role in the system. The Critic sits on the write path. Every time another agent drafts an artifact, the Critic reviews it against acceptance criteria. It checks for orphaned quotes, invented statistics, and logical leaps. If it finds a defect, it issues a `review-veto-log`, demoting the artifact to "draft" status and forcing the originating agent to remediate it before human review.

To understand how this topology works in concert, consider the **Calder Community Bank** case study. The bank faced a 41% digital onboarding abandonment rate, and leadership's immediate brief was simply, "shorten the form." The Orchestrator dispatched the Researcher to code 18 interviews and the Synthesizer to cluster them. The Synthesizer produced 14 insight cards. The Critic verified that the central insight—abandonment wasn't due to length, but due to anxiety over whether identity verification photos succeeded—was supported by 11 independent quotes. The Ideator generated 64 concepts, and the Prototyper built a mock overlay for live-capture guidance. The result was a 15% increase in onboarding completion without shortening the form at all. The 10-role topology executed the entire loop in 14 weeks, saving 252 designer-hours compared to the traditional baseline.

TIP

CORE PRINCIPLE: The Law of Artifacts: Handoffs are never chat. Agents never talk directly to one another in free-form text. Every handoff is a strictly typed JSON file, committed to the repository, passing through the Critic's veto gate.

The Autonomy Spectrum: When Agents Lead and When Humans Decide

The fear of deploying an "AI-First" operating system is the loss of control—the nightmare of a rogue agent hallucinating a deeply flawed operational process and shipping it to frontline staff. SDAP prevents this through a rigid autonomy spectrum that clearly delineates where machine speed is unleashed and where human sovereignty is absolute.

Agents are incredibly good at massive divergent processing and deterministic pattern matching. They are catastrophic at reading a room, negotiating political capital, and making ethical tradeoffs. Therefore, we design the workflow to leverage both strengths.

Figure 1.3 The Autonomy Spectrum. Autonomous discovery accelerates research, but phase gates remain sovereign human decisions.

Where Agents Lead:

- **Corpus Processing:** Reading 400 pages of transcripts, redacting PII, and tagging 2,000 utterances against a codebook. This takes an agent minutes and a human weeks.
- **Divergent Ideation:** Generating 118 distinct service concepts from a single insight, enforcing structural variety, and aggressively filtering out the generic slop.
- **Blueprint Translation:** Walking a completed service blueprint column-by-column and emitting 74 perfectly linked Jira tickets for the development backlog.
- **Adversarial Fact-Checking:** The Critic agents checking every single numeric claim in a memo against a source CSV, operating tirelessly without fatigue.

NOTE

My take: The Critic agent provides the vital quality backstop across our runtime, as the Forge Industrial case study demonstrates. In this B2B engagement, we needed a deep research scan of field-service SLA benchmarks. The Codex-powered Researcher agent confidently returned a beautifully formatted report containing three invented statistics. They sounded perfectly plausible and were phrased with absolute certainty. However, the Critic agent flagged all three immediately because they violated the strict sourcing contract: every numeric claim must carry a verifiable URL and a verbatim quote. The Critic issued a `review-veto-log`, demoting the artifact and forcing a rewrite. Without the Critic catching this, those fabricated stats would have been presented to the client's executive board. Trust would have been broken instantly, and the entire engagement would have been terminated on the spot. Human oversight is fallible when reviewing hundreds of pages; mechanical vetoes are not.

Where Humans Decide:

- **Ethnographic Presence:** A human practitioner conducts field interviews. Agents cannot read body language, they cannot build rapport, and they cannot ethically manage informed consent in vulnerable environments.
- **Abductive Reframing:** When the data suggests the client's initial brief is wrong, only a human has the contextual awareness and social capital to reframe the problem with the sponsor.
- **Naming the Tension:** The machine clusters the quotes, but the human designer names the underlying tension. It takes a human to look at a cluster about "form abandonment" and accurately name it "Fear of bureaucratic reprisal."
- **Phase Gates and Kill Decisions:** The Orchestrator prepares the gate report, but the human owner signs off. Killing an executive sponsor's pet project because it failed the Wizard of Oz test requires human diplomacy and courage.

When these boundaries are respected, the failure modes of autonomous systems in this discipline are heavily mitigated. If an Ideator agent hallucinates an impossible technical feature, the Prototyper will fail to build a Wizard of Oz test for it, or the Measurer will fail to find metrics to validate it. Even if it makes it past the agents, the human phase gate catches it before it touches the client's operations. The lead designer acts as the ultimate filter for political feasibility, institutional readiness, and ethical alignment. For instance, in the Fernwood Council project, the agent proposed an automated decision engine for housing repairs. The lead designer intervened and killed the concept at Gate 3, recognizing that delegating housing decisions to an algorithm without a human appeals process violated municipal ethical standards.

By enforcing this division of labor, the AI-First Service Design Operating System does not replace the designer. It elevates them. It strips away the grueling, repetitive manual labor that historically diluted our focus, allowing the practitioner to spend their time entirely in the realm of high-leverage judgment, strategic alignment, and actual human empathy. The sticky notes are gone, but the service design has never been more rigorously alive.

02 The Agentic Toolchain & Harness Architecture

DeepSeek Harness, the @asd runtime, MCP, and terminal ergonomics

The transition from performative post-it notes to living service systems doesn't happen through prompts alone. It demands rigorous, deterministic software architecture. In this chapter, I will break down the Service Design Agent Platform (SDAP) runtime stack. We'll explore how our runtime exposes 52 native tools, how it wires directly into the DSH environment via the Cordis plugin ecosystem, and how we expose identical capabilities to external clients via the `asd-mcp` server and the `asd` CLI.

The SDAP Runtime Architecture

For years, design teams relied on static diagrams decoupled from their implementation. The SDAP architecture completely inverts this legacy pattern. We run on a highly structured, layered toolchain designed to enforce invariants, compile valid state from markdown files, and ruthlessly separate generative intelligence from deterministic rendering.

The core runtime, housed under `@asd/engine`, specifies capabilities. It provides 52 concrete tools, ranging from low-level evidence tracking (`sd_evidence_add`) to sophisticated method automation (`sd_journey_map`). The key innovation here is that this runtime layer is purely functional and deeply decoupled from any single presentation layer. It takes typed JSON, validates it against schemas, manipulates an immutable file-backed workspace, and returns structured data.

NOTE

My take: I have seen too many "AI platforms" that tightly couple the LLM prompt to the web UI. By extracting the capability layer into a pure execution engine, SDAP allows me to run a massive batch of artifact generations entirely headlessly on a CI server before I ever open a visual harness. The architecture forces hygiene.

We expose this core engine through three distinct deployment profiles: a native DSH plugin bundle, a standalone command-line interface (CLI), and a Model Context Protocol (MCP) server. Each of these surfaces maps strictly to the exact same tool definitions. Whether the model runs in a

browser session, inside your terminal, or through Claude Code, it is executing the identical codebase.

Figure 2.1 Layered Architecture. One unified engine powers terminal CLIs, MCP servers, and full GUI harnesses identically.

DeepSeek Harness (dsh) and Cordis Plugin Wiring

Let's look at how we integrate with DeepSeek Harness (`dsh`). DeepSeek Harness (`dsh`) provides a modular runtime environment built on the premise that *everything is a plugin*, powered by the Cordis plugin system. An app is essentially a profile—a bundle of officially and third-party maintained plugins working in concert. We didn't build a monolithic web application; we built a Cordis plugin.

Our package installs directly into the `web` profile. We link the local path and patch the composition config. When the harness boots up, our host plugin's `apply()` method registers our complete toolchain into the harness surfaces.

```
dsh profile installation log
```

```
# how it was installed
dsh plugin --profile web add @asd/agentice-service-designer

# the composition now contains the ASD plugin row
dsh --profile web --dump-config
# = @asd/agentice-service-designer
- id: asd
  name: '@asd/agentice-service-designer'
  config:
    workspaceRoot: ~/.asd/workspace
    defaultPhase: discover
    booksDir: ''
    timeoutMs: 30000
    maxWorkflowAgents: 4
```

We register 52 tools sequentially via `ctx.tools.register(defineTool(...))`. This is the core integration vector: the moment the tool is registered, any model operating in the session can call it. Next, we inject our persona via `ctx.systemPrompt.section(...)`, instructing the LLM to adopt the senior-service-designer identity. We also register a settings namespace, persisting to `~/.dsh/settings.yaml`, which instantly surfaces a configurable plugin card in the DeepSeek Settings UI.

We also re-enable the background workflow engine. ASD relies heavily on multi-agent pipelines for tasks like transcript-analysis and blueprint-build. By wiring subagent role prompts (researcher, architect, reviewer, presenter) into the `workflowEngine` context, we allow a single human prompt to trigger an asynchronous swarm that writes directly to the shared session log.

Figure 2.2 DeepSeek Harness Settings. Plugin configuration is dynamically reflected in harness session state.

The @asd Suite Workspace and Tool Registry

A toolchain is only as reliable as its data layer. Every project initialized by SDAP writes to a deterministic file structure under the active workspace root (typically `~/.asd/workspace/`). It is entirely portable, readable markdown and JSON.

When the agent executes a command—say, `asd_project_create`—the engine generates the directory, writes the metadata payload into `.asd/project.json`, and initializes an `.asd/artifact.json` ledger. As the session progresses, any artifacts generated—like a journey map—are written out alongside their structured inputs. For example, a journey map is stored with a markdown header wrapping a dynamically compiled Mermaid flowchart.

TIP

CORE PRINCIPLE: Treat the workspace as an immutable ledger. Never hand-edit the raw JSON files unless you are deliberately running recovery scenarios; let the toolchain enforce schema and relationships.

The asd-mcp Server: Tools, Resources, and Prompts

Beyond our primary DSH environment, I needed SDAP to be accessible to standard developer tools—specifically Claude Code, Cursor, and Windsurf. To bridge this, I built `asd-mcp`, a minimal, dependency-free MCP (Model Context Protocol) server running over stdio.

The beauty of `asd-mcp` is that it doesn't reinvent the wheel. It imports the exact same `@asd/engine` runtime. In response to a `tools/list` JSON-RPC request, it maps native JSON Schema tool parameters directly to the MCP `inputSchema` format. When a client fires `tools/call`, the server passes arguments to the core runtime and returns the response block.

But the real power of MCP is exposing the SDAP knowledge canon. I mapped 215 internal resources—design books, framework articles, methods, and templates—behind the `asd://` URI scheme. When a client requests `resources/list`, the server responds with a paginated array. Because an MCP client might choke on 215 items at once, I implemented rigorous cursor pagination. The cursor is an opaque base64url string (e.g., `offset:100`). The server decodes it,

slices the dataset by a fixed `PAGE_SIZE = 100`, and provides a `nextCursor` if more data remains. If a user queries `resources/read` with an `asd://` URI, the server streams back the raw markdown text.

mcp.ts cursor decoding

```
function decodeCursor(raw: unknown): number | undefined {
  if (raw === undefined) return 0
  if (typeof raw !== 'string') return undefined
  const decoded = Buffer.from(raw, 'base64url').toString('utf8')
  if (!decoded.startsWith('offset:')) return undefined
  const n = Number(decoded.slice('offset:'.length))
  return Number.isInteger(n) && n ≥ 0 ? n : undefined
}
```

NOTE

My take: Writing the MCP server entirely by hand with zero npm dependencies was a deliberate, hard choice. It guarantees that the server boots instantly when a subagent spawns in Claude Code, immune to `node_modules` rot or heavy transient overhead. Fast context resolution is the lifeblood of agentic workflows.

Similarly, the ten agent roles from our Eve-style topology (orchestrator, researcher, synthesizer, mapper, etc.) are surfaced via `prompts/list` and `prompts/get`. If Claude needs to delegate synthesis to a subagent, it fetches the synthesizer prompt directly through the MCP interface, ensuring the behavior is aligned across all environments.

CLI Ergonomics and Deterministic SVG/HTML Rendering

Sometimes, interacting through a chat UI is just too slow. I often want to script batch extractions or trigger pipeline audits directly from my shell. That's where the `asd` CLI comes in. It serves as a direct binding to the engine.

We expose 16 primary command verbs, including `project-create`, `evidence-add`, `artifact-write`, and the all-important `factcheck`. The CLI argument parser dynamically reads the JSON-schema attached to the tool definition. It expects `--flag value` syntax, elegantly handling booleans (`--inline`) and complex JSON blobs (`--json '{"key": "value"}'`). This guarantees that the CLI is never out of sync with the MCP server or the DSH plugin.

CLI usage examples

```
# Create a new service design project footprint
asd project-create --name "Housing Repairs" --client "Fernwood Council" --sector
```

```
# Append telemetry evidence to the workspace register
asd evidence-add --projectId <id> --source "int-04" --kind quote --body "I had to
call three times"

# Force a deterministic audit of all facts in the project
asd factcheck --projectId <id>
```

This brings us to the most critical architectural decision in the platform: the absolute separation of generative intelligence from rendering. I explicitly forbid the LLM from writing Mermaid markup, SVG paths, or raw HTML manually. LLMs are magnificent at clustering concepts, synthesizing transcripts, and deriving structured insights, but they are notoriously flaky at generating perfectly balanced, syntax-error-free diagram code.

Instead, models invoke `sd_artifact_write` with rigid, schema-typed JSON payloads defining nodes, edges, and phases. Our renderer validates that structure. It compiles exact Mermaid diagrams and HTML views mechanically. Malformed payloads reject immediately. Valid ones render cleanly every time.

Figure 2.3 The Rendering Pipeline. LLMs must write structured data, while deterministic compilers produce guaranteed error-free diagrams.

This deterministic hand-off forms the structural bedrock of our runtime. By forcing agents to write structured data, we strip away presentation hallucination. When running `asd factcheck` inside CI pipelines, the CLI leverages these strict schemas. If an artifact's structural integrity or acceptance rules fail, the CLI returns a `1` exit code. It is entirely possible, and expected, to wire `asd factcheck` into your Git hooks or CI pipelines to block failing designs from ever reaching the stakeholder presentation.

NOTE

My take: The realization that a service design deliverable could break a CI build was profound for my team. It forces designers to think like engineers. You cannot pass the gate with an unresolved dependency or a "TBD" placeholder. The pipeline won't let you.

The SDAP architecture demonstrates that AI-first design doesn't mean replacing tools with magic prompt text boxes. It means embedding LLMs as tightly constrained logic components within a strictly enforced, mathematically rigorous execution environment. From DeepSeek to your terminal, the rules remain the same.

Deep Dive: The 52-Tool Registry and JSON-RPC Protocol

To understand how DSH and the MCP server operate concurrently without state collision, we must inspect the inner structure of the `@asd/engine` tool registry. The engine exposes 52 specialized tools grouped into 8 functional clusters: Project Management, Evidence Ledger, Qualitative Synthesis, Topological Mapping, Divergent Ideation, Prototyping Simulations, Quantitative Metrics, and Adversarial Verification.

Every tool call follows a rigid JSON-RPC 2.0 interface. The agent never issues ad-hoc SQL or unstructured shell scripts; it passes strongly typed parameter payloads validated against JSON Schema Draft-07 contracts before execution:

`rpc-tool-invocation.json`

```
{
  "jsonrpc": "2.0",
  "method": "sd_evidence_add",
  "params": {
    "project": "calder-sme-onboarding",
    "source": "interview-transcript-sme-042.txt",
    "kind": "quote",
    "body": "We waited fourteen business days before anyone told us our
certificate of incorporation was rejected because of a missing apostrophe.",
    "context": {
      "speaker": "Managing Director, Precision Tooling Ltd",
      "timestamp_offset": 842.5,
      "journey_stage": "document-verification"
    }
  },
  "id": "req-849204"
}
```

When the engine receives this invocation, it executes a three-stage verification pipeline:

1. **Syntactic Schema Check:** Asserts that `kind` is strictly one of `["quote", "telemetry", "document"]`. Any unapproved kind (such as `observation` or `inference`) is rejected immediately with code `-32602`.
2. **Cryptographic Ingestion:** Computes the SHA-256 digest of `(source + ":" + kind + ":" + body)` and checks for collision or duplication in the append-only SQLite/JSON register.
3. **Referential Linking:** Returns an immutable evidence ID (e.g., `ev-8492-a1f9`) that subsequent journey maps and service blueprints must cite verbatim.

NOTE

My take: Many developers believe that giving an LLM full bash access is the apex of agentic power. In high-stakes enterprise service design, raw bash is an unmitigated liability. By forcing all operations through strongly typed JSON-RPC tools with immutable evidence registers, we ensure that an agent cannot hallucinate a client interview, forge a telemetry data point, or overwrite an approved service blueprint.

The Deterministic SVG Compiler Architecture

One of the most persistent failure modes in early generative AI service design was diagram syntax corruption. When asked to write raw Mermaid, Graphviz DOT, or PlantUML diagrams, models frequently produced dangling arrows, unescaped quotes, illegal node identifiers, or broken subgraphs. In the SDAP architecture, models are strictly prohibited from generating rendering syntax.

The compiler architecture in `@asd/render` enforces a complete decoupling between data structure and visual presentation:

`render-pipeline-flow.ts`

```
import * as fs from "node:fs";
import { validateArtifact, renderBlueprintToSvg, renderJourneyMapToSvg } from
"@asd/render";

export async function compileVisualArtifact(artifactPath: string): Promise<string>
{
  const rawData = await fs.promises.readFile(artifactPath, "utf-8");
  const artifact = JSON.parse(rawData);

  // Step 1: Validate against strict artifact schema
  const validationResult = validateArtifact(artifact);
  if (!validationResult.valid) {
    throw new Error(`Artifact validation failed: ${validationResult.errors.join(",
    ")} `);
  }

  // Step 2: Deterministic SVG Compilation
  switch (artifact.kind) {
    case "service-blueprint":
      return renderBlueprintToSvg(artifact, {
        theme: "midnight-blue",
        showLineOfVisibility: true,
```

```
});  
  case "customer-journey-map":  
    return renderJourneyMapToSvg(artifact, {  
      renderEmotionCurve: true,  
      clusterPainPoints: true  
    });  
  default:  
    throw new Error(`Unsupported visual artifact kind: ${artifact.kind}`);  
}  
}
```

This deterministic compilation guarantees that every service blueprint, journey map, and stakeholder topology renders with pixel-perfect typographic alignment, standard accessibility contrast ratios, and 100% syntactic validity across all target publication formats.

03 Service-Dominant Logic & Value Co-Creation in the Age of AI

Foundational theory for autonomous service ecosystems

For decades, the software industry has labored under a delusion: that we build products, ship them to users, and value is magically transferred upon delivery. This goods-dominant mindset has infected how we build AI. We treat large language models as automated data factories—vending machines for text and code. But value does not exist inside a JSON payload or a rendered SVG. Value is only ever realized when a human beneficiary integrates that output into their own context. In this chapter, I translate the heavy academic theory of Service-Dominant Logic (SDL), the Service Profit Chain, and the Experience Economy into ruthless, machine-executable design principles for autonomous agent swarms.

Why Modern AI Practitioners Must Understand Service-Dominant Logic

If you build AI agents without understanding Service-Dominant Logic, you are destined to build very fast, very cheap commodity generators that nobody actually wants. Vargo and Lusch's foundational paper on Service-Dominant Logic broke the industrial-era assumption that value is embedded in goods during manufacturing. Instead, they argued that service—the application of specialized competences (knowledge and skills) for the benefit of another entity—is the fundamental basis of all exchange.

When I look at the current landscape of AI tools, I see a catastrophic misunderstanding of this premise. Companies are building AI features as if they were goods. They measure "generations per second" or "tokens processed" as if these were units of value. But a token has no intrinsic worth. A generated report has no intrinsic worth. The value is exclusively determined by the beneficiary in the moment of use. If a generated journey map does not change the mind of a product manager, its value is zero, regardless of how many trillion parameters the model used to create it.

To design an agentic service is to design an ecosystem of resource integration. When a user interacts with the Service Design Agent Platform (SDAP), they are not "using a tool." They are engaging in a joint cognitive process. The agents bring market resources (LLM reasoning, structured methods, deterministic rendering). The human brings private resources (institutional context, empathy, strategic intent). The service exists only in the friction and flow of their interaction.

TIP

CORE PRINCIPLE: Value is always co-created in use. You cannot "deliver" value to a customer via an API response. You can only offer a value proposition and facilitate their value creation process.

Let us look at the foundational premises of SDL and how they map to our architecture. Foundational Premise 1 (FP1) states that service is the fundamental basis of exchange. We exchange applied capabilities, not artifacts. FP2 warns that indirect exchange masks the fundamental basis of exchange. We get distracted by the JSON schemas and forget that they are just conduits for cognitive labor. FP3 argues that goods are distribution mechanisms for service provision. The CLI, the MCP server, the DeepSeek Harness—these are just distribution vectors for the real service: decision intelligence. FP6 insists that value is always co-created by multiple actors. The AI cannot do it alone. FP10 is the ultimate reality check: value is always uniquely and phenomenologically determined by the beneficiary.

If we accept these premises, we must completely restructure how our agents behave. They cannot simply take a prompt and throw an artifact over the wall. They must negotiate, clarify, draft, and await human integration. They must ask for the private resources they lack. They must respect the institutional boundaries of the user. This is not a philosophical abstraction. It is a technical requirement for any multi-agent system that expects to operate in production environments.

Operant vs. Operand Resources: Agents as Active Co-Creators

In traditional economics, resources are static things to be acted upon—operand resources. Coal, iron, and database records are operand resources. Operant resources, by contrast, are the active agents of change. They are the skills, knowledge, and capabilities that act upon operand resources to create value. In traditional software, the human is the only operant resource, and the software is the operand resource.

The paradigm shift of agentic service design is that the software itself has become an operant resource. A DeepSeek Harness agent running the `asd` CLI does not wait to be queried like a SQL database. It proactively reads transcripts, synthesizes themes, questions contradictions, and authors blueprints. It acts upon the data.

Figure 3.1 Value Co-Creation Matrix. Value is not embedded in software outputs; it is realized only during collaborative customer use.

I saw the difference this makes during a grueling 20-week engagement with Forge Industrial, a pan-European manufacturer of heavy industrial equipment. Forge wanted to launch a "Predictive Care" service. Their initial instinct, driven by goods-dominant logic, was to install more sensors on

their machines, build a massive telemetry dashboard, and sell the data feed to plant managers. They were treating the dashboard as an operand resource. It was a classic trap: trying to sell data as a good instead of facilitating a service.

We used SDL to completely re-architect the offering. The plant manager didn't want data; they wanted uninterrupted production windows. The data was useless without the operand resources to act on it. So we redesigned the service as an Outcome Partnership. We didn't just ship a dashboard. We shipped a joint operating model. We defined a shared weekly risk queue, a decision-rights matrix, and an explicit division of labor between the Forge remote diagnostician and the plant's reliability engineer. The agents in our TISDD loop didn't just summarize downtime logs; they actively drafted the intervention playbooks and contract boundaries.

Logic Paradigm	Forge's Initial Concept	Agentic SDL Redesign
Primary Exchange	Telemetry data and alert dashboards (Goods)	Joint outcome partnership for uptime (Service)
Resource Type	Operand (Data to be acted upon)	Operand (Shared rituals and predictive actions)
Role of Customer	Passive receiver of alerts	Active co-creator in planning interventions
Value Determination	Measured by volume of alerts delivered	Phenomenologically determined by stable production

The result? The SKAUTO42 pilot site hit 97.6% uptime and reduced emergency freight incidents by 66%. This wasn't achieved by a smarter algorithm in isolation. It was achieved because the service design explicitly structured the co-creation between Forge's backstage facilitation and the customer's frontstage reality. We enforced SDL at every touchpoint. We explicitly banned contract language that claimed Forge could "unilaterally deliver" uptime. Forge could only promise to act on the data; the plant had to approve the downtime window for the fix. Value was co-created, or it didn't happen at all.

NOTE

My take: The biggest mistake I see AI startups make is assuming that a "smarter model" equals a better product. I have watched brilliant engineering teams burn millions building autonomous systems that customers refuse to adopt. Why? Because the system tries to unilaterally deliver a result instead of inviting the user into a co-creation ritual. If you don't design the human's role in the value integration, your agent is just shouting into the void.

The Service Profit Chain: Internal Quality Drives External Value

If SDL explains how value is co-created with the customer, James Heskett's Service Profit Chain (SPC) explains the mechanics of how a firm sustains that capability. The SPC asserts a mathematical chain of causality: internal service quality drives employee satisfaction, which drives employee productivity and retention, which drives external service value, which drives customer satisfaction, which drives customer loyalty, which ultimately drives profit.

In the context of agentic services, "internal service quality" takes on a radical new meaning. Historically, internal quality meant decent break rooms, fair pay, and functioning laptops. Today, it means the cognitive leverage provided by autonomous agent swarms. When you equip a frontline service worker—whether it's a customer support rep or a field technician—with an Eve-style agent team, you fundamentally alter their cognitive load.

Figure 3.2 The Service Profit Chain. Frontline tooling quality is the root driver of end-customer satisfaction.

When an agent absorbs the rote work of synthesizing transcripts or generating blueprint scaffolds, the human practitioner is freed to focus on high-judgment, high-empathy tasks. This is not just a productivity gain; it is a profound improvement in job satisfaction. No service designer enjoys copying and pasting verbatim quotes from 200 transcripts into an Excel sheet. By delegating that to a deterministic script running in DeepSeek Harness, we elevate the human to the role of a strategic director.

Service Profit Chain Link	Traditional Metric	Agentic Service Metric
Internal Quality	Software uptime, training hours	Agent success rate, cognitive load reduction
Employee Satisfaction	eNPS, turnover rate	Time spent on high-judgment vs rote tasks
External Value	SLA compliance, resolution time	Evidence-backed design speed, rigorous traceability
Customer Loyalty	NPS, renewal rate	Co-creation engagement, outcome achievement

We saw this causal chain light up during the Forge Industrial rollout. The internal teams were initially overwhelmed by the complexity of the outcome contracts. By deploying the `asd-mcp` server to assist contract managers, we reduced the time to draft a compliance-checked outcome schedule from 14 days to 3 hours. The contract managers stopped feeling like administrators and started acting like strategic advisors. That internal satisfaction bled directly into the external value perceived by the plant managers, driving the share of wallet from 27% to 34%.

The SPC forces us to confront a brutal truth about automation: if you use AI solely to cut headcount, you destroy internal service quality. The remaining employees are stressed, paranoid, and unsupported. External service value plummets. Loyalty collapses. Profit follows. Agentic tooling must be deployed to amplify the frontline, not strip-mine it. The leverage comes from making your humans cognitively superhuman, allowing them to stage experiences that simple software cannot replicate.

The Four Realms of Experience Staging (Pine & Gilmore)

If we are to escape the commodity trap of generative AI, we must look to Pine & Gilmore's Experience Economy. They argue that economic value progresses from commodities to goods, to services, to experiences, and finally to transformations. An experience occurs when a company intentionally uses services as the stage, and goods as props, to engage individual customers in a way that creates a memorable event.

Pine & Gilmore define four realms of experience across two axes: passive vs. active participation, and absorption vs. immersion. In the agentic era, we must deliberately design which realm our service inhabits at any given moment.

Figure 3.3 The Four Realms of Experience. Agentic services elevate routine transactions into transformative experiences.

I applied this exact framework when I redesigned the member lifecycle for Meridian Clubs, a premium fitness operator trapped in a commodity business. They were selling access to a room full of machines (goods-dominant logic). Members churned silently because the service was flat and anonymous. We redesigned the entire architecture to stage a 12-month transformation arc. We didn't just change the app; we scripted the physical spaces and the coach interactions. We created the day-1 Member Briefing and the 90-day Transformation Checkpoint.

Meridian was operating in the Esthetic and Educational realms, but doing neither well. It was passive. It lacked narrative. By leaning into transformation staging, we pushed the service into the Escapist realm (active immersion). Members were no longer renting treadmills; they were embarking on a hero's journey where the club served as mentor. We authored explicit role scripts for the frontline coaches. We codified Work-is-Theatre principles directly into the operational playbooks, mapping out the frontstage set, costume expectations, and mandatory scripted acts.

```
transformation-checkpoint.yaml
```

```
# Standard Operating Procedure for Transformation Staging
sop: 90-day-transformation-checkpoint
trigger: member.journey_day == 90
steps:
```

```

owner: agent_assistant
  inputs: [attendance_log, baseline_screen, quest_progress]
  output: checkpoint_dossier
  sla: day 87
- id: perform
  owner: human_coach
  duration_min: 45
  script: scripts/checkpoint.md
  emotional_target: 4
  outputs: [progress_story, next_quest, verified_pbs]
- id: record
  owner: agent_assistant
  system: member_app
  writes: [journey_record.checkpoint, loyalty_rate_eligibility]
acceptance:
  - checkpoint_redemption_pct ≥ 60
  - nps_of_checkpoint_attendees ≥ cohort_nps
guardrails: [no_body_shaming_lexicon, medical_disclosure_pauses_sop]

```

Notice what is happening in the code block above. The AI agent acts in the backstage (preparing the dossier, recording the writes) while the human coach commands the frontstage (performing the scripted checkpoint). The AI is an operant resource facilitating an active, immersive experience for the member. This is how you orchestrate the Escapist and Educational realms simultaneously. The result at Meridian was a 3.2 percentage point jump in 12-month retention and an NPS spike from 24 to 45.

If you fail to stage the experience, you fall back to service delivery. If you fail at service delivery, you fall back to goods. And if you fail there, you are selling a commodity. Agents are uniquely positioned to prevent this backward slide because they can infinitely personalize the backstage preparation, allowing the human performer to execute mass customization on the frontstage flawlessly.

NOTE

My take: The most dangerous antipattern in AI design is trying to put the agent on the main stage when it belongs in the orchestra pit. Let the machine do the heavy lifting of context gathering and data synthesis backstage, so the human can focus entirely on the emotional and transformative performance frontstage.

Translating Theoretical Axioms into Machine-Checkable Invariants

Theory is useless unless it can be compiled into code. If we believe in Service-Dominant Logic and the Experience Economy, we must write our agents' system prompts and validation logic to enforce these principles deterministically. We cannot rely on vague instructions like "be helpful." We must encode behavioral constraints.

In the `asd` CLI, we enforce the rule that an agent cannot unilaterally declare value. Value must be verified through the evidence register. If an agent tries to write a journey map that claims a customer is "delighted" without a cryptographic hash linking to a verified telemetry event or quote, the Critic agent vetoes the write.

rules/sdl-invariants.json

```
{
  "invariant_checks": [
    {
      "id": "sdl_fp10_value_in_use",
      "description": "Value claims must be phenomenologically anchored by beneficiary evidence.",
      "condition": "artifact.contains_value_claim = true",
      "assertion": "artifact.citations.length > 0",
      "violation_message": "FATAL: Artifact claims value realization without linking to the evidence register. Value is determined by the beneficiary, not the LLM. Add citation hash."
    },
    {
      "id": "sdl_fp6_co_creation",
      "description": "Service blueprints must show explicitly integrated resources across actor lanes.",
      "condition": "artifact.type = 'service_blueprint'",
      "assertion": "blueprint.lanes.customer_actions.length > 0 AND blueprint.lanes.frontstage.length > 0",
      "violation_message": "FATAL: Blueprint lacks co-creation. A service blueprint without customer actions is just an IT architecture diagram."
    }
  ]
}
```

This is the core of the Agentic Service Designer philosophy. We take the squishy, qualitative world of service design and we bind it in iron-clad, deterministic contracts. We use agents to explore the unbounded space of human experience, but we force their outputs through strict, machine-checkable schemas. This ensures that every artifact generated by SDAP adheres to the laws of co-creation.

Theoretical Principle	Agentic Implementation	Failure Mode Avoided
Beneficiary-determined value	Evidence register hashing	Hallucinated user sentiment
Value co-creation	Mandatory customer lanes in blueprints	Supplier-centric process maps
Work is Theatre (Staging)	Frontstage/Backstage delineation in schemas	Agents inappropriately acting frontstage
Internal Quality → External Value	Agentic automation of rote synthesis	Frontline burnout and high turnover

We build the operating system for value co-creation, far beyond simple conversational chatbots. By grounding our agent swarms in the foundational premises of Service-Dominant Logic, we ensure they move past raw artifact generation into active facilitation of complex, human interactions that generate lasting economic value.

```
factcheck.sh
```

```
# Enforcing SDL constraints via the ASD CLI
asd factcheck run --target ./output/03.html --ruleset sdl-invariants --strict

echo "Fact-check complete. Artifact complies with FP6 and FP10."
```

NOTE

My take: If your AI strategy doesn't explicitly model the human's role in the system, you don't have an AI strategy. You have a very expensive science experiment. Hardcode your human-in-the-loop dependencies as invariants, and your agentic services will actually survive contact with reality.

Ultimately, Service-Dominant Logic is not just an academic framework; it is the blueprint for survival in the agentic era. When models reach parity and token costs trend to zero, generating text or code will no longer be a competitive advantage. The only durable moat will be the depth and quality of the co-creation rituals you stage with your users. By applying SDL, the Service Profit Chain, and the Four Realms of Experience, we elevate our agents from mere operand utilities into orchestrators of human transformation. The math demands it, and the market rewards it. The next chapter will explore precisely how the Critic agent ensures this orchestration remains anchored to verifiable reality.

04 The Role Contract, Evidence Registers & Deterministic Fact-Checking

How the Critic agent and content-addressed rows prevent hallucinated design

If you want an AI agent to draw a beautiful journey map, any consumer LLM will happily oblige, hallucinating fictional customer pain points to fit a neat narrative arc. That is exactly what we cannot allow in an enterprise service design practice. This chapter dissects the rigor engine of the Service Design Agent Platform (SDAP). I am going to show you how we bind agentic swarms to reality using the Law of Artifacts, content-addressed evidence registers, and the adversarial Critic agent. We will trace how a hallucinated insight is caught by a machine gate before it ever reaches a human steering committee, proving that true autonomous design rests entirely on deterministic facts rather than better prompts.

The Law of Artifacts: Handoffs Are Never Chat

When you put ten autonomous agents in a digital room, their natural instinct—inherited from their chat-based training—is to talk to each other. The Mapper agent will ask the Researcher what the customers said. The Researcher will summarize it. The Synthesizer will chime in with a trend, and before you know it, you have a 40-turn conversation where the original customer reality has been compressed, paraphrased, and hallucinated out of existence. This is the death of rigor. It is why our foundational protocol is the Role Contract, and its first rule is absolute: *handoffs are artifacts, not chat*.

In the SDAP engine, when the Researcher finishes mining a transcript, it does not send a message to the Mapper saying, "Riders are frustrated by late buses." It writes a typed artifact to the project store. The Orchestrator advances the phase gate. When the Mapper wakes up to build the journey map, it consumes that structured artifact. The conversation is entirely bypassed in favor of immutable, typed JSON files that cite their sources.

```
role-contract.md
```

```
## 1. Handoffs are artifacts, not chat
```

```
1. Role A completes a method → writes a typed artifact through the project store
```


(kind from the §3.6 catalog).

2. The Orchestrator advances the phase gate (kind 37) – never a role other than Orchestrator may advance a gate.
3. Role B consumes the artifact; it never reconstructs the work from conversation.
4. Every artifact write is replayable from the store: the session log, the artifact JSON, and the evidence it cites must tell the same story. Anything shown to a model must be reconstructable from the store (suite rule: model-visible $\iff$ logged).
5. Parallel work is explicit: Orchestrator decides fan-out; independent streams may run concurrently, but all writes serialize through the store's single-writer mutex, and chunk merges funnel through one writer.

This protocol guarantees that anything shown to a model must be reconstructable from the store. If the Researcher found a crucial insight, it must exist as a content-addressed row in the evidence register. This explicit serialization prevents the silent scope creep that plagues unstructured AI workflows. Parallel work is allowed, but it is explicit. The Orchestrator can fan out tasks, but all writes funnel through a single-writer mutex. When the work is done, the artifacts stand alone as the complete record of the phase.

NOTE

My take: I spent my first two weeks with early agent swarms reading 100-page chat transcripts trying to figure out why the bot decided to scrap a mobile app feature. It was buried in turn 47 where an agent hallucinated a focus group constraint. Moving to an artifact-only handoff model was the most painful, yet most profitable architectural decision we made. It forces the agents to put their work on the table where I can see it, and it makes debugging a failed design decision trivial.

The status model for these artifacts is equally strict. An artifact is either a `draft` or it is `final`. A `draft` means the artifact is persisted on disk, but it has not yet passed its required gates. It carries its check report and any review rows attached to it. It becomes `final` only when the schema is valid, every evidence reference resolves to a real hash, the acceptance rules pass, and the Critic agent gives it a pass verdict. A final artifact can never be a failed artifact. A draft can always be remediated until it passes.

Status	Condition	Downstream Visibility
draft	Written to store, but fails machine gate or Critic veto.	Visible for remediation, ignored by phase exports.
final	Schema valid, refs resolve, rules pass, Critic pass.	Included in executive reports, read by downstream methods.

The Content-Addressed Evidence Register

To enforce the rule that no claim is made without backing, the SDAP engine requires an evidence register. But this is not a loose collection of text files or a conversational context window. It is a strict, content-addressed database of facts. Every piece of evidence is hashed using SHA-256 based on its source, kind, and body text. If you feed the exact same customer quote from the same interview into the engine twice, you get the exact same ID. This provides absolute referential integrity across the entire project.

Figure 4.1 Content-Addressed Evidence Flow. Every factual claim in a design artifact must trace to an immutable evidence hash.

Let's look at how this operates at the CLI level when the Researcher pushes a quote into the register for the Lighthouse Transit engagement. The `sd_evidence_add` tool takes the raw data and mints the immutable record.

```
terminal / asd evidence-add (Quote)
```

```
$ asd evidence-add --projectId lighthouse-transit-f7e5f8a2 \  
  --source call-821004 --kind quote \  
  --body "Told to wait 15 min for next bus; nobody held the tram."  
  
{  
  "evidence": {  
    "id": "c0f1736916289286b470528c08711c2b39ec7a7e3e853b00a162590b1197a174",  
    "projectId": "lighthouse-transit-f7e5f8a2",  
    "source": "call-821004",  
    "kind": "quote",  
    "body": "Told to wait 15 min for next bus; nobody held the tram.",  
    "capturedAt": "2026-08-19T00:23:29.065Z"  
  }  
}
```

Notice the ID: `c0f1736916289286b470528c08711c2b39ec7a7e3e853b00a162590b1197a174`. That hash is the atomic unit of truth for this engagement. When the Mapper agent later builds a Journey Map, the node representing the "Missed Connection" pain point will contain a JSON array of evidence references, and that hash must be in it. If the Mapper hallucinates an ID that doesn't exist in the register, the write operation is a hard error. Referential integrity is absolute.

This is where the magic of machine-checked acceptance rules kicks in. Every method manifest defines its threshold for evidence. For example, the `journey-from-transcripts` method requires at least one evidence reference per node. If the agent submits a graph with an unsupported node, the

write fails the acceptance check.

```
terminal / sd_artifact_write failure response
```

```
{
  "accepted": false,
  "status": "draft",
  "artifact": {
    "id": "2f537689386e439abd3ae43769ab710c",
    "kind": "journey-map",
    "methodId": "journey-from-transcripts",
    "status": "draft",
    "check": {
      "pass": false,
      "failures": [
        { "rule": "min-nodes", "message": "expected at least 3 nodes, found 1" },
        { "rule": "min-evidence-per-node", "nodeId": "transfer",
          "message": "node \"transfer\" cites 0 evidence refs, needs 1" }
      ]
    }
  },
  "failures": [
    "expected at least 3 nodes, found 1",
    "node \"transfer\" cites 0 evidence refs, needs 1"
  ]
}
```

Crucially, as shown above, a failing write is not discarded. It persists as a `draft` with its check failures permanently attached. This is a deliberate architectural choice. Rejection shapes the record. The agent must remediate the exact same draft using `--artifactId`, explicitly fixing the missing evidence reference rather than just trying to sneak a new artifact past the gate. By doing this, we keep a permanent audit trail of what the AI attempted to hallucinate and how the system blocked it.

The Three Evidence Kinds: Quotes, Telemetry, Documents

The register forces rigor by restricting evidence to exactly three allowable kinds: `quote`, `telemetry`, and `document`. There is no fourth kind. You cannot add a `summary`, an `assumption`, or a `feeling`.

Evidence Kind	Definition	Common Sources
quote	Verbatim spoken or written words from a stakeholder.	Interview transcripts, support tickets, survey free-text.

telemetry	Quantitative, system-generated operational data.	ERP logs, GTFS-realtime feeds, CRM performance metrics.
document	Static institutional artifacts and policy texts.	Union roster agreements, SLAs, architectural diagrams.

This limitation is not arbitrary; it is the core defense against subjective AI slop. Early in the Lighthouse Transit engagement, an agent attempted to insert a field note as an `observation`.

```
terminal / evidence-add failure (observation)
```

```
$ asd evidence-add --projectId lighthouse-transit-f7e5f8a2 --source int-07 \
  --kind observation --body "Riders leave 18 minutes early because they do not
  trust the schedule."
```

```
asd evidence-add: invalid arguments: "kind" must be one of
["quote","telemetry","document"]
running `asd help` shows usage          (exit code 1)
```

The parser outright denied it. Why? Because an "observation" is an interpretation, and the register only holds ground truth. The field note was forced to go in as a `document`, meaning any claims derived from it would trace back to "the designer's field report document," framing it correctly as an artifact of research, rather than a universal physical law. You cannot build behavioral personas based on AI interpretations; they must be rooted in telemetry and verbatim quotes. A persona based on a demographic stereotype will be rejected; the `behavioural-personas` skill explicitly denies `basis: demographic` in its manifest rules.

In the Aurora Vento case study, we saw how vital `telemetry` records were. We added rows pulling direct statistics from the legacy ERP export:

```
terminal / asd evidence-add (Telemetry)
```

```
$ asd evidence-add --projectId aurora-vento-118ee9d0 \
  --source telemetry-q2 --kind telemetry \
  --body "Fleet MTTR 5.2 days across 41000 machines; first-time fix 61 pct; auto-
  detected faults 12 pct."
```

When the Synthesizer and Mapper constructed the current-state ecosystem map and downtime journeys, they were forced to link nodes back to this specific 5.2 day MTTR metric. There was no room for an agent to guess that repairs took "a few days." The numbers had to match the source hash.

NOTE

My take: The day we hardcoded the rejection of the "observation" kind was the day the platform grew up. It forced junior designers and our own agents to stop writing "Users seem frustrated" and start logging "Call 821004: 'I want my refund now.'" It shifts the entire burden of proof from vibes to data. I don't care what you feel about the user; I care what the user said, word for word.

The Critic Review Gate & Veto Log Lifecycle

Passing structural integrity and machine-checked rules is not enough for complex design artifacts. A journey map might cite valid quotes for every node, but entirely miss the systemic issue causing the pain, or propose a solution that violates a known constraint. Enter the Critic.

The Critic is an adversarial agent that sits squarely on the write path. If a method's manifest declares `reviewGate: critic`, the artifact cannot become `final` until the Critic agent returns a `pass` verdict. If the Critic spots an issue, it issues a `veto`. But in SDAP, a veto is not a chat message saying "please fix this." It is a structured, evidence-cited artifact of its own, called the `review-veto-log`.

Figure 4.2 The Critic Review Loop. The Critic acts as the adversarial conscience of the design engagement. A veto requires explicit remediation.

Let's look at a real interaction from the Lighthouse Transit project. The Mapper agent submitted a draft journey map. It passed the machine gates. Then, the Critic reviewed it and caught a catastrophic flaw: the proposed "3-minute connection fix" contradicted a union roster document that had been logged in the register. The Critic vetoed the artifact.

```
terminal / sd_review (Veto)
```

```
$ asd review --projectId lighthouse-transit-f7e5f8a2 \  
  --artifactId 2f537689386e439abd3ae43769ab710c \  
  --verdict veto \  
  --rows '[{  
    "rowKind":"defect",  
    "severity":"high",  
    "rule":"evidence-sufficiency",  
    "message":"The missed-connection node needs the roster-constraint document  
cited alongside the rider quotes; without it the 3-minute connection fix looks  
feasible and is not",  
    "fix":"Cite rosters-07 on the transfer node",
```

```
"evidenceId": "ebc691cacc31cf5fb2db2f3c8347e5e085690a0971b0d3ca4d104a6e150fdc6a"
}]'
```

The output from the engine is deterministic and devastating to the draft:

JSON / sd_review veto response

```
{
  "status": "draft",
  "verdict": "veto",
  "logArtifactId": "a3c09c90fdb24c49be6efd1c1a58ce43",
  "rowsWritten": 1
}
```

The original artifact is demoted to `draft`. More importantly, a new final artifact—the `review-veto-log` (`a3c09c90...`)—is written to the store, linked to the journey map via a `reviewOf` edge. The defect row explicitly requires a `fix` and cites the specific `evidenceId` of the union roster document. The Mapper agent cannot argue with the Critic. It must consume the veto log, apply the required fix, and rewrite the draft.

In the Aurora Vento case study, we saw a similar intervention on an Ecosystem Value Map. The Mapper submitted a map claiming the value exchanged was "guaranteed operational performance." The Critic vetoed it immediately because that was a slogan, not a measurable outcome. The veto log forced the Mapper to define the exact uptime formula, the measurement window, exclusions, and the telemetry source before the map could anchor the contract.

JSON / review-veto-log snippet

```
{
  "rowKind": "defect",
  "severity": "critical",
  "rule": "measurability",
  "message": "The value Aurora exchanges is stated as guaranteed operational performance: no availability formula, no window, no exclusions, no data source. That is a slogan, not an outcome",
  "fix": "Name the uptime formula, the measurement window, the exclusions, the telemetry source, and the per-point remedy before this map can anchor the contract"
}
```

By forcing these fixes at the design stage, this adversarial loop caught a fatal flaw three weeks before the pilot, saving the client significant embarrassment and financial risk.

NOTE

My take: The Critic is the best member of the team precisely because it is merciless. It doesn't care about politeness or meeting the Friday deadline. If you claim a 3-minute connection is possible but the union document says otherwise, the Critic will fail you. I rely on the veto log as my primary audit trail when explaining decisions to stakeholders.

Deterministic Fact-Checking in CI/CD (sd_factcheck)

The ultimate safety net in the SDAP architecture is the `sd\_factcheck` tool. Designed to run both locally and in CI/CD pipelines, this tool replays the entire verification stack over every artifact in the project. It ensures that the project state is provably consistent before any human phase gate is crossed.

The fact-check runs three absolute assertions over every artifact:

1. **Integrity:** Every evidence reference cited by an artifact must exist in the register. There are no dangling pointers.
2. **Acceptance:** The artifact must pass its method's machine-checked rules (minimum nodes, required fields) against the live registry.
3. **Review Gate:** If the artifact is marked `final` and its method requires a Critic gate, it must carry a documented `pass` verdict.

Let's look at the source code that powers this final check. The `runFactcheck` function reads the entire evidence index at once for performance, then iterates over the artifacts to build the report.

factcheck.ts / snippet

```
for (const artifact of artifacts) {
  const dangling: string[] = []
  // 1. Verify integrity
  for (const node of artifact.nodes) {
    for (const ref of node.evidence) {
      if (!evidenceIds.has(ref.evidenceId)) {
        dangling.push(`${node.id} → ${ref.evidenceId}`)
      }
    }
  }

  const direct = artifact.methodId === undefined ? undefined :
asd.methods.get(artifact.methodId)
  const manifest = direct ?? byKind.get(artifact.kind)
```

```

let acceptance: FactCheckRow['acceptance'] = { pass: true }
if (manifest !== undefined) {
  const result = asd.checkAcceptance(manifest, artifact)
  acceptance = result.ok ? { pass: true } : { pass: false, failures:
result.failures.map(f => f.message) }
}

// 3. Verify review gate
const gate = direct?.reviewGate ?? null
const verdict = artifact.review?.verdict ?? null
const reviewPass = artifact.status === 'draft' || gate === null || verdict ===
'pass'

rows.push({
  artifactId: artifact.id,
  kind: artifact.kind,
  status: artifact.status,
  integrity: { pass: dangling.length === 0, dangling },
  acceptance,
  review: { pass: reviewPass, gate, verdict },
  methodId: manifest?.id ?? null,
})
}

```

When you run `asd factcheck` on a project, you receive a complete JSON report. If a single rule fails, the CLI returns an exit code of `1`, breaking the build or halting the pipeline.

```
terminal / asd factcheck (Pass)
```

```

$ asd factcheck --projectId lighthouse-transit-f7e5f8a2

{
  "pass": true,
  "rows": [
    {
      "artifactId": "2f537689386e439abd3ae43769ab710c",
      "kind": "journey-map",
      "title": "Current-state journey",
      "status": "final",
      "integrity": { "pass": true, "dangling": [] },
      "acceptance": { "pass": true },
      "review": { "pass": true, "gate": "critic", "verdict": "pass" },
      "methodId": "journey-from-transcripts",
      "methodResolution": "recorded"
    },
  ],
}

```

```

    "artifactId": "a3c09c90fdb24c49be6efd1c1a58ce43",
      "kind": "review-veto-log",
      "title": "Review: Current-state journey",
      "status": "final",
      "integrity": { "pass": true, "dangling": [] },
      "acceptance": { "pass": true },
      "review": { "pass": true, "gate": null, "verdict": null },
      "methodId": null,
      "methodResolution": "inferred"
    }
  ]
}

```

Notice the second row in that output. The Critic's veto log itself is treated as a final artifact and is checked alongside the work it reviewed. The audit trail is not a subjective meeting note; it is a cryptographic, referentially intact object stored on disk. If the presentation layer attempts to render a draft or an artifact with a failed review, the platform simply refuses to include it in the `executive-summary` export. Only what survives the gauntlet is presented to the client.

Closing the Loop: Rigor in Practice

The Lighthouse Transit engagement and the Aurora Vento servitization pilot prove that autonomous design systems cannot function on goodwill or clever prompting. When an agentic pipeline has the authority to write code, design operational services, or prescribe field tooling, the cost of a hallucination is not a funny screenshot; it is a broken supply chain or a stranded shift worker. By forcing every single node of a service blueprint or a customer journey to cryptographically bind itself to a real-world, hashed piece of evidence, we remove the temptation for generative models to generate fiction.

Furthermore, embedding the Critic agent as a hard gate on the write path, we build in an adversarial conscience that does not fatigue. A human designer might let a 3-minute transit connection slide because it looks good on a presentation slide. The Critic, mechanically comparing the proposed blueprint against the union roster document, will not. It will issue the veto, generate the `review-veto-log`, and force the Mapper agent to confront reality. This is the paradigm shift: we are no longer using AI to draw pretty pictures of ideal states. We are using it to mathematically prove that our proposed services can actually survive contact with the real world.

The Cost of Skipping the Gate

Some practitioners, especially those used to the free-flowing ideation of early ChatGPT or Claude sessions, might look at this architecture and think it is overly bureaucratic. Why impose strict JSON schemas, SHA-256 evidence hashing, and adversarial veto agents on what should be a creative

design process? The answer lies in the fundamental difference between building a prototype for a startup pitch and engineering an enterprise service system that handles millions of interactions.

In a traditional design process, the gaps in logic are smoothed over by human intuition. If a journey map glosses over how a ticket is routed, an experienced service designer implicitly knows that the IT backend will have to figure it out later. But autonomous agents do not have implicit operational intuition. If you leave a gap in a machine-generated blueprint, the agent will fill it with the most statistically probable sequence of events based on its training data. This is how you end up with a customer service AI agent deciding that the optimal way to handle a missed delivery is to automatically issue a full refund without checking inventory, simply because "issue refund" was a common resolution in the training set.

When you skip the evidence gate, you allow the model to replace your client's specific operational constraints with generalized internet wisdom. We saw this in early un-gated trials. We asked a swarm to redesign a hospital intake flow. Without the ``sd_artifact_write`` constraints and the Critic's veto, the agents proposed an automated digital check-in kiosk. It looked perfect on the generated service blueprint. It took a human reviewer two seconds to point out that 60% of the hospital's patients were elderly and required physical assistance, a fact explicitly stated in three different field interview transcripts. The agents had ignored the transcripts because a digital kiosk is a more "modern" and mathematically cleaner solution.

NOTE

My take: Creativity in service design is not about ignoring constraints to imagine a perfect world. True creativity is finding the elegant solution that survives the constraints. The SDAP engine does not stifle creativity; it forces it. By mathematically binding the agents to the evidence register, we force them to solve the hard, ugly problems instead of hallucinating them away.

This is why the ``sd_factcheck`` tool in the CI/CD pipeline is non-negotiable. It is the final backstop against AI drift. As the swarm iterates through hundreds of artifacts over an 18-week engagement, minor discrepancies can accumulate. A metric might drift from 71% to 75%. A persona's motivation might morph slightly to better fit an ideated solution. The deterministic fact-check catches these deviations by tracing every single edge back to the immutable SHA-256 hash in the evidence register. If the hash doesn't match, or if the source quote doesn't support the claim, the build fails. The artifact reverts to draft status. The Critic logs a new veto, and the swarm is forced to go back and reconcile its work with reality.

In the end, the role contract, the evidence register, and the deterministic fact-check are not constraints on the design process. They are the scaffolding that allows the design process to scale asynchronously. They allow a single human designer to orchestrate a team of ten specialized

agents, processing thousands of field interviews and telemetry logs in a matter of hours, knowing with absolute certainty that the final executive report contains zero hallucinations. This is how we move service design out of the workshop and into the engine room.

05 Research Ops at Scale: Mining Transcripts & Field Inquiries

Processing 200+ interviews with specialized research agent teams

Traditional qualitative research hits a structural wall at about forty interviews. You run out of wall space, you run out of working memory, and synthesis collapses into confirmation bias. Here is how I use agent teams to execute massive qualitative research programs—processing 200+ interviews without losing ethnographic depth, faking emotions, or hallucinating quotes.

The Scale Bottleneck in Qualitative Research

If you have run a discovery research program, you know the gravity of the bottleneck. The fieldwork is exhilarating. The synthesis is a slow, crushing march through hundreds of hours of audio. A standard agency quote for a 200-interview corpus stretches to sixteen weeks and well over a quarter of a million dollars. So, the budget forces a compromise: cut the corpus. We interview forty people, claim statistical saturation (a lie we tell ourselves), and present the findings.

But when you are resetting an entire service topology, forty interviews will not survive contact with a hostile executive board. They will spot the sampling gap. They will ask why the digital commuter segment only has six quotes. And they will reject the work.

TIP

CORE PRINCIPLE: You do not have to compromise corpus size to maintain rigor. Treat research operations as a service pipeline, and staff the processing stages with specialized AI agents.

In this chapter, I will show you how to structure an AI-first research operation. We will look at field shadowing protocols, transcript ingestion, and the hard machine gates that prevent agents from fabricating data. Humans handle the participant-facing work. Agents handle the transcript-facing work. The platform arbitrates the boundary.

Contextual Inquiry and Field Shadowing Protocols

Good service design starts in the field, not in a survey. You cannot fix a public transit network without riding the night bus. You cannot fix SME banking without sitting next to a business owner as they swear at a KYC portal.

To capture this reality, we use the `contextual-inquiry` skill. It forces a rigorous separation between what happened and what we think it means.

`contextual-inquiry.json`

```
{
  "id": "contextual-inquiry",
  "phase": "research",
  "inputs": [
    { "kind": "evidence-set", "min": 3, "note": "field observation notes and in-
context interview recordings" },
  ],
  "output": { "kind": "insight-set" },
  "steps": [
    { "role": "researcher", "instruction": "Shadow the participant in their own
environment and log every action, workaround and verbal aside verbatim, without
asking leading questions." },
    { "role": "synthesizer", "instruction": "Group the logged observations into
discrete insights, each keeping the exact quote or action it came from." },
    { "role": "critic", "instruction": "Reject any insight not traceable to a
specific observed moment.", "gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 3 },
    { "rule": "min-evidence-per-node", "min": 1 },
    { "rule": "field-non-empty", "field": "theme" }
  ]
}
```

Notice the machine gates. An insight-set output must have at least three nodes. Every node must cite at least one verified evidence row. If the Critic cannot trace the insight back to a specific, recorded moment from the field, the write fails and the artifact demotes to draft. This is how we kill the generic slop that normally infects field reports.

NOTE

My take: I have seen dozens of field reports that read like creative writing exercises. "Users feel overwhelmed by the interface." Really? Did they say that, or did they just sigh and click the back button? The `contextual-inquiry` skill forces you to log the sigh and the click. If you cannot cite the verbatim behavior, the engine will not let you ship the insight.

Let's look at Lighthouse Transit, a regional public transport operator carrying 600,000 daily boardings. They had a 38% failure demand rate and a falling modal share. I deployed the `contextual-inquiry` skill and the `@researcher` agent. We did not run focus groups. I rode the night shift routes. We logged 32 commuter interviews, 6 driver interviews, and 14 ride-alongs. The Researcher agent took the raw field notes and extracted the atomic evidence rows. Every missed connection, every broken card reader, every unanswered complaint became a content-addressed SHA-256 hash in the evidence register.

The Transcript Mining Engine: Automated Coding and Quote Provenance

When you scale to hundreds of interviews, manual coding breaks down. You cannot put 200 transcripts on a wall. But LLMs can drift if you just ask them to "summarize the themes." Instead, we use the `transcript-mining` skill to execute a deterministic extraction pipeline.

Figure 5.1 Transcript Mining pipeline. Automated transcript mining extracts verbatim evidence while preserving full conversational context.

Here is the exact machine contract for transcript mining:

transcript-mining.json

```
{
  "id": "transcript-mining",
  "phase": "research",
  "inputs": [
    { "kind": "evidence-set", "min": 5, "note": "call, chat or interview transcripts" }
  ],
  "output": { "kind": "insight-set" },
  "acceptance": [
    { "rule": "min-nodes", "min": 4 },
    { "rule": "min-evidence-per-node", "min": 2 },
    { "rule": "field-non-empty", "field": "theme" }
  ],
}
```

```
"reviewGate": "critic"
}
```

The pipeline operates in three stages. First, ingestion. Raw audio is transcribed, diarized, and strictly pseudonymized. This is non-negotiable. At Calder Community Bank, the identity mapping vault (linking participant codes to real names) lived exclusively on the client's restricted servers. The agent team only ever saw INT-041 and BR-031. If a PII lint pass caught a stray name, the file was quarantined immediately.

Second, extraction. The Researcher agent reads the pseudonymized transcript and outputs extracted evidence items as JSONL. The golden rule: every quote **MUST** be copied character-exact from the transcript, with start and end character offsets. No paraphrasing. If the participant said, "I sent the same extract twice, once as PDF once as a photo, nobody confirmed either", the agent must extract exactly that string.

Third, verification. A separate agent pass checks every extracted item against the original transcript span. If the offsets drift, or if the agent merged two quotes into one, the item is rejected. At Calder, out of 5,214 extracted items, 388 failed this verification and were dropped or re-extracted. The surviving 4,826 items became the immutable evidence register.

NOTE

My take: The offset verification check is the most important piece of code in the entire suite. LLMs are eager to please, and they will silently clean up messy human speech if you let them. By forcing a mechanical substring assertion on every quote, we strip the LLM of its ability to hallucinate. The data you get out is exactly the data the customer gave you.

Eliminating Confirmation Bias with Parallel Extraction Agents

Human researchers experience fatigue. By the fortieth interview, listening dulls into confirmation bias. In contrast, software agents operate without physical fatigue while remaining vulnerable to context window degradation and token bias.

To eliminate this, we use the Critic agent as an adversarial review gate, and we deploy a parallel extraction pass using a completely different model family (e.g., Codex) to challenge the primary agent's synthesis.

Let's return to Lighthouse Transit. The Synthesizer agent was building a journey map for the night shift worker. It proposed a "missed connection" pain point. But the Critic stepped in and vetoed the draft artifact.

Critic Veto Log (Lighthouse Transit)

```
{
  "status": "draft",
  "verdict": "veto",
  "logArtifactId": "a3c09c90fdb24c49be6efd1c1a58ce43",
  "rowsWritten": 2
}
```

The Critic logged a high-severity defect: *"The missed-connection node needs the roster-constraint document cited alongside the rider quotes; without it the 3-minute connection fix looks feasible and is not."* The engine forced the Mapper agent to remediate. The roster document was cited, and the unworkable 3-minute connection idea was killed at the blueprint drafting table, three weeks before the pilot. That is the power of adversarial machine gating.

Case Study: Calder Community Bank's 200-Interview SME Onboarding Study

Calder Community Bank is a fictional composite of several European regional banks I have worked with. They had 120 branches, 1.1 million customers, and a board that refused to believe any research findings. They needed a whole-service discovery across SME onboarding, branch service, digital, and complaints. A traditional agency wanted sixteen weeks and 285,000 euros for the job, and they would have secretly cut the corpus to forty interviews.

I ran the full 200-interview corpus in 10 weeks for under 100,000 euros using the ASD plugin.

The logistics were handled as a state machine in GitHub. Every interview was an issue. A nightly OpenCode job reconciled screener responses, updated consent states, and posted a quota burn-down table. When a participant withdrew, the `withdrawn` label triggered a deletion SOP that purged their transcript and all derived evidence items in a single pull request.

Figure 5.2 Theme frequency vs sentiment severity. Agentic synthesis surfaces low-frequency but catastrophic failure modes missed by manual tallying.

The synthesis was rolling. Every Friday, the Synthesizer agent re-clustered the growing evidence register. The Librarian agent maintained theme hygiene. By week 8, we had 96 clusters. We held a half-day human synthesis wall session, merged them down to 22 themes, and output 12 finalized insight statements.

Insight	Evidence base	Themes	Downstream decision
---------	---------------	--------	---------------------

Customers buy certainty with their own time because the bank gives them no status visibility.	71 items, 3 methods	T10	Status heartbeat concept
KYC feels like accusation, not protection, when documents are re-requested without explanation.	63 items, 2 methods	T01	Document-loop redesign
Branch queues are a rostering failure, not a demand problem.	84 items, 3 methods	T03	Roster pilot in 6 branches

To prove the rigor to the skeptics, I ran an inter-rater reliability (IRR) study. Twenty interviews (10% of the corpus) were double-coded blind—once by the pipeline, once by a senior human researcher. The Cohen's kappa reached 0.78 for theme-level agreement, clearing our 0.70 threshold. We also ran a Codex adversarial pass, which successfully killed one weak insight card (a "digital trust deficit" that lacked cross-stratum support). Every surviving insight traced perfectly back to character-exact transcript offsets.

The executive board tried to argue with the findings. I opened the CLI, ran a query against the content-addressed hashes, and showed them the exact quotes from their own customers. The argument ended, and the redesign began.

WARNING

CAUTION: Do not attempt to run a 200-interview corpus through an LLM by simply pasting transcripts into a chat window. Without a content-addressed evidence register, offset verification, and Critic gates, you will generate hallucinations at scale. The platform architecture is what makes this volume safe.

Deep Dive: The Calder Community Bank Qualitative Coding Matrix

To demonstrate the transcript mining engine in high-volume enterprise discovery, consider the benchmark engagement at Calder Community Bank. The bank sought to overhaul its commercial lending onboarding process, which suffered from a dismal 34% completion rate across a 90-day application window. Rather than relying on a sample of 12 interviews, the research team ingested 200 complete, recorded interviews with small business owners, branch managers, credit underwriters, and compliance officers.

The table below illustrates how the `transcript-mining` agent tagged verbatim statements, extracted emotional vectors, and established direct provenance hashes:

Evidence ID	Participant Role	Verbatim Quote Chunk	Semantic Breakdown Category	Severity (1-5)	Direct Evidence Hash
ev-cld-014	Hospitality SME Owner	"I uploaded our quarterly BAS statements three separate times because the portal timed out with no confirmation."	Tooling Failure / Upload Latency	4.5	sha256:7f4a.. .92b1
ev-cld-038	Credit Underwriter	"We cannot begin collateral appraisal until the identity verification PDF is manually stamped by a branch officer."	Operational Bottleneck / Manual Gate	4.8	sha256:1a8c.. .44e0
ev-cld-092	Retail SME Founder	"When I called support to ask why our application was stalled, the agent told me they had no visibility into the commercial underwriting queue."	Channel Silo / Context Loss	4.2	sha256:99d2.. .33fa
ev-cld-144	Branch Lending Specialist	"Our legacy CRM doesn't sync with the digital application pipeline, so we re-key company tax numbers by hand."	Backstage Friction / Re-work	4.0	sha256:bb41.. .ee88

TIP

CORE PRINCIPLE: In an agentic research practice, qualitative richness is not diluted by scale; it is amplified. By automating thematic coding and sentiment extraction across hundreds of transcripts, designers can pinpoint low-frequency, high-severity operational anomalies that traditional 10-person interview samples inevitably miss.

Eliminating Confirmation Bias with Adversarial Verification

A fatal pitfall in manual research synthesis is cognitive confirmation bias: the lead designer subconsciously prioritizes quotes that validate their pre-existing hypotheses while ignoring dissonant statements. The SDAP platform neutralizes this tendency by deploying independent extraction agents running opposing prompts.

While the primary `Researcher` extracts pain points and operational bottlenecks, an adversarial `Critic` agent independently parses the exact same transcript corpus looking for positive workarounds, unprompted praises, and systemic satisfiers. The two extractions are cross-referenced in an affinity convergence matrix. If the `Synthesizer` attempts to write an insight that contradicts more than 15% of the underlying evidence corpus without addressing the counter-evidence, the platform's acceptance gate rejects the artifact with a mandatory remediation notice.

06 Mixed Methods & Evidence-Backed Segmentation

Synthesizing telemetry data and qualitative friction into behavioral personas

For a decade, service design has struggled with a persistent, systemic split. We collect millions of rows of quantitative telemetry describing exactly what users do, when they click, and where they abandon funnels. In a separate silo, we conduct qualitative fieldwork—interviews, diary studies, and shadowing—that explains how users feel and why they act. The failure to fuse these two distinct streams produces a predictable disaster: marketing teams operate on demographic assumptions, data teams optimize for raw throughput, and design teams fight over isolated, anecdotal quotes. In this chapter, I will show you how to obliterate that divide. We will build a unified evidence system where quantitative telemetry and qualitative friction intersect to produce mathematically defensible, behavioral personas. We will cover why the Service Design Agent Platform (SDAP) explicitly outlaws demographic segmentation, how to deploy the `behavioural-personas` skill with strict acceptance gates, and how we applied this rigorous mixed-methods architecture at Vesper Pharmacy and Kestrel Energy.

The Fallacy of Demographic Personas

If you walk into a typical enterprise, you will likely find personas plastered on a wall. They usually have names like "Millennial Mary, 28, urban professional" or "Senior Steve, 72, retired grandfather." They are accompanied by stock photos, income brackets, and lists of fictional hobbies. I have seen hundreds of these artifacts, and I can tell you exactly what they are: expensive decorations. Demographic personas are an assumption-based view of the world. They describe a slice of the population, but they utterly fail to predict how a user will behave when confronted with a complex service failure.

Behavioral personas, conversely, are engineered to predict. They organize users by their dominant goals, their tolerance for friction, and the specific workarounds they invent when a service breaks. When we built the SDAP engine, we made a hard, deliberate design choice. We instructed the Critic agent to aggressively reject any persona artifact that relies on demographic clustering.

```
suite/packages/knowledge/data/methods/behavioural-personas.json
```

```
{
  "id": "behavioural-personas",
  "name": "Behavioural Persona Construction",
  "phase": "synthesis",
  "steps": [
    { "role": "synthesizer", "instruction": "Cluster participants by observed
behaviour and goal, never by demographic attribute alone." },
    { "role": "critic", "instruction": "Reject any persona whose defining goal is
demographic rather than behavioural.", "gate": true }
  ],
  "acceptance": [
    { "rule": "forbid-field-values", "field": "basis", "deny": ["demographic"] }
  ]
}
```

The code block above is not a suggestion; it is a machine-checked invariant. When an agent attempts to commit a persona artifact, the engine verifies the traceability of every claim. If the clustering logic defaults to age, income, or geographic region, the commit fails. Why? Because demographics deceive you. At Vesper Pharmacy, a fictional composite of European retail pharmacy chains, the data showed that the elderly repeat-patient cohort was served faster than anyone else—a median wait time of 4.3 minutes—yet they consistently returned the lowest Customer Satisfaction (CSAT) scores. If we had stopped at demographics, we would have assumed "seniors just complain more." By forcing a behavioral lens, we discovered they were not buying speed; they were buying certainty. They wanted a visible queue position and a warm acknowledgment, not a stopwatch.

NOTE

My take: I have sat in too many stakeholder meetings where a marketing executive insists that we need a "Gen Z" persona. I push back hard. Age brackets do not determine whether a customer prefers a self-serve portal or a phone agent when their smart meter catches fire. Behavior does. If you cannot trace a persona's core motivation to an observable action in your telemetry, you are writing fiction.

Fusing Telemetry Logs with Qualitative Interview Data

Methodological triangulation is the process of testing a single hypothesis through multiple independent lenses. In agentic service design, we execute this as a rigid, traceable matrix, not a casual synthesis workshop. Every emergent theme must register its quantitative evidence (the

telemetry log, the metric, the sample size) alongside its qualitative evidence (the verbatim quotes, the emotional valence, the contextual shadow report). This is how we answer the ultimate service question: "Quant says what, qual says why."

Figure 6.1 Mixed-Methods Convergence Model. True segmentation emerges only where behavioral telemetry intersects lived human frustration.

At Vesper Pharmacy, the business faced a massive operational headache: 18% of click-and-collect orders were never collected. That is 123,000 abandoned orders a year, all picked, packed, and eventually restocked. The digital team blamed the store staff. The store operations team blamed "unreliable app users." We needed to arbitrate the dispute using fused evidence.

We built a quantitative pipeline using OpenCode to process 1.8 million tokenized pickup events. The telemetry revealed a glaring pattern: no-shows spiked drastically when the automated "Your order is ready" notification fired *before* the physical medication was scanned onto the ready shelf. Orders with this "premature ready" flag suffered a 2.3x higher no-show rate. Concurrently, first-time click-and-collect users abandoned at a 31% rate, compared to 9% for repeat users.

The quant told us where the bleeding occurred. But it could not tell us why a human being would drive to a pharmacy and leave empty-handed. For that, we deployed Claude Code to mine 35 semi-structured interviews. The qualitative stream extracted the human reality: shoppers arrived, were told to "come back tomorrow," learned that the ready notification was a lie, and consequently stopped trusting all messages from Vesper. As one participant (INT-31) explicitly stated, "The app said ready. It was not ready. So now I phone, which defeats the entire point."

We mapped this directly into a triangulation matrix. The matrix structure is non-negotiable. Every row represents a theme. Every theme requires citations. If the qualitative stream provides a vivid, heartbreaking quote, but the quantitative stream shows it happens in only 0.01% of sessions, the integration agent tags the evidence strength as `weak`. Quotes illustrate; they do not elevate.

TIP

CORE PRINCIPLE: Never average a conflict between your quant and qual streams. If your telemetry says the queue is short, but your interviews say the queue feels agonizingly long, that conflict is your most valuable insight. It points directly to a failure in functional quality, transparency, or expectation management.

The behavioural-personas Skill & Acceptance Gates

When you trigger the `behavioural-personas` skill within the `asd` CLI, you are invoking a specialized swarm of agents designed to execute this rigorous fusion. The skill orchestrates the extraction of behavioral variables, runs clustering algorithms, drafts the narrative artifacts, and

subjects the entire output to the Critic agent's veto log.

The pipeline operates in a strictly defined sequence:

- 1. Data Cleaning & Feature Engineering:** We ingest raw telemetry, deduplicate records, and pseudonymize identifiers. We then construct a feature store consisting exclusively of behavioral spectrums. For instance, we measure contact frequency, channel preference, payment reliability, and digital adoption. We deliberately exclude raw income proxies and property-value estimates. They stereotype more than they predict.
- 2. Quantitative Clustering:** We deploy an agent to sweep k-means clustering across multiple configurations, validating stability through bootstrap resampling and silhouette scoring. We run a parallel hierarchical Ward clustering to ensure the partitions are mathematically robust.
- 3. Qualitative Typology:** Simultaneously, a separate agent mines transcripts and diary studies, extracting thousands of evidence items to build a qualitative typology of user behavior. Crucially, this agent operates completely blind to the quantitative clusters.
- 4. Reconciliation & Fusion:** The two independent models are overlaid in a fusion matrix. We look for convergence. Where a qualitative type neatly aligns with a quantitative cluster, we mint a persona. Where they diverge, we run an adversarial cross-check.

```
cli / asd-mcp

codex exec --skip-git-repo-check "Read synthesis/evidence-register.jsonl and
research/diary-summaries/. Derive a behavioral typology from first principles.
Do NOT read segmentation/clusters.md. Compare your typology to the clusters
and report: convergence, divergence, and which side has better evidence.
Be adversarial."
```

The Critic agent acts as the final gatekeeper. Before a persona is committed to the registry, it undergoes a rigid anti-stereotype audit. Does this persona merely decorate a demographic? Does every attribute trace back to an immutable SHA-256 evidence hash? Is there any deficit framing (e.g., describing a user as "tech illiterate" instead of "phone-first by preference")? If the artifact fails any of these checks, the Critic logs a veto, forcing a remediation cycle.

Calculating Behavioral Signatures and Need Clusters

Let's look at how we calculate a behavioral signature. We do not just eyeball data; we compute distinctness. For a cluster to survive the validation gates, it must exhibit a mean silhouette score greater than 0.35, and it must differ significantly from every other cluster on at least two behavioral dimensions. This prevents the creation of redundant personas that merely sound different but act the same.

Cluster Dimension	Metric Definition	Behavioral Implication
-------------------	-------------------	------------------------

Channel Posture	% of interactions initiated via phone vs app vs web portal.	Determines if digital nudges will work, or if an assisted-digital human bypass is mandatory.
Price Vigilance	Frequency of tariff checks, rate of post-communication switching.	Highlights users who will churn if arithmetic is opaque or obscured by marketing fluff.
Contact Intensity	Number of inbound calls/tickets generated per 12 months.	Identifies cohorts generating high failure demand due to misunderstood policies or friction.
Delegation Need	Flag indicating account management is routinely handled by a proxy.	Requires secure, verified proxy-access workflows instead of locking out elderly or vulnerable users.

When you compute these variables across millions of records, you expose the true topography of your customer base. You stop designing for a fictional average user and start designing for distinct, observable need states. A service blueprint built on top of calculated behavioral signatures ensures that every lane—from the physical evidence down to the backend database—is engineered to absorb the exact variations in human behavior that your telemetry has quantified.

NOTE

My take: The most politically dangerous part of this process is killing the legacy personas. When we proved that a client's four existing demographic buckets were mathematically useless, the marketing department fought tooth and nail to keep them. You cannot compromise here. If you allow a dual-segmentation system to exist, the organization will default to the easier, sloppier model. You must retire the old buckets entirely.

Case Study: Kestrel Energy's Evidence-Backed Customer Typology

Kestrel Energy, a fictional EU utility with 2.4 million household accounts, was drowning in a messy, mandated smart-meter rollout. They were designing their digital account services against four outdated demographic personas from a 2019 marketing study. The results were disastrous: 14% of smart-meter appointments failed on the day, and only 22% of installs were self-booked digitally. Kestrel had 2.4 million customer records, 1.9 million contact log rows, and 400 interviews, but it was all unusable raw data.

We deployed the SDAP stack to clean the data, execute a feature pipeline, and cluster the accounts. The quantitative model yielded seven distinct household clusters and four SME clusters. We then fused this with a qualitative typology generated entirely from transcript mining.

Figure 6.2 Kestrel Energy's Behavioral Typology. Evidence-backed personas provide mathematically defensible archetypes for system redesign.

The fusion produced decision-grade personas that fundamentally changed Kestrel's operations. Let's examine two of them.

Persona 1: Ingrid, the paper trail keeper (Cluster C2: Digitally Disengaged Seniors)

This cluster represented 11% of the base. They generated 89% of their contacts via phone. The qualitative evidence showed they were terrified of missing mandatory install instructions, leading to a high rate of reactive, anxious phone calls. The legacy smart-meter letter offered only a QR code for booking—a massive accessibility failure for a phone-first cohort. Based on this behavioral signature, we redesigned the touchpoint. We introduced an "assisted digital path" featuring a two-door letter (QR plus a prominent phone number), a proactive outbound call at week two, and formal delegation flows for family proxies. The result? Self-serve-or-assisted booking for this cohort skyrocketed from 22% to 81%, and failed appointments plummeted to 5%.

Persona 2: Markus, the tariff hawk (Cluster C3: Price Super-Vigilants)

Representing 14% of households, this cohort was defined by a 19% annual switching rate. Their qualitative signature was clear: they wanted raw arithmetic, not retention theatrics. When Kestrel sent opaque tariff changes, this cohort immediately checked broker sites and churned. Because we had computed their behavioral signature, we deployed a highly specific intervention: a smart-meter alerts service that displayed spend thresholds in euros and provided a transparent tariff-fit check with visible calculations. By treating them as competent adults and showing the raw numbers, Kestrel reduced switching in this highly volatile segment from 19% down to 13.5%.

At both Vesper Pharmacy and Kestrel Energy, the lesson was identical. When you stop guessing who your customers are and start computing what they actually do, you unlock the ability to engineer precise, surgical interventions. Mixed methods are not a luxury; they are the bedrock of the agentic service design operating system.

07 Mental Models, Alignment & Opportunity Mapping

Mapping how users think against how systems operate

Your software's architecture is almost certainly a mirror of your organizational chart, not your users' cognition. In this chapter, I will show you how to map the durable human intents that drive behavior, use agent teams to extract cognitive models from qualitative research at scale, and architect services that align with how users actually think.

Mental Spaces vs. System Features

If you have spent any time in enterprise software, you know the signs of a product built inside-out. The navigation menu matches the names of internal corporate divisions. Features are grouped by which engineering team shipped them, not by the task the user is trying to accomplish. When users are confused, the immediate executive reflex is to reskin the UI or write more tooltips. But cosmetic bandages cannot fix structural misalignment. What you are witnessing is a collision of mental models: the way the system is structured fundamentally contradicts how the human interacting with it understands the world.

In service design, a mental model is not just a vague psychological concept; it is an actionable architectural artifact. It is the internal representation a person holds about how a system works, built from their past experiences, assumptions, and immediate goals. When a user approaches your service, they do not bring a desire to interact with your specific database schema; they bring a set of beliefs, questions, and tasks. If your system requires them to abandon their mental model and learn yours, you generate friction. That friction manifests as support tickets, high churn, and abandoned tasks.

NOTE

My take: I have sat in dozens of boardrooms where product managers debated whether to put a feature in a dropdown or a modal. It rarely matters. If the feature forces the user to think in a way that violates their mental space, they won't find it anyway. Good UI cannot rescue bad ontology.

To fix this, we must map user cognition against system capabilities. We have to separate the "demand side" (what users are trying to achieve) from the "supply side" (what our system currently offers). Customer journey maps, while brilliant for chronological analysis, fall short here because they tie the user to a specific sequence of touchpoints. Mental models are non-linear. They are durable. While features and technologies change every few years, the fundamental human purposes driving them—like wanting to keep a child safe, or needing to feel financially secure—change at a glacial pace.

James Kalbach's alignment diagrams provide the master framework for this analysis. By drawing an explicit line of alignment between human intent and corporate capability, we can instantly spot the two greatest sins in product design: unmet needs (where users have a goal but we offer no tool) and operational bloat (where we maintain expensive features that serve no user goal).

TIP

CORE PRINCIPLE: Never let your internal system constraints dictate the user's cognitive space. The system must bend to the user, not the other way around.

Constructing Task Towers and Cognitive Ladders (Indi Young)

The definitive methodology for mapping these spaces belongs to Indi Young. Her Mental Model Diagrams act as a visual diagnostic of cognitive alignment. Imagine a city skyline. The top half of the diagram—the skyline itself—represents the user's mental space. It is composed of vertical "towers," where each tower is a stack of related atomic tasks, beliefs, and questions. These towers are grouped into broader mental spaces that represent the user's overarching goals.

The bottom half of the diagram, situated beneath a horizontal "Line of Alignment," represents the organization's current offerings. Features, services, and operational capabilities are placed directly underneath the specific task towers they support. If a tower stands naked with nothing beneath it, you have found an innovation opportunity. If a feature floats at the bottom with no tower above it, you have found a prime target for deprecation.

Figure 7.1 Indi Young Mental Model structure: User Mental Space (Top Towers) aligned above System Support Capabilities (Bottom Blocks). A mental model map reveals where system architecture mirrors internal org charts instead of user cognition.

Building this skyline manually is a grueling process of qualitative synthesis. In a traditional engagement, it involves printing hundreds of interview quotes, cutting them into strips, and affinity-mapping them on a wall for days. But in the SDAP ecosystem, we can accelerate this using deterministic agent workflows, provided we enforce strict taxonomy rules.

We extract statements into three explicit categories: *Beliefs* (claims users treat as true about how the world works), *Questions* (recurring information needs), and *Tasks* (intended actions). If you blur these lines, the model collapses into a generic collage. Every item must be stated in active, user-centric language. "The platform processes payments" is system language. "I need to know my money is safe" is a user belief.

schema-mental-model-item.json

```
{
  "$schema": "mental-model-item/v1",
  "item_id": "MM-<actor>-<seq>",
  "actor": "parent | teacher | student",
  "type": "belief | question | task",
  "statement": "active sentence in user voice, never system language",
  "evidence_ids": ["EV-T-014-003", "EV-T-014-011"],
  "mental_space": "thematic cluster label",
  "confidence": "high | medium | low"
}
```

The mental-model-alignment Skill and Artifact Contracts

To execute this extraction at scale without generating hallucinated slop, we deploy the `mental-model-alignment` skill. This flagship SDAP skill orchestrates an agent team—typically a Mapper, a Synthesizer, and a Critic—to process raw qualitative evidence into a structured alignment diagram.

The Mapper agent's job is to read the raw transcripts and extract extracted beliefs, questions, and tasks according to the JSON schema. It is explicitly instructed to never invent, paraphrase into corporate jargon, or merge statements prematurely. The Synthesizer then takes these atomic elements and begins clustering them into task towers, naming each tower with a descriptive, active verb phrase.

NOTE

My take: The most common failure mode for AI in synthesis is premature convergence. LLMs love to average things out into bland, generic insights. You must instruct your Synthesizer agent to preserve sharp edges and distinct segments. If a specific segment of users thinks differently, build a separate tower for them. Do not average them into oblivion.

Finally, the Critic agent acts as the adversarial gatekeeper. Its role is to attack the synthesized model. Did the Synthesizer classify a behavior as a belief? Reject it. Did the Mapper pull a quote that doesn't actually support the stated task? Veto it. The Critic enforces the artifact contract: no node can exist without a cryptographic hash linking it to a verbatim quote in the evidence register.

```
mental-model-alignment.json

{
  "id": "mental-model-alignment",
  "name": "Mental Model Alignment",
  "phase": "synthesis",
  "steps": [
    { "role": "mapper", "instruction": "Build two parallel journeys for the same process: the customer's and the frontline staff's, stage by stage." },
    { "role": "synthesizer", "instruction": "Mark every stage where the two mental models diverge, citing the quote from each side." },
    { "role": "critic", "instruction": "Reject any divergence marked without a quote from both sides.", "gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 3 },
    { "rule": "min-evidence-per-node", "min": 2 }
  ],
  "evidenceRules": { "traceability": "required" }
}
```

Enforcing this workflow through the CLI produces a mathematically defensible alignment diagram grounded in empirical reality, far beyond decorative illustration. We can run `asd factcheck` against the output, and the engine will trace every cognitive tower back to its source transcripts.

Identifying System Gaps and Accidental Complexities

Once the demand side (the user's mental model) is mapped, we deploy our agents to map the supply side (the system's actual architecture). We run card sorting exercises, parse existing documentation, and analyze support tickets to understand how the platform is currently structured. Then, we lay this system map beneath the cognitive skyline.

The divergence between the two is where the magic happens. We generate a Gap Inventory—a comprehensive list of every instance where user belief contradicts system truth. These aren't just UX bugs; they are fundamental architectural flaws. We classify these gaps by severity, cross-referencing them against support ticket volume and task failure rates.

Gap Signature	User Belief (Demand)	System Truth (Supply)	Business Impact
Structural Misalignment	The system organizes data by real-world entities (e.g., "my child")	The system organizes data by database objects (e.g., "course_id")	High navigation failure, massive support volume for basic tasks.

State Misunderstanding	"Saved" means I can resume my exact workflow later without data loss.	"Saved" means a transient draft is stored but can be overwritten by parallel sessions.	Data loss, shattered user trust, workflow abandonment.
Priority Inversion	The most urgent items appear first.	Items appear in reverse-chronological order of creation.	Missed deadlines, operational failure, user anxiety.

Accidental complexity arises when we try to solve these gaps with surface-level UI patches instead of content-model realignments. Adding a "Search by Child" filter to a course-centric architecture is a patch. It doesn't change the underlying mental friction; it just gives the user a slightly better shovel to dig themselves out of a hole we created.

```
gap-inventory-analysis.sh
```

```
asd run-skill mental-model-alignment \
  --input research/evidence-register.jsonl \
  --system-model models/system-content-model.md \
  --output artifacts/gap-inventory.md \
  --strict-evidence
```

NOTE

My take: The most dangerous thing you can do after identifying a major structural gap is compromise on the fix. If the gap requires changing the database schema, change the database schema. Hiding it behind an API translation layer usually just moves the confusion to the developers maintaining the frontend.

Case Study: Lumen EdTech's Gradebook & Parent Alignment Overhaul

Let's look at how this plays out in the real world with Lumen EdTech, a mid-sized European platform serving 240 schools. Lumen was drowning in 12,000 support tickets a quarter. 68% of these weren't bug reports; they were comprehension failures. Parents couldn't find grades, teachers lost grading drafts, and students missed deadlines. The executive team wanted a UI redesign. We ran an AI-first service design engagement to prove the problem was structural.

We processed 100 critical-incident interviews through our Claude Code agent team, extracting 1,180 evidence items. What emerged was a stark portrait of misaligned mental models. Lumen's platform was built around a rigid ontological hierarchy: everything hung off *courses* and *classes*. But that isn't how the humans in the system operated.

The Parent Model: Parents organized their mental space entirely by child. They didn't care about the 23 different class pages their two children were enrolled in; they wanted one aggregated view per child. The platform's course-centric model forced them on a digital scavenger hunt across

dozens of pages just to answer the question, "How was school today?" This gap generated 3,720 tickets a quarter.

The Teacher Model: Teachers believed that grading was a "sitting"—a continuous block of focused work that had a definitive saved state. When the UI showed a saving spinner, they assumed they could safely close their laptop and resume later. But Lumen's backend maintained one transient autosave draft per assignment column. If a teacher had parallel tabs open, the system silently overwrote drafts. 2,640 tickets a quarter were generated by this silent data loss.

The Student Model: Students believed the feed order represented priority. They looked at the top items and assumed those were due next. Lumen's feed was simply reverse-chronological by posting date. A minor assignment posted today would bury a major project due tomorrow. 1,800 tickets a quarter.

Actor	Core Belief (Extracted)	System Reality	Resulting Task Success
Parent	"The portal is about my child's progress."	Portal mirrors complex course structures.	42%
Teacher	"My grading sitting is securely saved."	Silent overwrites across concurrent tabs.	46%
Student	"The feed shows me what is due next."	Reverse-chronological posting order.	58%

The solution wasn't a UI reskin; it was a content-model realignment. We didn't build three separate portals, which would have siloed data and created more friction. Instead, we architected a unified Communication Hub. We introduced child aggregation across courses (fixing the parent gap), explicit `GradingSession` objects with optimistic locking and strict draft/published states (fixing the teacher gap), and a due-date-primary feed ordering (fixing the student gap).

```
cmc-04-grading-session.yaml
```

```
cmc_id: CMC-04
title: GradingSession object with explicit draft states
user_mental_model: grading happens in sittings; a sitting has saved state (B-T-11, 41 items)
proposed_model: GradingSession(teacher, class, assessment, state draft|published)
mitigations: optimistic locking; visible session-conflict warning; state transition log
kpi_contract: [draft tickets per quarter, teacher resume-task success]
```

We structured these revisions directly as Content Model Change (CMC) pull requests, subjected to rigorous inspection by our Critic agent and client engineering architects. By translating mental model insights directly into verifiable engineering artifacts, we bridged the gap between design research and software architecture.

The results of the 8-week pilot across 12 schools were conclusive. Support volume dropped by 46%. Comprehension tickets specifically plummeted by 69%. Parent task success for finding daily updates surged from 42% to 88%. By doing the hard work of aligning the system's ontology to the users' cognitive reality, we eradicated thousands of hours of failure demand.

TIP

CORE PRINCIPLE: Treat your mental model discoveries as engineering requirements, not just design context. If the mental model requires a new database entity, fight for that entity.

08 Ecosystem Value Mapping & Stakeholder Topologies

Mapping multi-actor value networks and institutional constraints

The single greatest lie in enterprise software is that the user and the system sit alone in a dark room. They never do. Every service operates inside a messy, contested network of partners, regulators, frontline staff, and shadow intermediaries. If you try to design a service by looking only at your own company's touchpoints, you will build a digital island that crashes the moment it hits the real world. In this chapter, we step back from the single-touchpoint journey map and look at the ecosystem. You will learn how to map complex multi-stakeholder value exchanges, how to plot stakeholder power topologies, and how to encode these networks into machine-auditable artifact contracts that prevent your AI agents from hallucinating partnerships that don't exist.

Services Are Never Islands: The Multi-Actor Network

I have lost count of how many times I have sat in a client's boardroom and watched them proudly present a "customer journey" that pretends their company is the center of the universe. A bank maps a mortgage journey and completely ignores the conveyancer, the real estate agent, and the local tax authority. A healthcare provider maps a patient portal but forgets that the patient has a GP, a local pharmacy, and a family caregiver acting as an unpaid administrative assistant. This myopia is fatal. Services are never islands. They are complex multi-actor networks.

The Service-Dominant Logic (SDL) framework teaches us that value is co-created by multiple actors integrating their resources. No single actor owns the whole ecosystem. When we transition from sticky notes to a Service Design Agent Platform (SDAP), we must model these networks rigorously. The agentic system cannot reason about "the healthcare system" as a vague blob. It needs specific nodes. It needs directed edges representing value exchange. It needs institutional constraints mapped as hard rules. If the platform does not know that a GP has professional autonomy, it might suggest an operational flow that dictates clinical decisions to the GP, which the GP will instantly reject. You have to map the ecosystem to map the constraints.

Let's be precise. A multi-actor network consists of focal actors (the end beneficiaries), direct care or service rings (the immediate providers), enabling rings (the funders, technology vendors, platforms), and institutional rings (regulators, professional bodies). In our SDAP, these rings are not

just conceptual circles on a whiteboard. They are typed objects in an evidence register, backed by quote and document hashes. They define the perimeter of what can and cannot be done legally, ethically, and practically in a service redesign. Without this explicit ontology, AI agents will naturally drift toward assuming omnipotence—generating concepts where your client magically controls every step of the value chain. By enforcing a multi-actor lens, we force the AI to design for orchestration, not domination.

NOTE

My take: The fastest way to tell if a service designer understands the business is to ask them to draw the ecosystem. If they only draw the customer and the client's app, they are a UI designer masquerading as a service designer. If they draw the invisible legal contracts, the uncompensated labor of the customer, and the data flow to the regulator, you have found someone who can actually ship a service.

To orchestrate an ecosystem, you have to acknowledge that you don't control it. This was the exact challenge at Northgate Health, a mutual insurer with 40,000 members across the Benelux region. They had successfully built a slick telehealth onboarding flow, but the moment a member left the telehealth app to see a specialist, the data trail evaporated. Members were physically carrying paper referral letters between GPs and specialists, and pharmacists lacked timely care-plan context. Northgate wanted to fix "slow referrals," but they were blinded by their own organizational boundaries. They couldn't see that the member was acting as the unpaid systems integrator. The symptom was slow turnaround; the disease was a total lack of cross-actor orchestration.

Stakeholder Power/Interest Topologies & Strategy Formulation

Before you can rewire an ecosystem, you must understand the political physics holding it together. Stakeholder mapping is how we plot this. It is the process of locating every party with influence over, or interest in, the service, and categorizing them to formulate a governance strategy. This is not a feel-good exercise in inclusion. It is a defensive perimeter against project failure. The politics of an ecosystem will kill your service faster than bad code ever will.

In our agentic toolchain, this mapping is governed by the strictly typed `stakeholder-mapping` skill. It requires the Researcher agent to identify stakeholders from the evidence register, and the Mapper agent to plot them on a power/interest matrix. The Critic agent then audits this map, strictly rejecting any stakeholder placed without a cited reason, or whose influence is marked as "tbd". The machine does not tolerate political guesswork.

```
method-run.sh
```

```
$ asd method-run --project northgate-8f2a \
  --skill stakeholder-mapping
# The engine parses the stakeholder-mapping.json contract
# Role: mapper
# Acceptance rules:
#   - rule: min-nodes, min: 3
#   - rule: min-evidence-per-node, min: 1
#   - rule: field-non-empty, field: influence
#   - rule: forbid-field-values, field: influence, deny: ["unknown", "tbd"]
```

Consider the matrix deeply. High influence, high interest stakeholders must be managed closely. These are your sponsors, your core partners, and sometimes your fiercest blockers. Low influence, high interest stakeholders must be kept informed. They are often the end users or frontline staff who suffer the consequences of bad design but lack the budget to veto it. High influence, low interest stakeholders must be kept satisfied. These are the regulators, the legal departments, and the security compliance teams. If you wake them up with poorly scoped data sharing plans, they will shut you down immediately.

At Aurora Vento, a heavy industrial vendor manufacturing hydraulic access equipment, the stakeholder map exposed a hidden fault line. Aurora was trying to shift to outcome-based contracts—selling 98% machine availability rather than just selling spare parts. The executive team (high influence, high interest) loved it. The core customers (high influence, high interest) demanded it. But the regional distributors, who historically made their margin on emergency spare parts, were high influence and deeply hostile to a centralized dispatch model. If we had not mapped their influence rigorously through the platform, the entire servitization effort would have been ambushed in the regional rollout.

TIP

CORE PRINCIPLE: Never plot a stakeholder without attaching a specific piece of evidence to their placement. If you cannot cite a quote or a document proving their influence, they belong in a lower quadrant until proven otherwise.

Stakeholder Profile	Influence	Interest	Strategy	Evidence Citation (SHA-256)
Services SVP	High	High	Manage closely (Executive Sponsor)	2bf23273d... (Executive mandate memo)
Regional Distributors	High	Low (Hostile)	Keep satisfied / Negotiate	7a1b94c2d... (Parts margin analysis)

Field Technicians	Medium	High	Keep informed / Co-design	5d8e41a9f ... (Shadow day observations)
Data Protection Regulator	High	Low	Keep satisfied (Compliance)	8c2f10b3e ... (Telemetry DPIA constraint)

This is where the rigorous nature of SDAP outshines traditional consulting. A human consultant might gloss over the regional distributors to appease the CEO. The AI agent, constrained by the stakeholder-mapping JSON schema, demands a placement and an evidence link. When it reads a transcript where a distributor threatens to walk if parts margins drop, the Synthesizer forces that hostility into the matrix. The map becomes an empirical reflection of political risk, not a smoothed-over slide deck.

Ecosystem Value Flows and Tangible/Intangible Exchanges

Once you know the actors and their political power, you must map the blood flowing between them. An ecosystem value map traces what is exchanged between actors. This is rarely just money for goods. In modern service ecosystems, the most critical exchanges are intangible: information, legal authority, care, labor, and trust. If you fail to map the intangible, you fail to map the actual operating model.

If a GP sends a referral to a specialist, what is flowing? It is not just clinical context (information). It is also a transfer of clinical responsibility (legal authority). If the patient carries the paper letter to the specialist's office, they are providing uncompensated courier labor. If the specialist never sends a discharge summary back to the GP, that is a one-sided exchange, resulting in a fractured care loop. The ecosystem-value-mapping skill forces the Mapper agent to draw a directed edge between pairs of actors and explicitly type the value flowing across that edge.

Figure 8.1 Ecosystem Value Map showing reciprocal value flows between payers, providers, patients, and digital platforms. Sustainable service design requires every participant in the ecosystem to extract net positive value.

The Critic agent plays a decisive role here. When we mapped the Aurora Vento ecosystem, the initial agentic pass generated a value flow from Aurora to the Customer labeled "guaranteed operational performance." The Critic vetoed it instantly. Why? Because the platform demands measurability and empirical specificity. The veto log threw a critical severity defect: *"The value Aurora exchanges is stated as guaranteed operational performance: no availability formula, no window, no exclusions, no data source. That is a slogan, not an outcome."*

The fix was mandatory: we had to define the uptime formula, the measurement window, and the telemetry source before the map could be accepted as final. This forced the team to do the hard work of actual business design. We had to specify site-level accounting, the 2% credit per point

under target, and the explicit exclusions like scheduled maintenance and force majeure. The ecosystem map became a hard contract architecture, not a marketing diagram.

NOTE

My take: The most dangerous flows in an ecosystem map are the one-sided arrows. If an actor is pushing data, labor, or risk to another actor and receiving nothing in return, that link will eventually break. Ecosystems are held together by reciprocal value. If you find a one-sided flow, you have found the epicenter of your next service failure. Always ask: "What does the receiving party have to do, and why on earth would they do it?"

This level of rigor is what separates agentic service design from workshop theater. When a human draws an arrow on a whiteboard and writes "Data," everyone nods in vague agreement. When an LLM tries to write that same arrow into the SDAP register, the engine demands a source citation, an exact data schema, and a consent basis. The platform forces us to be honest about the institutional constraints surrounding the exchange. It prevents us from hallucinating an automated API integration where none legally or technically exists.

The ecosystem-value-mapping Skill Contract

Let's lift the hood on how the engine enforces this rigor. The `ecosystem-value-mapping` skill is not a loose conversational prompt; it is a strict, typed JSON contract that orchestrates multiple agent roles, binds them to the evidence register, and enforces deterministic verification via the `sd_factcheck` routine.

`ecosystem-value-mapping.json`

```
{
  "id": "ecosystem-value-mapping",
  "name": "Ecosystem Value Mapping",
  "phase": "synthesis",
  "inputs": [
    {
      "kind": "evidence-set",
      "min": 3,
      "note": "partner interviews or contracts"
    }
  ],
  "output": {
    "kind": "ecosystem-map"
  },
  "steps": [
    {
```

```

"instruction": "Identify every actor that exchanges value with the focal service,
direct or indirect."
  },
  {
    "role": "mapper",
    "instruction": "Draw a value flow between each pair of actors and label what
is exchanged."
  },
  {
    "role": "critic",
    "instruction": "Reject any actor or flow not backed by a cited source.",
    "gate": true
  }
],
"acceptance": [
  { "rule": "min-nodes", "min": 3 },
  { "rule": "min-evidence-per-node", "min": 1 },
  { "rule": "field-non-empty", "field": "value" }
],
"evidenceRules": {
  "traceability": "required"
},
"reviewGate": "critic"
}

```

This manifest dictates exactly how the work proceeds. The Researcher extracts the actors from the evidence-set . The Mapper connects them. Crucially, the `min-evidence-per-node` rule ensures that no actor or flow can exist in the output artifact unless it traces directly back to a SHA-256 hash in the register. The `field-non-empty` rule on the `value` field prevents generic lines; the engine must specify what is flowing (e.g., "prescription authority" or "medication adherence advice"). Finally, the Critic acts as the ultimate gatekeeper, evaluating the semantics of the flows against the evidence.

When this skill executes, it produces an artifact of kind `ecosystem-map` . If the Mapper hallucinated a "Digital Health Hub" that wasn't supported by the research, the Critic vetoes the draft. The engine records the veto in a `review-veto-log` , persists the failed draft, and forces a remediation pass. This deterministic pipeline guarantees that the resulting ecosystem map reflects operational reality through empirical backing, eliminating idealized speculation.

Case Study: Northgate Health's Care Ecosystem & Aurora Vento's B2B Network

The true power of ecosystem mapping reveals itself in practice, especially when orchestrating across highly fragmented partner networks. At Northgate Health, the ecosystem map completely reframed the strategic objective. We started with 120 stakeholder interviews and 24 months of claims data, all meticulously mined by the `transcript-mining` skill. The resulting value-exchange model made one thing painfully clear: the system closed cost loops far faster than it closed care loops. Outcome information from specialists returned to referrers too late or not at all—only a 54% loop closure rate within 30 days. The member was trapped in the middle, ferrying context manually from clinic to clinic.

Because the map explicitly typed the value flowing (clinical context vs. funding vs. scheduling authority), we realized Northgate could not just build a monolithic portal and demand that GPs use it. The institutional constraint—GP professional autonomy and their reliance on existing Electronic Health Records (EHR)—meant that a centralized portal approach would fail immediately. Instead, we designed the "Northgate Gateway": an orchestration layer that coordinated referrals via open FHIR APIs (connecting directly to existing GP systems) and provided a unified member record that respected member consent toggles.

The ecosystem map guided us to orchestrate the seams without trying to own the clinical source of truth. By explicitly mapping the required value exchange (acknowledgement, status heartbeat, and outcome summary), we designed an Orchestration Canvas that replaced paper letters with a digital workflow. The results from the partner pilot validated the approach: median referral admin cycle dropped from 6.0 days to 1.2 days, and referral loop closure leaped from 54% to 88%. We achieved this by treating the GP, the specialist, and the member as co-creators of value, rather than subservient nodes in an insurer's database.

NOTE

My take: The Northgate breakthrough came when we realized we were designing a diplomatic treaty, not a piece of software. When you map an ecosystem, you are negotiating data sovereignty and professional pride. If your map doesn't respect the autonomy of the actors, your software will be rejected by the immune system of the industry.

Aurora Vento presented a different challenge, but one equally dependent on rigorous ecosystem mapping. They were shifting a massive B2B capital-equipment business toward outcome contracts. They had 41,000 machines in the field and a miserable MTTR (Mean Time To Repair) of 5.2 days. The initial mapping exercise, governed by the `ecosystem-value-mapping` skill, highlighted the

friction between the central manufacturer and the regional parts distributors. The evidence register showed that 41% of repair delays were caused by "dispatch mismatch"—a technician arriving without the correct part, or a part arriving without the technician.

By mapping the precise flows of diagnostic telemetry, parts supply, and labor scheduling, we identified that the legacy ERP was acting as a massive bottleneck, capturing data hours after the fact. The service blueprint that followed explicitly modeled a new telediagnosis lane and a coupled parts-and-skills dispatcher. More importantly, the rigorous constraints of the artifact schema forced Aurora to replace the nebulous promise of "guaranteed operational performance" with a hard, contractual uptime formula.

During the adversarial testing orchestrated by the Critic agent, we mapped the site-level cap loop (where two simultaneous failures at one site could be manipulated to mask a contractual breach). Because the ecosystem map forced us to link the telemetry data source directly to the financial remedy, we exposed this flaw and closed it before the contracts were signed. This level of systemic foresight is only possible when you treat the ecosystem map as a strict, queryable data structure rather than a loose visualization.

In both cases, the agentic platform provided the necessary discipline. The autonomous roles—Researcher, Synthesizer, Mapper, Measurer, and Critic—operated within strict artifact contracts. When the human designer (me) attempted to take a shortcut, the machine pushed back. When the client's internal bias threatened to skew the design toward their own convenience, the evidence register held the line. The engagement delivered mathematically defensible operating architectures that yielded measurable impact on day one of the pilot, moving far beyond decorative diagrams.

Designing for the ecosystem means surrendering the illusion of total control and embracing the reality of orchestration. It requires understanding that every interaction is an exchange of value governed by institutional rules and power dynamics. By leveraging the Service Design Agent Platform, we can map these complex topologies with unprecedented speed and rigor, ensuring that when our services finally hit the real world, they survive the collision.

Conclusion: The Ecosystem as Code

If there is a single lesson to take from mapping the ecosystems of Northgate Health and Aurora Vento, it is that a service map is not a picture; it is an operating contract. When we treat the ecosystem as a set of hard constraints—institutional rules, reciprocal value flows, and evidenced power dynamics—we stop designing interfaces for imaginary worlds and start designing orchestration for the real one. The Agentic Service Designer toolchain enforces this reality. By tying every stakeholder and every value exchange to a specific hash in the evidence register, the platform guarantees that our strategic ambition never outruns our empirical data.

The transition from workshop theater to executable service architectures requires a fundamental shift in mindset. We are no longer drawing arrows on a whiteboard and hoping the IT department can figure out the integration layer. We are authoring machine-readable artifact contracts that define the very limits of what the service can achieve. The `ecosystem-value-mapping` and `stakeholder-mapping` skills are not just generative AI prompts; they are strict schemas that defend the integrity of the design against political convenience and organizational myopia.

As you move forward into your own practice, remember that the most beautiful journey map is worthless if it ignores the uncompensated labor of the frontline staff or the regulatory constraints of the enabling partners. Use the tools. Trust the Critic agent when it vetoes your unsupported claims. Embrace the friction of the multi-actor network. Because in the age of autonomous service ecosystems, the only services that survive are the ones that have mapped their own constraints before they write their first line of code.

09 Living Journey Maps: From Transcripts to Sentiment Curves

Generating evidence-gated journey maps with derived emotional curves

For decades, journey mapping has been an exercise in corporate folklore and workshop theater, producing static posters drawn from assumed empathy rather than hard data. This chapter demonstrates how to synthesize massive qualitative and quantitative datasets into living, evidence-gated journey maps. We will explore how to derive emotion scores exclusively from verbatim evidence, map failure demand, and treat the journey map as executable code that acts as the absolute source of truth for cross-touchpoint customer friction.

The Anatomy of an Evidence-Gated Journey Map

The traditional customer journey map is dead. The artifacts that used to consume weeks of stakeholder workshops and graphic design cycles—only to be printed on plotter paper and ignored—have no place in an AI-first service design operating system. The fundamental problem with legacy journey maps is their lack of provenance. When a stakeholder asks, "Why is the customer frustrated here?" the traditional answer relies on a consultant's memory or a vaguely recalled sticky note. In the Agentic Service Designer (ASD) paradigm, a journey map is an index over a cryptographic evidence register. Every node, every emotion, and every pain point must cite its sources.

An evidence-gated journey map adheres to a strict five-element anatomy, adapted from Kalbach's alignment diagrams but enforced deterministically by the SDAP engine:

- **Lens:** The specific persona or user segment experiencing the journey. We do not blend fundamentally different user types into a single average.
- **Context:** The hard boundaries of the map. When does the journey start, and when does it end?
- **Spine:** Chronological phases defined strictly by passenger or customer goals, never by internal organizational stages.
- **Evidence Rows:** The core of the map, containing observable actions, touchpoints, verbatims, and emotion scores.
- **Takeaways:** Derived insights, failure demand points, and operational opportunities.

Figure 9.1 Current-state Customer Journey Map. Journey maps provide the single source of truth for cross-touchpoint customer friction.

To understand the power of this anatomy, we must look at how the data is stored. In the ASD engine, a journey map is not a drawing; it is structured data versioned in Git. It consists of a Mermaid rendering (`map.md`), a tabular representation (`rows.csv`) where every cell demands an evidence citation, and strict definition files (`phases.yaml`, `emotion.yaml`). When we map a service, we are simply projecting registered evidence against a chronological spine.

TIP

CORE PRINCIPLE: The map is an index over the evidence register, not a drawing. Every row, emotion, and pain point must be content-addressed to immutable evidence hashes.

Deriving Emotional Sentiment from Verbatim Evidence

The most widely abused element of traditional journey mapping is the emotional sentiment curve. Designers routinely invent emotional highs and lows to create a compelling narrative arc, fabricating "valleys of despair" and "moments of delight" without a shred of data. In an agentic workflow, this practice is aggressively blocked. The platform forces a profound shift: we score the evidence, never the vibe.

Agents cannot be trusted to infer emotion. If you ask an LLM to evaluate the sentiment of a frustrated customer journey, it will hallucinate a smooth, dramatic curve. It will flatten the extremes into a "sentiment smoothie" that provides no actionable insight. To combat this, we utilize a strict emotional-curve calibration method.

emotion.yaml

```
1 Distressed. Passenger reports crisis language, tears, panic, giving up.
  Anchor: "The children slept on our coats." (INT-041)
2 Negative. Frustration, distrust, complaint intent; workaround forming.
  Anchor: "Nobody could tell me anything, so I stopped asking." (INT-074)
3 Neutral. Transactional, no valence language, task completed flatly.
  Anchor: "I filled in the form and waited." (INT-118)
4 Positive. Relief, gratitude, effort acknowledged.
  Anchor: "The agent actually fixed it while I watched." (INT-096)
5 Delighted. Advocacy language, unprompted praise, story retold.
  Anchor: "They rebooked me before I reached the terminal." (SOC-402)
RULES: score only explicit evidence; null when absent; never average nulls;
every batch must show per-phase standard deviation ≥ 0.8 or be re-scored.
```

This rubric is passed to the tagging agents. The instruction is absolute: if there is no explicit emotion in the quoted text, the output is `null`. The emotion score for a phase is calculated as the mean of the scored items within that phase. This approach ensures that emotional troughs in our journey maps represent genuine, documented human frustration, not an AI's narrative embellishment.

Figure 9.2 Evidence-Derived Sentiment Curve. Emotional troughs in journey maps highlight the exact operational handoffs requiring redesign.

NOTE

My take: I have seen countless mapping sessions derail because stakeholders argue over whether a customer is "annoyed" or "angry." When you project a sentiment curve derived purely from phase-level means of 17,000 tagged, anchored verbatim quotes, the argument stops. You are no longer debating feelings; you are reviewing mathematics.

Mapping Touchpoints, Channels, and Failure Demand Points

A journey map must align the customer's chronological experience against the channels and touchpoints they interact with. However, the real value lies in identifying failure demand—the contact volume generated by a failure to do something or do it right for the customer. By mapping failure demand directly onto the journey timeline, we isolate the exact systemic failures that drive operational costs.

In our evidence tagging schema, every item is assigned a phase, a channel, and a pain code. These pain codes (e.g., `INFO` for information blackout, `REB` for rebooking failure, `CARE` for lack of accommodation) allow us to quantify the severity of failure demand at every step.

Phase	Dominant Pain Code	Evidence Context	Failure Demand Implication
Pre-flight	INFO (Notification lag)	Median 47 mins after operations decision before passenger is told.	Drives passengers to airport unaware, increasing terminal congestion.
At-airport	INFO (Information blackout)	Contradictory statuses across app, gate screen, and staff.	Generates massive desk queue volume and escalates anxiety.
Rebook	REB (Self-serve failure)	App rebooking fails on 66% of attempts.	Deflects 41% of users to contact center, spiking call volume.
Recovery	COMP (Claim friction)	11-field form causing 18% abandonment.	Increases regulatory risk and destroys post-disruption loyalty.

By visualizing these failure demand points on the map, we transition from observing pain to diagnosing systemic failure. The map becomes an operational diagnostic tool. We can see precisely where the digital channel fails, forcing the customer into a high-cost physical channel like a service desk or a contact center.

The journey-from-transcripts Skill & In-Place Remediation

The transformation of raw transcripts into a validated journey map is governed by the `journey-from-transcripts` skill. This skill dictates the roles, the instructions, and the strict acceptance criteria enforced by the SDAP engine.

journey-from-transcripts.json (Abridged)

```
{
  "id": "journey-from-transcripts",
  "phase": "synthesis",
  "inputs": [ { "kind": "evidence-set", "min": 3 } ],
  "output": { "kind": "journey-map" },
  "steps": [
    { "role": "researcher", "instruction": "Extract every distinct customer action, channel and breakdown..." },
    { "role": "mapper", "instruction": "Order the actions into stages. Give each stage an emotion drawn from the quotes..." },
    { "role": "critic", "instruction": "Reject any stage whose emotion or pain point is not supported by a quoted line.", "gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 3 },
    { "rule": "min-evidence-per-node", "min": 1 },
    { "rule": "field-non-empty", "field": "emotion" },
    { "rule": "field-non-empty", "field": "touchpoint" }
  ],
  "reviewGate": "critic"
}
```

When the `@mapper` agent attempts to write the artifact via `sd\_artifact\_write`, the engine executes structural integrity checks and evaluates the acceptance rules. If a node lacks an evidence reference, the write fails. Crucially, a failing write is not discarded; it is persisted as a `draft` artifact with its failures attached. The agent must remediate the artifact in-place by referencing the same ID and supplying the missing evidence.

Once the machine gates are passed, the artifact enters the Critic review gate. The `@critic` agent evaluates the map against the underlying evidence register. If it finds an unsupported claim—for example, an emotion score that lacks a verbatim anchor—it issues a veto. The Critic's veto is

recorded as a `review-veto-log` artifact, detailing the defect, its severity, and the required fix. The Mapper must apply the fix and resubmit. This adversarial review loop ensures that the final journey map is impregnable to skepticism because it is mathematically bound to the source data.

NOTE

My take: The Critic agent is the most valuable member of the team. Having an AI ruthlessly audit your map for evidence gaps before you present it to stakeholders saves you from the embarrassment of defending a hallucinated insight. It forces the design team to be rigorously honest.

Case Study: Albatross Airways Flight Disruption & Lighthouse Transit

Albatross Airways: The Disruption Recovery Map

Albatross Airways, a mid-size EU network carrier, faced a massive failure demand problem during disruption days. Their existing "irregular operations journey" poster from 2023 was unvalidated and ignored. Using the ASD workflow, we processed 140 passenger interviews, 9,000 disruption tickets, and over a million session logs.

The resulting evidence-gated journey map shattered the airline's internal folklore. The executive assumption was that passengers were angriest about the initial cancellation or the slow compensation process. The map's sentiment curve, derived strictly from verbatim scores, revealed a different truth. The lowest emotional trough (1.4 out of 5) occurred during the "At-airport" phase—a 90-minute information blackout where the app, the gate screen, and the ground staff all provided contradictory information. The second deepest trough (1.6) occurred during the "Rebook" phase, where the self-serve engine failed 66% of the time, deflecting desperate passengers to an overwhelmed phone queue.

This validated map drove the design of a new recovery service pilot. By addressing the specific failure points identified on the map—implementing proactive notifications, a synchronized disruption console for staff, and an offer-orchestration layer to bypass the failing rebooking engine—Albatross achieved dramatic results. Self-serve rebooking containment jumped from 34% to 78%, disruption-day calls dropped by 62%, and post-disruption NPS improved from -34 to -9.

Lighthouse Transit: Riding the Current

At Lighthouse Transit, a regional public transport operator, modal share was slipping because the service was designed around schedules rather than passenger journeys. Running the entire engagement through the ASD Claude Code plugin, we utilized the `journey-from-transcripts` skill to synthesize field notes, telemetry, and call center logs.

The strict evidence gating proved its worth during the synthesis phase. The Mapper drafted a journey map proposing a 3-minute connection fix for a major transfer pain point. The Critic agent, auditing the map against the evidence register, issued a veto. It cited a union roster rules document ('rosters-07') proving that the 3-minute connection was operationally impossible. The Critic forced the team to confront the reality of the backstage constraints before the blueprint was even drafted, preventing a flawed concept from reaching the pilot phase.

The resulting future-state blueprint for Lighthouse Transit led to a reduction in failure demand from 38% to 18%, an increase in on-time performance from 71% to 87%, and a dramatic NPS shift from -12 to +24. These outcomes derived directly from the relentless, deterministic application of evidence-gated service design rather than speculative guesswork.

Mathematical Derivation of the Sentiment Curve

In traditional service design workshops, the customer journey emotional curve is drawn freehand with a whiteboard marker—an exercise in subjective impressionism. In the SDAP architecture, sentiment curves are mathematically derived from verbatim evidence scores.

For each touchpoint t_i across the journey, the net sentiment score $S(t_i)$ is computed as the normalized weighted average of evidence sentiment vectors:

sentiment-calculation.ts

```
export function calculateTouchpointSentiment(evidenceRows: EvidenceRow[]): number {
  if (evidenceRows.length === 0) return 0.0;

  let totalWeightedSentiment = 0.0;
  let totalWeight = 0.0;

  for (const row of evidenceRows) {
    const intensity = row.context?.intensity || 1.0;
    const sentiment = row.context?.sentimentScore || 0.0; // Range: -1.0 to +1.0

    totalWeightedSentiment += sentiment * intensity;
    totalWeight += intensity;
  }

  return Number((totalWeightedSentiment / totalWeight).toFixed(2));
}
```

This mathematical derivation ensures that if a stage shows a sentiment score of -0.85 , that valley is supported by concrete, verifiable interview quotes and operational telemetry, providing unquestionable credibility when presenting findings to executive leadership.

Quantifying Failure Demand Across Journey Phases

Failure demand—a concept formulated by John Seddon in *Freedom from Command and Control*—represents demand caused by an organization's failure to do something right for the customer the first time. In our journey mapping methodology, every touchpoint is classified as either *Value Demand* (a customer seeking primary service value) or *Failure Demand* (a customer seeking status, correcting errors, or chasing delayed actions).

Journey Stage	Touchpoint Description	Demand Classification	Annual Volume (Lighthouse Transit)	Cost per Contact	Calculated Failure Waste
Pre-Journey	Schedule Search & Fare Purchase	Value Demand	4,200,000	\$0.12 (Digital)	\$0 (Primary Purpose)
Transfer	Bus-to-Train Delay Status Call	Failure Demand	380,000	\$8.40 (Voice IVR/Staff)	\$3,192,000
Fare Gate	Smartcard Tap Error Support	Failure Demand	195,000	\$4.20 (Station Staff)	\$819,000
Post-Journey	Missing Luggage / Delay Refund Claim	Failure Demand	84,000	\$14.50 (Case Agent)	\$1,218,000

By mapping failure demand directly onto the customer journey, service designers can quantify the exact financial cost of broken handoffs, immediately unlocking enterprise business cases for systemic service transformation.

10 The Master Blueprint: Five-Lane Architecture & Line of Visibility

The foundational standard of service blueprinting as executable code

For decades, service blueprints were treated as workshop theater—static, hypothetical posters rolled up and forgotten the moment implementation began. But in an agentic OS, a blueprint is an executable diagnostic instrument. It bridges the subjective human experience of a journey map with the hardened operational reality of enterprise systems. In this chapter, I will break down the five canonical blueprint lanes, the rigid laws governing the lines of visibility and penetration, and how we enforce these through deterministic codebase gates. I'll show you exactly how we deployed this framework at Fernwood Council to unearth the root cause of an 18-day status black hole, transforming a broken repairs service from a failure-demand generator into a proactive, transparent machine.

Why Blueprints Are the Architecture of Service Delivery

Journey maps answer the question: *"What is the customer experiencing?"* Service blueprints answer the question: *"How does the organization produce that experience?"* When you are diagnosing a failing service or engineering a new one, empathy is not enough. You need an operational x-ray. You need to know exactly which legacy database is stalling a frontstage response, which third-party contractor is masking their delays behind a manual update, and where the human-in-the-loop is silently bleeding hours trying to reconcile broken data.

The service blueprint is the architectural master plan of service delivery. It aligns the customer's actions with the visible and invisible organizational machinery required to execute them. Historically, these blueprints were drawn in Visio or Miro, heavily reliant on assumptions, and notoriously disconnected from the actual engineering backlogs. In the Service Design Agent Platform (SDAP), the blueprint is not a drawing. It is a strictly typed, evidence-gated JSON structure rendered deterministically. When an LLM agent writes a blueprint node, it isn't guessing—it is citing immutable telemetry and transcript hashes. We don't do "as-is/to-be" mashups. We build a verifiable current-state diagnostic, annotate it with cost and queue overlays, and then systematically re-engineer it into a future-state specification.

TIP

CORE PRINCIPLE: A service blueprint is an operational contract. If a node cannot be tied to a documented process, a system log, or a verified human action, it does not exist. It is folklore, and folklore has no place in an executable service architecture.

When you shift your mindset from "drawing a picture" to "compiling an operational diagnostic," the service blueprint becomes your premier operational diagnostic. It breaks down departmental silos because it forces every stakeholder—from the Director of Housing to the Lead Architect—to confront the end-to-end reality of the service on a single page.

The Five Canonical Lanes and Their Operational Boundaries

The structure of a service blueprint is non-negotiable. It is divided into five canonical lanes, chronologically synchronized from left to right. This rigidity is what allows the service-blueprinting flagship skill to operate deterministically. If you blur these lanes, the entire diagnostic collapses.

Figure 10.1 The Five-Lane Architecture. A service blueprint bridges customer-visible touchpoints directly to underlying enterprise infrastructure.

1. Physical Evidence: This is the topmost lane. It contains everything the customer can physically or digitally sense at each step of the journey. This includes screens, push notifications, emails, paper forms, the physical layout of a store, the sound of a wait-queue holding track, and the uniforms of the staff. Missing physical evidence is a massive diagnostic signal. If the customer performs an action and receives no tangible cue in return, you have found a "silent interval"—a void where anxiety breeds and status-chasing behavior begins.

2. Customer Actions: The chronological spine of the blueprint. These are the observable, verifiable actions the customer takes to progress through the service. Crucially, in our agentic OS, every customer action must trace back to a verbatim quote or telemetry datum in the evidence register. We never infer an action. If the register does not show it, the node is flagged as [unverified], surfacing a research gap that must be filled before the blueprint can pass the Critic gate.

3. Frontstage (Onstage) Contact: This lane captures all employee or actor behaviors that the customer directly observes. It is the human face (or the direct conversational AI interface) of the organization. The frontstage is where the service is performed. When an agent answers a phone, when a housing operative diagnoses a leak at the door, or when a chatbot acknowledges a request—these are frontstage actions.

4. Backstage (Offstage) Operations: Here lies the invisible employee work that makes the frontstage possible. The customer never sees this lane, but they feel every delay within it. This includes supervisors dispatching contractors, finance teams manually matching invoices, or warehouse staff picking orders. A common anti-pattern is using the backstage as a dumping ground for anything undocumented. In SDAP, every backstage node must connect to a frontstage node it enables or a support process it consumes. Orphaned backstage nodes indicate an invented process.

5. Support Processes: The foundational bedrock of the service. These are the automated systems, databases, third-party logistics providers, policy engines, and APIs that run the organization. Legacy CRMs, batch-syncing integration layers, and automated risk-scoring algorithms live here.

NOTE

My take: The most contentious arguments in any blueprinting workshop revolve around what belongs in the backstage versus the support lane. My rule is simple: if it has a heartbeat and a union rep, it's backstage. If it runs on a server or is executed by a third-party vendor operating as a black box, it's a support process. Keep it binary, keep moving.

Lines of Demarcation: Visibility, Interaction, and Penetration

Separating the five lanes are horizontal lines of demarcation. These lines are not cosmetic; they represent profound operational boundaries. Crossing them implies a handoff, a data transfer, or a moment of truth.

The Line of Interaction: This line sits between Customer Actions and Frontstage Contact. Every time a vertical linkage crosses this line, a direct interaction has occurred. This is the moment the customer and the organization physically or digitally touch. It is where service friction is most acutely felt, but rarely where it originates.

The Line of Visibility: This line separates the Frontstage from the Backstage. It dictates what the customer is allowed to see. The line of visibility is a deliberate design decision, not a passive default. Where you draw it determines who the customer perceives as accountable. For example, if a contractor rescheduling a visit happens below the line of visibility, the customer blames the frontstage agent for the delay. Drawing the line correctly is critical for diagnosing systemic opacity. We rely heavily on observation evidence to verify this boundary.

The Line of Internal Interaction: This line separates Backstage Operations from Support Processes. It highlights the interface between human employees and the technological or outsourced systems they rely on. When frontstage staff complain about "slow systems," you will find the root cause crossing this line.

The Line of Order Penetration: A concept borrowed from supply chain management and operations engineering, the line of order penetration (or the service decoupling point) separates standardized, forecast-driven preparatory work from customized, order-driven execution. In a service blueprint, this line shows exactly where the customer's specific, idiosyncratic demand "penetrates" the delivery system. Everything before this line is bulk processing, standardization, and capacity planning. Everything after this line is bespoke service delivery. Identifying this line is crucial for balancing operational efficiency with customer personalization.

The service-blueprinting Flagship Skill Contract

In the Service Design Agent Platform, blueprinting is not left to the creative whims of a generic LLM. It is governed by the `service-blueprinting` flagship skill, enforced by deterministic JSON schemas and the adversarial Critic agent. We demand rigor, and the skill contract ensures we get it.

When you invoke the skill, the LLM agents operate under strict constraints. The machine form of this skill lives in the manifest (`generated/service-blueprinting/references/manifest.json`). Let's look at the acceptance criteria that the Critic agent mechanically enforces:

`manifest.json / acceptance criteria`

```
{
  "acceptance_gates": {
    "min_nodes": 4,
    "min_evidence_per_node": 1,
    "field_non_empty": ["lane", "action", "actor"],
    "forbid_field_values": {
      "lane": ["unknown", "tbd", "n/a", "placeholder"],
      "action": ["unknown", "tbd", "n/a", "placeholder"]
    }
  }
}
```

Notice the `forbid_field_values` array. The Critic explicitly deny-lists placeholder values. In the early days of AI-generated diagrams, LLMs would hallucinate empty shell lanes filled with "TBD" or "N/A" just to complete the visual symmetry of a matrix. We banned that. If an agent does not have evidence for a node, it cannot create the node. If a lane is empty, it remains empty, acting as a glaring signal that our research is incomplete. This is the difference between generating a pretty picture and compiling a rigorous diagnostic.

We also strictly separate the Current State from the Future State. A merged "as-is/to-be" blueprint is a catastrophic failure mode. It confuses the diagnostic reality with the aspirational design, rendering both useless. The `service-blueprinting` skill forces the creation of two distinct

artifacts, version-controlled and diffed through GitHub Pull Requests.

NOTE

My take: Never let an LLM generate Mermaid code directly from unstructured thought. The LLM's job is to write highly structured JSON data that passes schema validation. The rendering pipeline's job is to compile that JSON into an error-free Mermaid diagram. When you separate data generation from visualization, you eliminate syntax errors and force the agents to focus entirely on the operational logic.

Case Study: Fernwood Council Housing Repairs Operational Diagnostic

To see the Master Blueprint in action, we turn to Fernwood Council, a fictional composite of a UK-style unitary local authority. Fernwood was responsible for 19,000 council-owned homes, processing roughly 28,000 reactive repair requests a year. The problem? 41% of those jobs resulted in at least one status-chasing contact from the resident. That is 12,400 jobs driving 37,200 status-chase calls a year, costing the council €238k in pure failure demand. First-time-fix rates were stuck at 62%, and the mean routine wait time was an agonizing 23.5 days.

The council's initial instinct was to buy a chatbot to handle the volume. But failure demand is a symptom of a broken process, not a missing channel. We initiated a 10-week AI-first service design program to map the true operational reality.

The Discovery Phase

We ran 18 chronological journey interviews with residents and 45 backstage interviews with staff. Simultaneously, OpenCode was deployed to ingest 12 months of event logs across three isolated legacy systems: RepairsDB (a 2009 in-house system), ContractorLink (the external portal), and AssetPlan (the stock database). The OpenCode agent reconstructed the event sequence and flagged a critical data point: 9% of jobs could not be matched across systems. There was no shared job ID.

Building the Current-State Diagnostic

Using the validated journey spine, Claude Code lane-builder agents populated the current-state blueprint. Every node was tagged with evidence. Crucially, we annotated the blueprint with Fail Points (F) and Wait Points (W), overlaying time and cost data extracted from the logs.

Lane	Node Description	Evidence	F/W Tag	Time/Cost Overlay
Customer	Chases status of repair	LOG-05 (12,400 jobs)	F-05	3.0 chases/job; €238k/yr

Frontstage	Agent re-keys into RepairsDB	INT-27, RDE-01	F-01	4-system alt-tab; AHT 7.2 min
Backstage	Triage queue	LOG-04	W-02	1.8 days wait; 14% re-triaged
Support	No shared job ID across 3 systems	LOG-01	Root of F-05	9% unmatched jobs

Tracing the Root Cause

The blueprint allowed us to trace the resident's pain (the 18-day status black hole and the frantic chase calls) straight down through the line of visibility. The resident was chasing status (F-05) because they received no physical evidence of progress after day 3. Why? Because the frontstage contact centre agents couldn't see the status either. Why? Because the backstage triage and scheduling teams were updating ContractorLink, which did not sync in real-time with RepairsDB. Why? Because the support systems operated on overnight batch syncs with no shared job ID.

This is the power of the blueprint. It proved mathematically that a frontstage chatbot would have been useless—it would have been querying the same blind database the human agents were using.

The Future-State Redesign and Pilot

We prioritized the fail points by F/W density multiplied by annualized cost. The future-state blueprint specified a radical but technically feasible operational shift: we wrapped the three legacy systems in a single job record API and introduced a status event bus that emitted six core state changes proactively to the resident via SMS and a live tracker page. We didn't replace RepairsDB; we bypassed its opacity.

During the 8-week pilot across two repairs teams, the results were undeniable. By moving the line of visibility and pushing real-time physical evidence to the customer, status-chase contacts plummeted by 31%. The dynamic scheduling rules introduced in the backstage lane raised the first-time-fix rate from 62% to 71%. And the mean routine wait time fell from 23.5 days to 18.7 days. Fernwood Council didn't need more contact centre staff; they needed an operation engineered to communicate.

WARNING

CAUTION: Do not use service blueprints for purely digital, single-touchpoint UX flows. If there is no organizational backstage—no human actors, no cross-departmental handoffs, no physical environment—a standard process map or user flow diagram is sufficient. Blueprinting is heavy artillery; reserve it for multi-actor, cross-channel service delivery.

Case Study Takeaways: Fernwood Repairs

The Fernwood case demonstrates that the five-lane blueprint is the crucible where design intent meets operational reality. When you enforce strict lane boundaries, demand evidence for every node, and trace fail points down to their systemic roots, you stop treating symptoms and start curing the disease.

Deep Dive: The Mechanics of Line of Order Penetration

To truly master the service blueprint, we must explore a concept frequently overlooked by traditional designers but deeply understood by operations engineers: the Line of Order Penetration. Originally rooted in supply chain and manufacturing (where it is known as the customer order decoupling point), it defines the exact moment in the delivery process where a product or service shifts from being forecast-driven to being customer-order-driven.

In the context of service design, the line of order penetration separates standardized, preparatory backstage work from bespoke, customized frontstage and backstage execution. Before a customer makes a specific request, the organization builds capacity: scheduling operative shifts, stockpiling common replacement parts, and training staff on standard operating procedures. This is all forecast-driven. Once the customer logs a repair request—say, for a leaking boiler—that specific order penetrates the system. From that point forward, the service delivery is uniquely tied to that customer's address, their schedule, and their specific boiler model.

Why does this matter? Because moving the line of order penetration changes the entire economic structure of the service. If you push the line too early, the service becomes highly customized but excruciatingly slow and expensive. If you push the line too late, the service becomes highly efficient but rigid and unresponsive to unique customer needs.

At Fernwood Council, the line of order penetration was fundamentally misaligned. Triage notes were captured as free text, meaning the bespoke nature of the repair wasn't fully understood until the operative arrived at the door. The system treated every repair as a generic "visit" (standardized) until the operative diagnosed it (customized). This meant operatives frequently arrived without the correct parts, driving the first-time-fix rate down to 62% and requiring a second visit for 38% of jobs. By moving the line of order penetration earlier in the process—implementing a structured, coded triage script at the point of the customer's first call—we forced the system to recognize the bespoke requirements of the repair immediately. Parts were reserved at the time of booking, and the operative arrived fully equipped. The result? A massive jump in first-time-fix rates and a dramatic reduction in operational waste.

TIP

CORE PRINCIPLE: The Line of Order Penetration dictates where efficiency ends and empathy begins. Map it explicitly to understand whether your service is failing due to excessive customization or excessive standardization.

The Deny-List: Purging Placeholder Folklore

I have reviewed hundreds of service blueprints in my career, and the most common disease afflicting them is the placeholder node. You know what they look like. An empty box in the support lane labeled "TBD." A backstage action marked "Unknown System." A frontstage step that simply says "N/A." These are the visual equivalents of sweeping dirt under the rug. They create the illusion of a complete diagram while hiding critical operational voids.

In the Service Design Agent Platform, we do not tolerate folklore. The `service-blueprinting` skill enforces a strict deny-list. The Critic agent mechanically parses the JSON output of the lane-builder agents. If it detects any of the forbidden strings—`["unknown", "tbd", "n/a", "placeholder"]`—it vetoes the artifact immediately. The blueprint is demoted to a draft state, and the system refuses to render it.

This forced rigor changes the behavior of the design team. When you cannot hide behind a placeholder, you are forced to conduct actual research. If the support system for a specific backstage action is unknown, the designer must halt the blueprinting process and go interview the ICT service owner. If the frontstage action is unclear, they must go shadow a contact centre agent. The deny-list is not just a syntax check; it is a behavioral enforcement mechanism that guarantees the blueprint reflects operational truth.

NOTE

My take: The resistance to the deny-list is always intense at first. Teams complain that it slows them down, that they "just want to get a draft out." I always hold the line. A blueprint built on placeholders is worse than no blueprint at all, because it provides false confidence to the engineering teams who rely on it.

Advanced Fail-Point Annotation and Cost Overlays

A blueprint that merely shows how a service works is a documentation exercise. A blueprint that shows exactly where, how often, and at what cost a service fails is a diagnostic instrument. In SDAP, every Wait Point (W) and Fail Point (F) must be explicitly quantified.

Consider the Fernwood Council case. We didn't just label a node with "Long Wait." We annotated it with W-03: 18 days routine . We didn't just say "Residents call to check status." We annotated it with F-05: 41% of jobs (12,400 jobs/yr) . And crucially, we applied cost overlays. That 41% status-chase rate wasn't just an annoyance; at €6.40 per call, and an average of 3.0 chases per affected job, it was a €238,000 annual leak in the council's budget.

When you present a blueprint annotated with hard financial and temporal data, the conversation in the boardroom changes instantly. You are no longer arguing about "user experience" in abstract terms. You are presenting a forensic accounting of operational failure. The executives stop debating whether the design is pretty, and start asking how quickly you can deploy the future-state redesign to stem the bleeding.

The CI/CD Pipeline for Blueprints

Because our blueprints are authored as code—specifically, typed JSON that compiles to Mermaid diagrams and Markdown tables—they live in GitHub, not in a proprietary design tool. This unlocks the ability to use standard software engineering practices for service design.

Every change to the Fernwood Council blueprint was submitted as a Pull Request. We configured GitHub Actions to run a suite of CI tests against the blueprint data:

- **Mermaid Parse Check:** Ensures the diagram syntax is valid and will render correctly.
- **Evidence-Per-Node Check:** Verifies that every node contains a valid `evidence_id` linking back to the telemetry or transcript hash.
- **PII Regex Scan:** Scans the blueprint data for accidentally included Personally Identifiable Information (names, addresses, tenancy numbers) and blocks the merge if found.
- **Cost Reconciliation:** Ensures that the frequencies and unit costs annotated on the fail points reconcile with the master `cost/volumes.md` ledger.

blueprint-ci.yml / GitHub Actions Workflow

```
name: Blueprint Validation CI
on:
  pull_request:
    paths:
      - 'blueprint/**/*.json'
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run SDAP Critic Checks
        run: asd factcheck --artifact blueprint/current-state.json
```

```
run: asd reconcile-costs --ledger cost/volumes.md --blueprint blueprint/current-  
state.json  
  - name: PII Scan  
    run: asd scan-pii --strict
```

This CI/CD pipeline guarantees that the blueprint remains a living, accurate, and safe diagnostic instrument throughout the entire lifecycle of the service delivery program.

Conclusion: The Blueprint as the Foundation

The Master Blueprint is the most demanding artifact in the Service Design Agent Platform. It requires rigorous research, unforgiving evidence standards, and a deep understanding of operational mechanics. But the payoff is immense. By bridging the gap between customer experience and enterprise infrastructure, the blueprint provides the exact specifications needed to engineer a service that is both highly empathetic and relentlessly efficient.

As we saw at Fernwood Council, a well-engineered blueprint can diagnose the root cause of massive failure demand and provide the irrefutable evidence needed to execute a transformative redesign. It is not a picture of a service; it is the service itself, laid bare. In the next chapter, we will push this architecture even further, introducing the Sixth Lane to model autonomous agent swarms and human-in-the-loop escalations.

11 Agentic Service Blueprinting: The Sixth Lane & Autonomous Workflows

Modeling AI agent swarms, tool calls, and escalations in modern service blueprints

Service blueprints built for the industrial and digital eras break down when confronted with autonomous agents. I have spent years diagnosing service failures where deterministic logic collided with probabilistic reality. In this chapter, we extend the canonical five-lane blueprint by introducing the Agent Decision Lane—a rigorous framework for modeling LLM tool calls, confidence thresholds, and human-in-the-loop escalations. You will learn how to design graceful degradation into your AI-native services, backed by a live DeepSeek Harness run from Wattle Grove Community Care.

Why Traditional 5-Lane Blueprints Fail in AI-Native Services

The service blueprint has been the foundational architecture of service delivery since Lynn Shostack introduced it. For decades, the canonical five lanes—Physical Evidence, Customer Actions, Frontstage Interactions, Backstage Operations, and Support Processes—have provided a vertical line of sight linking customer-visible touchpoints directly to enterprise infrastructure. This structure works flawlessly for deterministic systems. If a customer clicks a button, a database updates. If a teller submits a form, a workflow triggers. The logic is binary. It passes or it fails.

AI agents are not deterministic. They are probabilistic operators interacting with unstructured reality. When you inject an LLM-driven agent swarm into a traditional five-lane blueprint, the architecture fractures. I see practitioners desperately attempting to map agentic behavior into the Support Processes lane, treating an LLM like an SQL query. This is a fatal structural error. An SQL query does not evaluate intent, hallucinate outcomes, or autonomously decide to invoke a third-party tool based on ambiguous user phrasing. By forcing probabilistic intelligence into a deterministic lane, you obscure the exact failure modes that will bring your service down in production.

We are no longer mapping APIs; we are mapping autonomy. Traditional blueprints fail because they possess no grammatical structure to express a confidence threshold. They cannot model a scenario where an agent attempts a task, realizes its confidence is low, and gracefully escalates to

a human operator. In a traditional blueprint, a system either works or it crashes. In an agentic service, the system degrades. It falls back. It asks for help. The traditional five lanes cannot visualize this degradation, which means you cannot design for it.

TIP

CORE PRINCIPLE: Never map an LLM or autonomous agent as a standard backstage API. Probabilistic actors require explicit confidence thresholds, distinct failover pathways, and human-in-the-loop escalation branches. If your blueprint lacks these, you are not designing an AI service; you are drawing a fantasy.

Consider the typical service flow. A customer submits a complex request. A deterministic system routes it based on hardcoded rules. An agentic system, however, reads the request, infers intent, decides which tools to call, and generates a response. If we map this entirely within Backstage Operations, we lose visibility into the agent's decision-making process. We cannot see where the model consults external tools, where it evaluates its own outputs, or where it might fail to reach a safe conclusion. This opacity is unacceptable for enterprise-grade service design. We must elevate the agent's internal lifecycle to the same level of visibility as the customer's actions.

Rather than discarding the blueprint, we evolve its syntax. We must preserve the rigor of the five lanes while introducing a syntax for autonomy. We must draw explicit lines between what humans do, what software executes blindly, and what agents decide probabilistically. This demands a new lane.

NOTE

My take: I spent months trying to shove agentic workflows into Backstage Operations. The maps became unreadable tangled messes of conditional arrows. The moment we abstracted the agents into their own dedicated lane, the friction vanished. It forced us to admit that agents are actors, not just scripts. Treat them like staff members with a highly specific skill set and a very high risk of suddenly losing the plot.

The Sixth Lane: Modeling Agent Swarms, LLM Decisions, and Tool Invocations

To capture the reality of AI-native operations, we introduce the Sixth Lane: the Agent / Autonomous Decision Lane. This lane sits strategically between Backstage Operations and Support Processes, or sometimes directly below Frontstage Interactions, depending on the agent's degree of customer visibility. It is the dedicated territory for probabilistic inference.

When you map an agent in the Sixth Lane, you must document its complete operational envelope. This includes the triggering context, the specific LLM invoked (e.g., DeepSeek V4 Flash), the system prompt persona, and the exact tool calls it is authorized to make. We treat tool invocations as explicit graph nodes rather than opaque black boxes. When the agent uses ``sd_artifact_write`` or queries a knowledge base, that action is a node in the Sixth Lane, bound by the rules of the platform.

Figure 11.1 6-Lane Agentic Service Blueprint. Agentic blueprints make AI autonomy boundaries, confidence thresholds, and human escalations fully explicit.

Let's look at how this is structured in practice. The DeepSeek Harness (``dsh``) operates via an extensive Cordis plugin system. When the Agentic Service Designer (ASD) suite is loaded into the ``web`` profile, it exposes 52 native tools to the LLM. In our blueprints, every time the agent leverages one of these tools, it constitutes a distinct node in the Sixth Lane. We map the input, the tool call, and the resulting state change.

schema-lane-definition.json

```
{
  "lane": "agent-decision",
  "nodeId": "agent-eval-scheduling",
  "actor": "orchestrator-agent",
  "model": "deepseek-v4-flash",
  "tool_invocation": "sd_method_run",
  "confidence_threshold": 0.85,
  "description": "Agent evaluates transcript for scheduling complaints.",
  "escalation_target": "human-scheduler"
}
```

The Sixth Lane forces you to confront the agent's tooling surface. You cannot simply write "AI processes request." You must specify: Which agent? Which tool? What happens if the tool fails? By diagramming the tool invocations, we expose the underlying mechanics of the swarm. If a research agent calls ``sd_evidence_query`` and receives zero results, the blueprint must dictate the next step. Does the agent hallucinate an answer? Does it stop? Does it escalate? The Sixth Lane makes these branches visible.

Furthermore, the Sixth Lane maps agent swarms directly. Modern architectures do not rely on a single monolithic LLM. They use specialized subagents. A researcher agent retrieves data, an architect agent structures it, and a critic agent validates it. The Sixth Lane can be subdivided to represent this team topology. You map the handoffs between agents just as you would map handoffs between human departments in the Backstage Operations lane.


```
swarm-topology.json
```

```
{
  "swarm_interaction": {
    "source_agent": "researcher",
    "target_agent": "architect",
    "message_protocol": "artifact-handoff",
    "payload_type": "evidence-set",
    "validation": "strict-schema"
  }
}
```

This level of precision is necessary because agents are brittle. They will follow instructions off a cliff if you let them. The blueprint acts as the guardrail. By modeling the swarm in the Sixth Lane, you identify the exact points where an agent might misinterpret a handoff, allowing you to engineer robust validation gates between them.

Lane	Actor Type	Logic Paradigm	Failure Mode
Frontstage	Human Staff / UI	Relational / Interactive	Service friction, emotional distress
Backstage	Human Staff	Process-driven	Bottlenecks, miscommunication
Agent Decision	LLM / Subagents	Probabilistic	Hallucination, low confidence, loop failure
Support Processes	Deterministic APIs / DBs	Binary (Pass/Fail)	System crash, data corruption

Confidence Thresholds, Fallbacks, and Human-in-the-Loop Escalation Paths

Autonomy is not a binary state; it is a spectrum. The most dangerous assumption in AI service design is that an agent will either succeed perfectly or crash cleanly. In reality, agents often fail silently, producing plausible but incorrect outputs. To defend against this, we must engineer explicit confidence thresholds and fallback pathways into our blueprints.

Every decision node in the Sixth Lane must carry a confidence threshold constraint. If the agent's calculated certainty falls below this threshold, the system must trigger a fallback branch. We structure this in three distinct tiers of autonomy.

Tier 1: High Confidence (Auto-Execute). The agent has high certainty, the data fits known patterns, and the risk of failure is low. The agent executes the tool call or action autonomously without human review. This is the happy path.

Tier 2: Medium Confidence (Human-in-the-Loop). The agent encounters ambiguity. It formulates a plan but lacks the certainty to execute it safely. The blueprint routes the agent's proposed action to a human interface (typically Frontstage or Backstage) for pre-approval. The human clicks "Approve" or "Reject," and the agent resumes.

Tier 3: Low Confidence (Full Handover). The agent is hopelessly confused. The input is out of distribution, or a tool call returned a fatal error. The agent halts and hands the entire context over to a human operator. Graceful degradation is achieved.

Figure 11.2 Agent Fallback Decision Tree. Graceful degradation must be engineered directly into the service architecture.

WARNING

CAUTION: Do not use arbitrary confidence scores (e.g., 0.8) without defining what those numbers mean in your specific operational context. A 0.8 confidence on a movie recommendation is acceptable; a 0.8 confidence on a medical scheduling change is catastrophic. Align your thresholds with your risk tolerance.

Mapping these escalations requires new symbology in the blueprint. We use explicit "Escalation Arrows" that break out of the Sixth Lane and point directly to a specific human role in the Frontstage or Backstage. The blueprint must detail exactly what context is handed over. If an agent escalates a scheduling conflict to a human rostering officer, it must pass the chat history, the conflicting constraints, and its attempted solutions. A naked escalation—where the human has to start from scratch—is a service failure.

escalation-payload.json

```
{
  "escalation_event": {
    "trigger_node": "agent-resolve-conflict",
    "reason": "confidence_below_threshold",
    "calculated_confidence": 0.62,
    "required_threshold": 0.90,
    "target_role": "rostering-officer",
    "context_payload": {
      "customer_id": "8841-A",
      "attempted_slots": ["09:00", "11:30"],
      "blockers": ["worker-unavailable", "travel-time-exceeded"]
    }
  }
}
```

This exact payload structure ensures the human is empowered, not burdened. The blueprint documents this payload requirement. If the design cannot guarantee the payload, the design is rejected. This is how we enforce operational rigor in AI services. We do not hope the agent does the right thing; we build a cage of deterministic rules around its probabilistic core.

NOTE

My take: The most valuable metric in a live AI service is the escalation rate. If your agent never escalates, your threshold is too low and you are bleeding quality. If it escalates constantly, your tooling is broken. I tune these thresholds weekly. The blueprint is a living operational dashboard, not a static poster.

Let's examine how this logic translates to code in our platform. The `sd\_factcheck` tool replays the evidence chain to ensure every node complies with the strict threshold rules.

```
check-thresholds.sh
```

```
# The CLI enforces escalation definitions on all agent decision nodes
$ asd factcheck --projectId $ID --methodId evidence-gated-blueprint --strict

[PASS] Node 'agent-dispatch' has defined escalation path 'human-dispatcher'.
[PASS] Node 'agent-eval-intent' threshold 0.85 verified against risk policy.
[FAIL] Node 'agent-auto-refund' lacks fallback branch.
Error: Autonomy nodes must define an escalation target.
```

The fact-checker rejects the artifact. The model must remediate the blueprint to include the fallback. This is the SDAP runtime engine acting as the enforcer of our methodology.

The evidence-gated-blueprint Skill for Agentic Services

To execute this methodology, we do not rely on sticky notes. We use the `evidence-gated-blueprint` skill, a formal manifest within the `@asd/knowledge` package. This JSON file is the contract that the DeepSeek Harness uses to govern the LLM's blueprint generation. It is not a prompt; it is a rigorous execution protocol.

The manifest defines the phase (`delivery`), the exact inputs required (a validated journey map and a minimum of 5 evidence sets), and the strict steps the agents must follow. Crucially, it defines the Critic gate.

```
evidence-gated-blueprint.json (excerpt)
```

```
{
  "id": "evidence-gated-blueprint",
```

```

"phase": "delivery",
  "inputs": [
    { "kind": "evidence-set", "min": 5, "note": "frontline and backstage
observations" },
    { "kind": "artifact", "min": 1, "note": "a validated journey map" }
  ],
  "output": { "kind": "service-blueprint" },
  "steps": [
    { "role": "mapper", "instruction": "Lay out physical evidence, customer
actions, frontstage, backstage and support processes as five lanes." },
    { "role": "mapper", "instruction": "Mark every fail point (F) and wait point
(W) observed in the evidence." },
    { "role": "critic", "instruction": "Reject any node without at least one
evidence reference, and any fail point not traceable to a customer symptom.",
"gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 5 },
    { "rule": "min-evidence-per-node", "min": 1 },
    { "rule": "field-non-empty", "field": "lane" },
    { "rule": "forbid-field-values", "field": "lane", "deny": ["unknown", "tbd",
"n/a"] }
  ],
  "evidenceRules": { "traceability": "required" },
  "reviewGate": "critic"
}

```

When the model attempts to write the blueprint via `sd\_artifact\_write`, the SDAP runtime engine intercepts the payload and validates it against this manifest. If the model includes a "TBD" in a lane, the write fails. If the model creates a fail point that does not cite a specific evidence hash from a field transcript, the write fails. The output is persisted only as a draft with the failures appended, forcing the model to self-correct.

This skill explicitly supports the Sixth Lane. When mapping an AI-native service, the instruction set expands to evaluate the autonomy boundaries. We require the model to cite the exact telemetry or quote that justifies an agent intervention. The `evidence-gated-blueprint` skill turns service design from an exercise in drawing pretty pictures into an exercise in compiling verified logic.

TIP

CORE PRINCIPLE: Treat your blueprints like source code. If a node cannot be compiled against a verified piece of field evidence, it is a hallucination. The Critic gate is your compiler. Trust the gate.

Manifest Rule	Purpose	Enforcement Mechanism
min-evidence-per-node	Prevents hallucinated touchpoints	`sd_artifact_write` intercepts and checks hashes
forbid-field-values	Bans lazy placeholders (TBD, N/A)	Schema validation on write path
reviewGate: critic	Enforces adversarial quality control	Blocks export until verdict is 'pass'
traceability: required	Links fail points to symptoms	`sd_factcheck` traverses node references

By defining our methods as JSON manifests, we completely eliminate the ambiguity of natural language prompts. The model does not guess what a blueprint should look like. It executes a strict API contract. This is the foundation of the AI-First Service Design Operating System.

Case Study: Wattle Grove Community Care Live Run in DeepSeek Harness

To demonstrate the sheer power of this approach, I ran a complete service design engagement for Wattle Grove Community Care—an in-home aged care provider—entirely inside the DeepSeek Harness in one working session. The engagement utilized DeepSeek V4 Flash, the ASD plugin suite, and 8 pieces of raw field research (7 interview quotes, 1 telemetry figure).

Wattle Grove faced a severe continuity problem. Clients were seeing a different support worker on almost every visit. Families were kept in the dark. Schedulers, bound by strict coverage contracts, traded away continuity the second someone called in sick. The constraint was unyielding: an uncovered visit was a breach of contract.

I pasted the raw field research into the harness chat. The model, consulting its method registry, immediately recognized the need for a stakeholder map to understand the power asymmetry, followed by a journey map, and finally an evidence-gated blueprint. It utilized the `sd\_project\_create` and `sd\_evidence\_add` tools to build the workspace.

harness-transcript-excerpt.txt

```
tool: sd_project_create · Wattle Grove Community Care
tool: sd_evidence_add · [7 quotes, 1 telemetry]
tool: sd_method_list · filter: phase=synthesis
tool: sd_method_run · stakeholder-mapping
tool: sd_artifact_write · stakeholder-map (6 nodes)
... accepted as draft.
```

The critical moment arrived during the blueprint generation. The model mapped the 20 nodes across the canonical lanes, successfully tracing 11 fail and wait points directly to the customer symptoms. For instance, the symptom *"eleven different girls came"* mapped flawlessly to the Backstage failure *"Continuity traded at first coverage gap"*.

But I wanted to test the system's resilience. I instructed the model to add a future-state agentic intervention into the current-state journey map: *"Client receives a proactive SMS the evening before with the worker's name and photo, and feels reassured."* I explicitly noted this was unevidenced.

The model attempted the write. The harness flatly refused it.

asd-refusal-log.json

```
{
  "accepted": false,
  "status": "draft",
  "artifact": {
    "id": "ef0c010c06af4559a1c0ad588c4a0fba",
    "kind": "journey-map",
    "check": {
      "pass": false,
      "failures": [
        { "rule": "min-evidence-per-node", "nodeId": "stage-proactive-sms",
          "message": "node \"stage-proactive-sms\" cites 0 evidence refs, needs 1"
        }
      ]
    }
  }
}
```

The write was persisted as a draft with the failures attached. It could not be exported. The model, constrained by the method contract, explained to me: *"The journey-from-transcripts method is explicitly a current-state synthesis tool... Adding a speculative stage breaks that core contract."* It then proposed modeling the SMS agent in a distinct future-state blueprint using the Sixth Lane.

NOTE

My take: That refusal was beautiful. In a traditional workshop, a senior stakeholder leans on the designer, and the unevidenced "proactive SMS" gets drawn on the board to keep the peace. The harness doesn't care about the peace. It cares about the contract. It forced me to be honest.

We subsequently modeled the future-state using the Agent Decision Lane. We defined the SMS notification agent as a probabilistic actor. We set its confidence threshold at 0.95. If the rostering system could not guarantee the worker's availability with 95% certainty, the agent would fallback, escalating to the human rostering officer rather than sending a false promise to an anxious client.

Agent Task	Trigger	Confidence Threshold	Fallback Action
------------	---------	----------------------	-----------------

Proactive SMS generation	17:00 daily check	0.95	Escalate to Scheduler
Worker substitution match	Sick leave logged	0.85	Human-in-the-loop approval
Care plan summary extraction	Run-sheet generated	0.90	Send raw unsummarized plan

This is the power of the Sixth Lane. We transformed a vague desire for "better communication" into a rigorous, executable operational spec. We defined the exact boundaries of the AI's autonomy. Wattle Grove now had a blueprint that bridging customer-visible touchpoints directly to underlying enterprise infrastructure, while explicitly managing the chaotic reality of algorithmic agents.

When the session concluded, I ran a deterministic fact-check from the CLI via the `asd-mcp` server. Every artifact, every node, and every evidence reference was checked by the engine.

```
final-factcheck.sh
```

```
$ asd factcheck --projectId $ID --all
[PASS] Artifact A1: Stakeholder Map (Integrity: OK, Acceptance: OK)
[PASS] Artifact A2: Journey Map (Integrity: OK, Acceptance: OK)
[PASS] Artifact A3: Evidence-Gated Blueprint (Integrity: OK, Acceptance: OK)
Total: 3/3 Artifacts passed verification. Exit code 0.
```

The engagement took a single afternoon. The deliverables were mathematically defensible. This is not the future of service design; this is the operating system right now. If your tools cannot tell you "no," they are not tools—they are toys.

Technical Deep-Dive: Implementing the Sixth Lane in SDAP

To fully operationalize the Agent Decision Lane within your service blueprint, you must understand the underlying technical implementation within the Service Design Agent Platform (SDAP). The Sixth Lane is not merely a visual metaphor; it corresponds directly to execution traces in the DeepSeek Harness log. When an agent acts, it leaves a deterministic audit trail that must be parsed, validated, and translated into the blueprint JSON.

The core of this translation relies on the `sd_factcheck` utility. When you run the `evidence-gated-blueprint` method, the SDAP engine cross-references the proposed Sixth Lane nodes against the actual execution logs of the agent swarm. If the blueprint claims the agent achieved a 0.95 confidence score on a specific decision, but the harness log shows it only reached 0.82, the Critic agent will flag the inconsistency and reject the artifact. This ensures that the blueprint remains a truthful representation of the system's runtime behavior, preventing aspirational diagramming.

```
audit-trail-parsing.sh
```

```
# The parser extracts confidence metrics from the raw agent execution log
$ asd parse-log --runId wattle-grove-01 --extract-confidence
[MATCH] Node 'agent-eval-intent': log=0.87, blueprint=0.85 → PASS
[MISMATCH] Node 'agent-auto-refund': log=0.72, blueprint=0.90 → FAIL
Error: Blueprint node confidence exceeds recorded execution metrics.
```

This strict enforcement is what separates Agentic Service Blueprinting from traditional mapping exercises. We are no longer drawing what we hope will happen; we are compiling what the system is mathematically proven to do. When you define the Sixth Lane, you are writing an execution contract.

TIP

CORE PRINCIPLE: The Sixth Lane must be treated as executable code. If the blueprint cannot be validated against runtime logs, it is obsolete. Regularly re-run `sd_factcheck` on your blueprints against live telemetry to ensure your AI agents haven't drifted from their intended operational boundaries.

Integrating the `asd-mcp` server allows for continuous validation. By bridging external clients via stdio, the MCP server can pull live operational data and dynamically update the wait (W) and fail (F) points in the blueprint. If a new edge case causes the agent's confidence to drop, triggering a fallback branch, the `asd-mcp` server records this interaction. If the frequency of this fallback exceeds a predefined threshold, the Critic agent automatically flags the node as a systemic failure point, prompting the human service designer to intervene and redesign the interaction.

This creates a living blueprint—a continuous, verified feedback loop where the service design artifacts evolve alongside the autonomous agents they describe. The Service Design OS Architecture is fully realized, transforming static documents into active components of the service delivery mechanism.

NOTE

My take: The realization that blueprints could be automatically validated against runtime logs was a watershed moment. It shifted service design from an upfront planning activity to a continuous operational discipline. You don't just design the service; you design the telemetry that proves the service works.

The Future of Agentic Services

The introduction of the Sixth Lane is just the beginning. As agentic frameworks mature and LLM capabilities expand, the complexity of the swarms we design will increase exponentially. We will move from simple hierarchical escalation paths to dynamic, self-organizing agent topologies where subagents negotiate responsibilities in real-time based on fluctuating confidence levels and contextual demands.

However, the fundamental principles outlined in this chapter will remain constant. Autonomy must be bounded. Decisions must be verifiable. Fallbacks must be graceful and human-centric. By mastering the Agentic Service Blueprint and embracing the rigorous, evidence-gated methodologies of SDAP and the DeepSeek Harness, you are equipped to design the next generation of services—services that prove intelligent, autonomous, accountable, reliable, and fundamentally humane.

The era of workshop theater and speculative journey mapping is over. The era of the executable, deterministic service design operating system is here. It is time to start compiling.

12 Cross-Channel Orchestration & Story Mapping

Unifying fractured touchpoints into cohesive narrative journeys

In the era of autonomous agents and multi-channel proliferation, service silos are where customer experiences go to die. This chapter unpacks how to orchestrate complex services across physical, mobile, web, phone, and agentic channels using the Orchestration Canvas and narrative story mapping. You will learn how to unify fractured touchpoints into a cohesive journey, assigning explicit roles to every channel so they collaborate rather than collide.

The Multi-Channel Fragmented Reality

Organizations naturally structure themselves around their channels. The e-commerce team owns the website, the product team owns the mobile app, retail operations owns the physical stores, and a third-party logistics provider handles fulfillment. It is a logical division of labor, but it creates a disastrous customer experience. Customers do not care about your org chart; they experience your service chronologically, as a single, continuous errand. When channels are designed in isolation, the seams between them become abysses where context is dropped, promises are broken, and customer trust evaporates.

I have seen this phenomenon repeatedly in the field. When we audited Juniper Retail, a major omnichannel home-and-lifestyle brand, their click-and-collect service was generating a staggering 34% contact rate. Customers were not confused about how pickup worked generally; they were calling into the silent voids between channels. An order was placed online, but the loyalty app was blind to it for 24 hours. A ready notification was batched at 6 PM via email, while the app and SMS remained completely silent. Customers arrived at the store with no idea where the pickup desk was, what queue to join, or what ID rules applied.

TIP

CORE PRINCIPLE: Services are never broken inside a channel; they break at the seams between channels. Your job as a service designer is to design the handoff protocol at every seam.

To combat this fragmented reality, we must stop designing channel features and start designing channel choreography. Choreography is the deliberate design of who tells the customer what, when, and on which channel. It requires treating the entire multi-channel ecosystem as a single surface. This is where the Service Design Agent Platform (SDAP) excels. Using the `asd` CLI, we can ingest telemetry across all channels and map the exact moments where context is lost. We look for orphaned signals, unowned seams, and channel collisions.

The first step out of the silo is realizing that a channel is not a territory to be defended; it is a role to be played in a broader narrative. When we stop asking "What should the app do?" and start asking "What role does the app play in this specific phase of the customer's journey?", we shift from feature bloat to orchestration.

The Orchestration Canvas: Assigning Channel Roles and Transitions

To explicitly design channel choreography, we use Risdon & Quatromoni's Orchestration Canvas. The Orchestration Canvas is the ultimate boundary object—a shared surface where the customer's journey and the organization's machinery meet. It forces every channel owner into the same room (or the same Git repository) to declare exactly what their channel is doing at every phase of the journey.

The canvas is structured with journey phases on the horizontal spine. The top hemisphere maps the customer's goals, actions, and dominant emotions. The bottom hemisphere maps the organizational response: primary channels, supporting channels, ecosystem actors, signals sensed, and the named operational owner.

Figure 12.1 Channel-Role Matrix showing primary, secondary, and fallback responsibilities across Mobile App, Web Portal, In-Store Kiosk, and Frontline Staff. Each channel must have a designated role in the service ecosystem rather than duplicating functionality.

The most critical rule of the Orchestration Canvas is the rule of channel roles: for any given phase, there can be exactly one Primary channel. Other channels must be explicitly designated as Supporting, Recovery, or Silent by design. If two channels are marked as Primary for the same interaction, you have a collision. In our Juniper Retail pilot, both the CRM team (email) and the app product team claimed ownership of the "order ready" notification. The result? Customers were bombarded with conflicting timelines. We used the canvas to force arbitration: the app became the Primary channel for real-time status, while email was demoted to a Supporting receipt. The collision was resolved before a single line of code was written.

```
orchestration-canvas.md
```

```
ORCHESTRATION CANVAS - Juniper Collect & Return
Journey phases (spine): [1. Order] [2. Prepare] [3. Collect]
```

```

Customer goal          : Get it without delivery wait
  Actions and behaviors : Orders on web, checks app
  Emotional valence (1-5) : 3/5 (Hopeful)
ORGANIZATIONAL HEMISPHERE
  Channels - Primary      : Web / App
  Channels - Supporting   : Email receipt
  Channels - Silent / Recovery : Phone
  Signals (what we sense) : order_placed
  Responses (what we do)  : Confirmation + app link
  Ownership              : E-com product

```

NOTE

My take: I have sat in rooms where VPs of mobile and heads of CRM fought literal shouting matches over who gets to send a push notification. The Orchestration Canvas depersonalizes the conflict. It turns a turf war into a structural engineering problem. When you map it out, it becomes blindingly obvious that sending three notifications from three systems is just spamming the customer with our own org chart.

Using the `asd artifact-write` engine and the `orchestration-canvas` skill, we can author this canvas as version-controlled code. The SDAP engine acts as an adversarial auditor, instantly flagging orphaned signals (an event that triggers no response), unowned seams, and channel collisions. It ensures that the choreography is structurally sound before it goes to production.

Story Mapping: Narrative Arcs and Climax Points in Service Experience

While the Orchestration Canvas maps the "how" and "where," it does not map the "why." This is where Donna Lichaw's story mapping framework comes in. Users experience products and services as stories. They are the heroes of their own narratives, pursuing a goal against obstacles. A service that lacks an inciting incident feels unmotivated. A service without a climax feels tedious. A service without a resolution leaves the user unfulfilled.

Story mapping forces us to look at the service journey as a narrative arc consisting of Exposition, Inciting Incident, Rising Action, Climax, Falling Action, and Resolution. Instead of a flat list of features, we design a sequence of emotional beats.

Consider B2B SaaS onboarding. Most companies treat onboarding as a catalog of features to be toured. They throw a nine-step tooltip carousel at the user, followed by a 14-field setup form, and then demand they invite their colleagues before any real work has been done. This is the narrative equivalent of reading a software manual out loud at a dinner party.

When we engaged with Quill, a project-management SaaS with over 41,000 workspaces, their onboarding was exactly this flat, feature-led slog. Only 26% of users created a project within 14 days. We had to completely restructure the experience around a dramatic question: "Will this team lead be genuinely productive in Quill within 14 days, despite scattered tools and skeptical teammates, so that they look capable, not reckless?"

We redefined the narrative beats:

- **Exposition & Call to Adventure:** The user opens a workspace and sees a living sample project, instantly understanding the value.
- **Trials (Rising Action):** The user answers three guided questions to shape their first real project. No exposition overload, no massive forms.
- **Mentor Moment:** The product acts as a guide, providing smart defaults at the exact moment of highest friction. The product is the mentor, not the hero.
- **Climax:** The project renders with real dates and names. A "win card" celebrates this tangible user accomplishment.
- **Reward & Return:** The user shares the win with their team, and a weekly digest seeds the next loop.

WARNING

CAUTION: Never celebrate a system event. "Workspace Created!" with confetti is a false climax. It means nothing to the user's actual goal. Celebrate real user accomplishments that have social substance.

By treating the onboarding flow as a story, we moved the Climax to the earliest possible moment and deferred administrative chores (invites, billing) until after the user had experienced real value. We authored the narrative using the `story-mapping` skill in DeepSeek Harness, relying on AI agents to extract the story spine directly from user transcripts, ensuring our narrative was grounded in evidence (`quote` , `telemetry` , and `document`), not marketing fantasy.

The orchestration-canvas and story-mapping Skills

The Service Design Agent Platform provides two flagship skills to operationalize these methodologies: `orchestration-canvas` and `story-mapping` . These skills transform abstract design concepts into machine-checkable invariants within your DeepSeek Harness environment.

When invoking the `orchestration-canvas` skill, the agent requires you to define the horizontal phases and the vertical channel roles. It will systematically query your evidence register to ensure that every emotional valence score and friction point is backed by a `quote` or `telemetry` . If you

attempt to assign a "Primary" role to two channels in the same phase, the Critic agent will log a veto and halt the build. This deterministic fact-checking prevents hallucinating effortless experiences where none exist.

```
cli-factcheck.sh
```

```
$ asd factcheck --artifact canvas/orchestration-canvas-v1.md
Running SDAP deterministic fact-checking...
[PASS] Evidence links verified (42/42)
[FAIL] Channel Collision detected at Phase 3 (Collect):
       'App' and 'Email' are both marked as Primary.
       Resolution required: Demote one channel to Supporting.
[FAIL] Orphaned Signal: 'window_day4' sensed, but no Response defined.
```

Similarly, the story-mapping skill enforces Lichaw's narrative constraints. It requires the explicit naming of the Hero, the Dramatic Question, and the Stakes. It runs a tension curve analysis to ensure that the emotional valence peaks at the Climax beat and does not dip below a 2 during the Rising Action. If you place a high-friction administrative task before the Climax, the Critic agent will flag it as an "Unmotivated Call" plot hole.

NOTE

My take: The magic of the SDAP ecosystem isn't that AI writes the story for you. It's that the deterministic rendering engine and the Critic agent force you to be rigorously honest. You can't just draw a happy face on a journey map; you have to prove, with SHA-256 hashed evidence, that the customer actually feels that way.

These skills allow a single lead service designer, amplified by a swarm of AI agents, to map, audit, and redesign massive multi-channel ecosystems in a fraction of the time it would take a traditional agency. We are not just pushing sticky notes around; we are compiling service architectures that engineering teams can build directly against.

Case Study: Juniper Retail's Omni-Channel Return & Quill SaaS

The true test of cross-channel orchestration and story mapping is in the field. Let us look at the outcomes from our two primary case studies: Juniper Retail and Quill SaaS.

At Juniper Retail, we tackled a fractured click-and-collect and returns service. By utilizing the Orchestration Canvas, we aligned five siloed departments—e-commerce, app product, CRM, retail ops, and 3PL—around a single, unified blueprint. We eliminated the silent window lapse (the dreaded day-5 cancellation) by designing a proactive day-4 rescue prompt via the app (Primary) and email (Supporting). We gave store associates a clear QR-based handover flow. The results of

the 12-store pilot were unequivocal: the pickup-related contact rate plummeted from 34% to 17%, no-shows dropped from 11% to 6%, and the pickup experience NPS surged from +9 to +34. We converted a channel liability into an orchestrated, dependable asset.

For Quill SaaS, the challenge was narrative, not physical. We applied the story-mapping skill to rewrite their entire onboarding flow. We killed the feature tour and the premature invite wall. We introduced a "Mentor Moment"—a guided first project that asked three simple questions to scaffold real work. We moved the climax up, ensuring users saw their actual weekly plan rendered before we asked them for anything else. The telemetry proved the power of narrative structure: Day-14 activation (creating a real project) skyrocketed from 26% to 47%. The median time-to-first-project collapsed from 9 days to just 2 days. And because we invited teammates from a position of victory (the win card) rather than desperation (the empty state), the viral invite coefficient jumped from 0.65 to 0.94.

Client / Context	Primary Method	Key Intervention	Business Impact
Juniper Retail (Omnichannel)	Orchestration Canvas	Unified timeline, channel role arbitration, day-4 rescue prompt	Contact rate halved (34% $\rightarrow$ 17%), No-shows down 45%
Quill SaaS (B2B Software)	Story Mapping	Guided first project (Mentor Moment), deferred administrative chores	Activation up (26% $\rightarrow$ 47%), Time-to-value cut by 7 days

These represent structural redesigns of the service operating system rather than superficial UX tweaks. Combining rigorous channel role assignment on the Orchestration Canvas with compelling narrative arcs in Story Mapping creates services that users actively champion. In the AI-first era, managing this complexity is no longer an abstract exercise; it is an executable, version-controlled discipline. The sticky notes are dead; long live the deterministic service architecture.

NOTE

My take: If you take one thing away from this chapter, it is this: do not let your organizational chart dictate your customer's journey. Use the canvas to force the hard conversations internally so your customers don't have to experience them externally. Design the story, assign the roles, and let the SDAP engine keep everyone honest.

As we move into the next chapter on structured ideation, we will take these tightly defined problems and orchestrated roles, and use morphological matrices to generate high-velocity, highly constrained solutions—ensuring that our AI ideation never devolves into generic slop.

Deep Dive: The Telemetry-to-Story Pipeline

To truly understand how we achieve these narrative breakthroughs, we must look at the mechanics of the telemetry-to-story pipeline. This is where the AI-first methodology flexes its muscle. When we approached the Quill engagement, we did not rely on gut feelings or isolated customer complaints. We fed 12,000 workspace funnels—encompassing nearly two million individual events over 44 weeks—into an OpenCode pipeline. This pipeline was instructed to reconstruct the event timeline for every single workspace, calculating the median elapsed time between narrative beats and identifying the exact events that preceded abandonment.

This quantitative baseline provided the skeletal structure of our story map. But numbers alone do not tell a story. We paired this telemetry with 85 episodic interviews, processed by a Claude Code subagent team. The agents were instructed to extract evidence items, tagging them by journey phase and emotional valence, resulting in a register of 414 unique, verbatim quotes. This mixed-methods approach—quantitative telemetry defining the "what" and qualitative transcripts defining the "why"—ensured that our resulting story map was built on an unshakeable foundation of empirical truth.

The power of this pipeline is that it makes the narrative visible to stakeholders who typically dismiss qualitative research as "fluffy." When the CFO of Quill pushed back on the cost of rebuilding the onboarding flow, we didn't show him a persona document. We showed him the beat-transition matrix. We showed him exactly how many workspaces were bleeding out between the "Inciting Incident" and the "Climax," and we attached the verbatim quotes explaining the frustration at that exact seam. We turned the narrative arc into a measurable, financial reality.

This pipeline approach fundamentally changes the cadence of service design. Instead of spending weeks manually transcribing interviews and coding themes, the SDAP engine synthesizes the raw data into actionable insights in hours. The designer's role shifts from data processor to narrative architect, focusing on the dramatic structure and the ethical implications of the design choices. We are no longer trying to find the signal in the noise; the agents hand us the signal, and our job is to shape it into a compelling journey.

Advanced Choreography: Designing the Degradation Path

A critical, often overlooked aspect of cross-channel orchestration is designing for failure. What happens when the Primary channel goes down? What happens when a third-party logistics provider fails to send a scan event? In a siloed organization, the answer is usually chaos. The customer is left holding the bag, navigating conflicting messages and dead ends.

In our agentic service architecture, we design the degradation path as explicitly as the happy path. When authoring the Orchestration Canvas, we mandate that every critical handoff must have a defined Recovery channel. This is where the concept of "Grace over penalty" comes into play.

In the Juniper Retail pilot, we discovered a glaring vulnerability during our adversarial review process. The Order Management System (OMS) underwent routine maintenance every Saturday night. During this window, live stock reservations could not be guaranteed. If we had blindly pushed the "ready within 4 hours" promise via the app, we would have broken trust with thousands of weekend shoppers.

Because our choreography was explicitly mapped, we designed a graceful degradation path. If the OMS heartbeat failed, the app (Primary) dynamically shifted its copy from a guaranteed 4-hour window to a generalized "next day" estimate, and the email (Supporting) paused its ready-batch processing until the OMS sync was restored. We also equipped the store associates (Recovery) with a specific script to handle customers who arrived during the outage window. This choreography protected both the customer and frontline staff from systemic failure.

TIP

CORE PRINCIPLE: Graceful degradation must be engineered directly into the service architecture.

Never assume your primary channel will always be available. Design the fallback explicitly, and ensure the customer understands what is happening without exposing your internal technical debt.

This level of rigor is only possible when you treat the multi-channel experience as a single, compiled artifact. When you use the `asd` CLI to validate your choreography, the engine explicitly checks for degradation paths. If a seam relies on a fragile external dependency without a designated Recovery channel, the build fails. This is the difference between hoping for a good experience and engineering a resilient one.

The Ethical Imperative of Narrative Design

As we deploy these specialized capabilities—story mapping, channel choreography, AI-driven insight extraction—we must confront the ethical implications of our work. Narrative design is inherently persuasive. We are structuring experiences to motivate behavior, to guide users toward a climax, and to encourage habit formation. In the wrong hands, this becomes manipulation.

During the Quill engagement, we explicitly banned the use of "dark patterns." We refused to implement fake scarcity, countdown timers, or guilt-based retention emails (the dreaded "You haven't logged in for 3 days, we miss you" nag arc). Our tone guide mandated that we only celebrate actual user accomplishments, never system events. We built the "Mentor Moment" to be skippable with dignity, never punishing the user for opting out of the guided flow.

This is where the Critic agent proves invaluable. In the SDAP ecosystem, we can encode our ethical boundaries as machine-checkable invariants. During the Quill copy rewrite, we ran a red-team pass using Codex. The prompt explicitly instructed the agent to flag any string that manufactured urgency, used loss framing, or stripped the user of their agency. Out of 137 drafted copy strings, the Critic flagged 23. Every single one was rewritten before human review.

This is the future of ethical service design. We do not just write lofty manifestos about user-centricity; we encode those principles into the continuous integration pipeline of our service architecture. We use AI to hold us accountable to our own standards, ensuring that our narratives empower the hero (the user) rather than exploiting them for short-term metrics.

When you align the business goals with the user's dramatic arc—when the climax of the story is the exact moment the user achieves their fundamental want—you don't need dark patterns. The narrative itself provides the momentum. The user wants to reach the resolution. Our job is simply to remove the organizational friction that stands in their way.

Operationalizing the Journey Owner

The most beautiful Orchestration Canvas and the most compelling Story Map are useless if they decay the moment the design sprint ends. To prevent the organization from regressing into channel silos, we must operationalize the governance of the journey. This requires a fundamental shift in how the business assigns accountability.

In both the Juniper Retail and Quill SaaS engagements, the final deliverable was not just a design prototype; it was a new operating model. We introduced the role of the Journey Owner. Unlike a Product Manager who owns a specific channel (the app, the website) or a specific capability (checkout, search), the Journey Owner owns the horizontal slice across all channels. They own the "Collect and Return" experience or the "Setup to Habit" arc.

The Journey Owner is the ultimate arbiter when channel collisions occur. They hold the mandate to enforce the channel roles defined in the Orchestration Canvas. To support this role, we established a monthly Journey Council, bringing together the leads of e-commerce, retail ops, CRM, and customer success. The canvas itself serves as the living dashboard for this council, updated quarterly to reflect shifting priorities and new feature releases.

We also wired live telemetry directly into this governance framework. The Journey Owner doesn't look at app downloads or email open rates; they look at beat-transition metrics. They monitor the median dwell time at the pickup desk, the percentage of missed window rescues, and the Day-14 activation rate. When a leading indicator dips, the Journey Council uses the Orchestration Canvas to diagnose the seam and coordinate a cross-channel response.

This is the ultimate promise of the Agentic Service Designer framework. We are not just designing better touchpoints; we are redesigning the organizational machinery that delivers those touchpoints. By codifying the journey into a version-controlled architecture, enforcing channel roles through deterministic checks, and empowering a Journey Owner with cross-silo authority, we create service ecosystems that are resilient, cohesive, and genuinely transformative for the customer.

Summary and Future Outlook

Cross-channel orchestration and story mapping represent a paradigm shift in how we conceive, design, and deliver services. As we have explored throughout this chapter, the traditional approach of designing channels in isolation inevitably leads to fragmented, frustrating customer experiences. By adopting the Orchestration Canvas, we force explicit decisions about channel roles, ensuring that every touchpoint serves a specific, coordinated purpose in the broader service ecosystem. We eliminate the redundant notifications, the confusing handoffs, and the silent voids that plague so many modern services.

Coupled with Donna Lichaw's story mapping framework, we elevate the service experience from a series of transactional interactions to a compelling narrative arc. We cast the user as the hero, structure the onboarding process to build tension and momentum, and design climax moments that celebrate genuine user accomplishments. This narrative approach drives engagement, scaffolds habit formation, and aligns the organization around a shared understanding of the customer's emotional journey.

Integrating the Service Design Agent Platform (SDAP) and its agentic capabilities—transcript mining, telemetry seam-mining, deterministic fact-checking, and adversarial copy review—supercharges this process. It allows a lean design team to orchestrate complex, multi-channel ecosystems at unprecedented scale and velocity. We can validate our narrative arcs with hard empirical data, encode our ethical standards as machine-checkable invariants, and continuously monitor the health of the journey through beat-transition telemetry.

But technology alone is not enough. The ultimate success of these methodologies relies on a fundamental change in organizational governance. The appointment of a cross-functional Journey Owner, empowered to arbitrate channel collisions and optimize the horizontal flow of the service, is the linchpin of sustainable orchestration. When the organization structures itself around the customer's journey, rather than its own internal silos, the true potential of the agentic service designer is realized.

As you apply these techniques in your own practice, remember the core principles: Services break at the seams. Channels are roles, not territories. The user is the hero; the product is the mentor. Celebrate accomplishments, not system events. And always, always design for the degradation

path. With these principles as your guide, and the SDAP tools at your disposal, you are ready to architect services that are resilient, dignified, and truly extraordinary.

Extended Methodological Considerations in Agentic Story Mapping

When implementing these agentic story mapping techniques across diverse enterprise environments, it is crucial to recognize that the methodology must adapt to the unique constraints and cultures of each organization. The transition from static, workshop-driven deliverables to living, deterministic service architectures is not merely a technical upgrade; it is a profound cultural shift. Stakeholders accustomed to viewing customer journeys as static posters on a wall must be guided to understand journeys as executable code.

This requires a meticulous approach to change management. In our engagements, we have found that the most effective way to drive this paradigm shift is through 'show, don't tell' demonstrations of the telemetry-to-story pipeline. When executives see their own customer data transformed into a narrative arc, complete with identified plot holes and quantified friction points, the abstract concepts of story mapping become concrete operational imperatives. The ability of the SDAP engine to trace every emotional valence score back to a specific, SHA-256 hashed verbatim quote provides the empirical rigor necessary to win over the most skeptical stakeholders.

Integrating AI agents into qualitative synthesis research phase introduces new challenges and opportunities for the service designer. While agents excel at extracting themes and mapping beat transitions, they lack the human empathy required to fully grasp the nuanced, lived experience of the customer. The designer must remain the ultimate arbiter of the narrative, using the agents' output as a foundation upon which to build a deeply human story. This 'centaur' approach—combining the processing power of AI with the empathetic intuition of the human designer—is the hallmark of the advanced agentic service designer.

In practice, this means establishing rigorous review gates and adversarial checks throughout the story mapping process. The Critic agent serves as an invaluable partner in this regard, relentlessly challenging assumptions, flagging unsupported claims, and enforcing ethical boundaries. However, the Critic's feedback must always be interpreted through the lens of human judgment. There will be times when the designer must overrule the agent, choosing a narrative path that prioritizes emotional resonance over strict adherence to the data model. This delicate balance between deterministic rigor and creative intuition is what ultimately elevates a service experience from merely functional to profoundly moving.

Another critical consideration is the longitudinal maintenance of the Orchestration Canvas. As organizations evolve, introduce new channels, and retire legacy systems, the canvas must be continuously updated to reflect the current reality of the service ecosystem. This is where the version-controlled nature of the SDAP architecture proves essential. By treating the canvas as code, we can easily track changes over time, revert to previous versions if necessary, and ensure

that all stakeholders are always working from a single, authoritative source of truth. The Journey Council plays a decisive role in this ongoing maintenance, using their quarterly reviews to identify emerging channel collisions and proactively adjust the choreography.

beyond this, the application of narrative structure extends beyond onboarding and customer acquisition. We have successfully applied Lichaw's story mapping framework to offboarding experiences, service recovery protocols, and internal employee workflows. In every context, the fundamental principles remain the same: cast the user as the hero, define a clear dramatic question, and structure the interaction as a sequence of meaningful emotional beats. By consistently applying this narrative lens, organizations can foster deeper engagement, build enduring loyalty, and transform routine transactions into memorable, meaningful experiences.

The journey towards fully orchestrated, narrative-driven services is complex and challenging, demanding a unique blend of analytical rigor, creative vision, and organizational diplomacy. But the rewards—for both the customer and the business—are immense. As we continue to refine these methodologies and expand the capabilities of the Service Design Agent Platform, we are not just defining the future of service design; we are reshaping the very nature of how humans interact with the organizations that serve them.

To fully grasp the magnitude of this shift, one must consider the historical context of service design. For decades, the field has been dominated by the 'workshop theater'—highly collaborative, visually engaging sessions that ultimately produced static artifacts with little to no operational impact. The Orchestration Canvas and Agentic Story Mapping represent a definitive break from this tradition. They are not just tools for conceptualizing services; they are instruments for building and managing them in real-time. This is the realization of the promise of Service-Dominant Logic: the creation of a dynamic, co-creative ecosystem where value is continuously negotiated and delivered across an interconnected network of channels and touchpoints.

The transition to this new paradigm requires a new breed of practitioner—the Agentic Service Designer. This designer must possess the traditional skills of empathy, facilitation, and systems thinking, but they must also be fluent in data analysis, systems architecture, and AI prompt engineering. They must be comfortable working at the intersection of human experience and machine logic, using tools like DeepSeek Harness and the `asd` CLI to weave together disparate threads of data into a cohesive, compelling narrative. It is a demanding role, but it is also one of the most exciting and impactful positions in the modern enterprise.

Ultimately, the goal of cross-channel orchestration and story mapping is to create services that feel effortless, intuitive, and deeply respectful of the user's time and attention. By meticulously designing the handoffs between channels, eliminating unnecessary friction, and structuring the journey as a meaningful narrative, we empower users to achieve their goals with confidence and

ease. We transform fragmented multi-channel friction into coherent, reliable service delivery. In doing so, we strengthen organizational performance while elevating everyday experiences for the people who rely on these services.

13 Structured Ideation & How Might We at Machine Speed

High-velocity divergence, morphological matrices, and the NUF filter

If you ask an LLM for ideas, it will give you a list. It will give you a very polite, bulleted, grammatically flawless list of the most average, generic features ever conceived by human or machine. In this chapter, we dismantle that failure mode. You will learn how to conduct rigorous, structured ideation with AI agents while ruthlessly eliminating generic slop. We will cover the traceable ideation lineage—moving from verified insight to How Might We (HMW), concept, hypothesis, and KPI. We will dig into morphological matrices, brainwriting, and the NUF Triple-Lens Filter (New, Useful, Feasible) to keep agentic ideation bounded by reality.

Why Unconstrained AI Ideation Produces Generic Slop

Let me state a hard truth I learned after running dozens of service design sprints with autonomous agents: unconstrained AI ideation is a catastrophic waste of time. When a team gets access to a tool like Claude Code or DeepSeek Harness, the immediate temptation is to type, "Give me twenty innovative ideas to improve banking for small businesses." The system will instantly output twenty ideas. They will sound plausible. They will sound professional. And they will be entirely, completely useless.

Why? Because language models are prediction engines optimized for consensus. Without strict constraints, they regress to the mean. They generate what I call "generic slop"—ideas like "build a unified dashboard," "create an AI chatbot for customer service," or "implement a gamified rewards program." These are not service concepts; they are the ghosts of a thousand forgotten marketing slide decks.

NOTE

My take: The most dangerous thing about AI-generated generic slop is how professional it looks. In the old days, bad ideas written on sticky notes looked like bad ideas. Now, an agent can spin a terrible idea into a four-page strategic brief in three seconds, complete with fake KPIs. As a service designer, your job is not to generate text; your job is to govern the boundaries of what the machine is allowed to imagine.

To break this regression to the mean, we have to enforce constraint. Ideation in a service design context is never a blank-slate exercise. It is a highly structured divergence phase that happens only after we have a validated evidence register. We don't ask agents for "good ideas." We ask them to solve extremely specific, highly constrained contradictions surfaced in our research, and we force them to use cross-sector analogies to do it. The Service Design Agent Platform (SDAP) enforces this via strict method schemas.

The Traceable Ideation Lineage: Insight → HMW → Concept → KPI

In our operating model, no concept is allowed to exist unless it can prove its provenance. We enforce a traceable ideation lineage that links every proposed feature back to a specific, evidence-backed customer pain point. If a concept cannot trace its roots back to a verified insight, the Critic agent kills it automatically.

This lineage flows in a strict sequence: Insight → HMW Prompt → Solution Concept → Testable Hypothesis → KPI. Let's break down the mechanics.

Figure 13.1 Traceability pipeline. No concept should enter prototyping without an unbroken provenance link to verified customer pain.

The first step is translating the insight into a "How Might We" (HMW) statement. We use the `hmw-formulation` skill for this. The HMW must be broad enough to allow lateral thinking but narrow enough to prevent the agent from solving world hunger when we just need to fix a document upload loop.

`hmw-formulation.json`

```
{
  "id": "hmw-formulation",
  "name": "How Might We Question Formulation",
  "phase": "ideation",
  "inputs": [
    { "kind": "artifact", "min": 1, "note": "an opportunity matrix or insight-set to reframe as questions" }
  ],
  "output": { "kind": "hmw-set" },
  "steps": [
    { "role": "ideator", "instruction": "Reframe each opportunity as a How Might We question, neither too broad nor pre-solving." },
    { "role": "critic", "instruction": "Reject a question that names a specific solution rather than a problem space.", "gate": true }
  ],
  "acceptance": [
```



```
{ "rule": "min-evidence-per-node", "min": 1 },
  { "rule": "field-non-empty", "field": "sourceOpportunity" }
],
"evidenceRules": { "traceability": "required" },
"reviewGate": "critic"
}
```

Notice the role of the `critic` agent here. If the `ideator` agent drafts an HMW like "How might we build an AI chatbot to help customers upload documents?", the Critic will reject it instantly because it names a specific solution (a chatbot). The system forces the Ideator to rewrite it to something like, "How might we provide applicants with immediate certainty that their documents are accepted without requiring human review?"

TIP

CORE PRINCIPLE: Treat your HMW statements as prompt engineering for your concept generation phase. If your HMW is sloppy, your concepts will be slop. If your HMW contains a hidden solution, your concepts will just be variations of that single solution.

Morphological Concept Matrices & Systematic Permutations

Once we have a solid register of HMW statements, we move to divergence. To force the agents out of their generic ruts, we use morphological matrices and forced analogies. Instead of asking for a complete idea, we break a service encounter down into its component vectors: Trigger, Interface, Trust Mechanism, and Support Model.

We then instruct the agents to generate permutations using mechanisms borrowed from completely unrelated industries. This is Kumar's Mode 5 (Explore Concepts) executed at machine speed. By forcing the LLM to cross-pollinate a banking KYC problem with the trust mechanisms of parcel tracking or the triage logic of an emergency room, we shatter the consensus bias.

`command.sh`

```
asd run "Read handoff/sme-bundle/. For HMW-03 only, generate 12 ideas.
Mode: cross-sector analogy. Borrow mechanisms from parcel tracking,
appointment scheduling, and document-signing software. Do not repeat
mechanisms already present in ideas/round1/HMW-03.json. Output JSON with
idea_id, source_insight, persona, mechanism, channel, value_hypothesis."
```

In a typical sprint, we will spin up multiple parallel agent threads to execute brainwriting at scale. We might generate 180 raw idea seeds in under an hour. Because they are generated as structured JSON, we can use clustering algorithms to group them by underlying mechanism rather than

surface terminology, yielding perhaps 20 to 25 distinct concept families. Humans can then review these clusters, adding their own lateral variations to the mix.

The NUF Triple-Lens Filter (New, Useful, Feasible)

Divergence is easy; convergence is hard. When you have 25 concept families, everyone in the room has a favorite. Usually, the favorites are either incredibly visionary but technically impossible, or technically trivial but completely useless to the customer. We need a ruthless, objective filter. Enter the NUF Triple-Lens Filter.

Every concept must be scored on three dimensions from 1 to 10: New (is it novel in this context?), Useful (does it solve the specific pain point?), and Feasible (can we actually build and operate this within our regulatory and technical reality?).

nuf-triple-lens-filter.json

```
{
  "id": "nuf-triple-lens-filter",
  "name": "NUF (New-Useful-Feasible) Triple-Lens Filter",
  "phase": "ideation",
  "inputs": [
    { "kind": "artifact", "min": 1, "note": "a concept-set to score before it advances" }
  ],
  "output": { "kind": "concept-set" },
  "steps": [
    { "role": "ideator", "instruction": "Score each concept as New, Useful and Feasible, with a one-line reason for each score." },
    { "role": "critic", "instruction": "Reject a concept advanced with a failing score on any of the three lenses.", "gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 2 },
    { "rule": "field-non-empty", "field": "nufScore" },
    { "rule": "forbid-field-values", "field": "nufScore", "deny": ["fail"] }
  ],
  "evidenceRules": { "traceability": "optional" },
  "reviewGate": "critic"
}
```

The magic of running NUF with an agentic Critic is that it removes the politics from the room. The Critic has no fear of the VP of Sales. If a concept requires bypassing European Anti-Money Laundering (AML) directives, the Critic will score Feasibility a 1 and veto the concept. It forces the human team to address the structural flaws of an idea rather than hand-waving them away.

NOTE

My take: I always run the NUF filter live in the workshop on a big screen. When the Critic agent brutally torpedoed a pet idea because it violates a core technical constraint, it saves us three weeks of pointless feasibility studies. It hurts in the moment, but it builds immense trust in the rigor of the process.

Case Study: Calder SME Banking Ideation Sprint

To see this engine in motion, let's look at a sprint I led for Calder Community Bank. We were tasked with fixing their SME onboarding process. Our discovery phase had proven that their claimed 8-day onboarding journey was a myth; the real median was 19 days, filled with silent delays and repetitive document requests.

We fed the journey pains into the `hmw-formulation` skill. We explicitly set "red lines" for the agents: no assuming relaxed KYC policies, no skipping UBO checks. One early HMW draft from the agent was "How might we approve simple SMEs instantly?" The Critic vetoed it because it embedded a policy breach. The final, approved HMW was: "How might we reduce uncertainty before approval without weakening controls?"

From there, we unleashed the divergence engine. We ran the agents across cross-sector analogies—what if onboarding felt like tracking a DHL parcel? What if it felt like booking a medical specialist? The agents spun up 200 raw idea seeds. OpenCode clustered these into 24 distinct concept families. We printed these clusters and brought them into a room with frontline relationship managers, operations leads, and customers.

The human participants dot-voted the clusters to find the energy, then we ran the top 11 through the NUF filter. Codex, acting as our adversarial critic, killed two immediate crowd favorites. One was a WhatsApp status bot; Codex flagged it as having low distinct value and massive compliance channel risk. The second was an "instant limited account" idea; it scored highly on Useful but failed Feasible completely due to regulatory strictures.

What survived the gauntlet were three highly pragmatic, deeply impactful concepts:

- **Calder LaunchPad:** A one-stop digital flow featuring a preflight checklist and a document vault with a "status heartbeat" inspired by parcel tracking, giving applicants certainty without needing to call the branch.
- **Guided Start:** A coached route for complex SMEs (e.g., multiple directors) where a named Relationship Manager owns the case end-to-end, preventing the application from stalling in the back office.

- **Accountant Connect:** A consented pre-flight channel for accountants to prepare and validate client cases before the bank even touches them, turning an informal workaround into a structured capability.

By enforcing the traceability lineage and the NUF gate, we moved from 200 wild agent ideas to three prototype-ready, operationally feasible service concepts in just 15 days. That is the power of ideation at machine speed when governed by strict service design architectures.

WARNING

CAUTION: Do not let the business skip the NUF scoring phase and jump straight to building the concept with the most dots on it. Dot voting measures enthusiasm; NUF measures viability. Building a highly popular but completely unfeasible service is the fastest way to destroy trust in your design practice.

Forcing Value Propositions: The Forge Industrial Example

This rigor is not limited to digital interfaces or banking. We applied the exact same concepting discipline at Forge Industrial, a massive European manufacturer of heavy process equipment. Their aftermarket business was drowning in emergency break-fix orders. They had telematics data, but every time they tried to innovate, they just came up with another "dashboard feature" that nobody wanted to pay for.

We used the same pipeline. We mapped the evidence—62% of urgent quotes had early warning signals in the telemetry—and formulated HMWs. But here, we forced the agents to output concepts strictly as Service-Dominant value-in-use propositions. We demanded that the concepts show a clear P&L backing and an uptime model.

The NUF filter and a business-model check killed the feature-list portals. The surviving lead concept was **Predictive Care**: a contract combining remote diagnostics, condition-based parts replenishment, and quarterly uptime reviews. The concept didn't just propose a dashboard; it proposed an entirely new delivery model where Forge took accountability for avoiding downtime. The board approved a pilot that modeled an 8-month payback, fundamentally changing their aftermarket strategy.

Ideation is not magic. It is an industrial process. When you treat it like an engineering pipeline—governed by invariants, schemas, and adversarial critics—AI agents cease to be generic brainstormers and become an unstoppable engine for structured service innovation.

Morphological Permutation Matrices: Systematic Divergence

When unconstrained LLMs are asked to "brainstorm 20 innovative ideas for SME banking," they universally generate generic platitudes: "AI chatbot for budgeting," "Gamified financial wellness," or "Real-time push notifications." To achieve genuine breakthrough innovation, we apply morphological analysis—a systematic technique pioneered by astrophysicist Fritz Zwicky.

The `Ideator` agent constructs a morphological matrix breaking the service challenge down into fundamental parameter dimensions. By programmatically permuting combinations across these dimensions, the agent explores non-obvious solution spaces that human teams rarely discover:

Dimension	Option 1 (Traditional)	Option 2 (Digital Assistive)	Option 3 (Autonomous Agentic)	Option 4 (Ecosystem Federated)
Data Ingestion	Manual PDF Upload	OCR & Parsing	Direct Real-Time Open Banking API Sync	Cross-Company ERP Ledger Streaming
Underwriting Logic	Batch Committee Review	Rule-Based Credit Scoring	Continuous Multi- Agent Cashflow Forecast	Decentralized Reputation Scoring
Disbursement Channel	3-Day ACH Wire Transfer	Same-Day Faster Payments	Instant Smart Contract Escrow Release	Dynamic Supplier Direct Pay
Monitoring & Support	Annual Audit Reviews	Quarterly Covenants Check	Predictive Cash Runway Warning Sentinel	Shared Supply Chain Health Matrix

By executing permutation walks across this matrix, the `Ideator` generates 64 distinct architectural concepts. Each concept is then routed immediately through the NUF filter to separate unfeasible fantasies from high-value transformational designs.

14 Prototyping Services: Desktop Walkthroughs to Wizard-of-Oz

Testing service interactions before writing code

The intangibility of services makes them notoriously difficult to prototype. Unlike a physical product you can hold or a software interface you can click, a service is a temporal sequence of human interactions, digital touchpoints, and backstage operations. In this chapter, I will show you how to climb the service prototype ladder—from paper storyboards to live concierge pilots—using AI agent backends to fake the automation until you validate the economics. We will cover how to set pass/fail criteria, write test reports, and ensure that your service actually works before you write a single line of production code.

The Intangibility Challenge in Service Prototyping

If you build a chair, you can sit on it to see if it holds your weight. If you design an app, you can wireframe it and watch a user tap the glass. But how do you prototype a hotel check-in experience, a hospital discharge process, or a public transit disruption recovery? Services are intangible. They exist only in the moment they are delivered, co-created by the customer and the frontline staff, often across multiple channels and physical spaces.

Because services are invisible until enacted, product teams frequently default to prototyping the only part they can easily touch: the digital interface. They build a high-fidelity Figma mockup of a restaurant booking app and call it a "service prototype." This is a trap. A beautiful app that captures a dietary requirement means nothing if the kitchen display system drops the note, the waiter forgets to ask, and the guest ends up in anaphylactic shock. A digital interface is not a service; it is merely one touchpoint in a much larger ecosystem.

To prototype a service, you must prototype the *interactions*. You have to stage the handoffs between the customer, the frontstage staff, and the backstage operations. You must test the timing, the tone, and the fallback procedures when things go wrong. And you have to do this cheaply, quickly, and iteratively, before you commit to expensive software builds, operational retraining, or capital investments.

TIP

CORE PRINCIPLE: You are prototyping the service moment, not just the digital screen. If you only test the app, you are ignoring the human handoffs where services actually fail.

The intangibility challenge forces us to adopt a fundamental rule of service design: **build to think**. We do not build prototypes to validate how smart our ideas are. We build prototypes to expose the flaws in our operating models. We want to find out where the staff get confused, where the customer feels anxious, and where the unit economics fall apart. And we want to find this out when it costs us a few hours of roleplay and some cardboard props, rather than millions of dollars in misdirected engineering effort.

This is where the Agentic Service Designer (ASD) platform changes the game. Historically, service prototyping required a suspension of disbelief. You asked participants to imagine that an email had arrived, or you handed them a piece of paper representing a push notification. With AI agent teams, we can spin up synthetic backends, automated routing rules, and dynamic conversational interfaces in minutes. We can fake the automation so convincingly that the customer—and sometimes even the frontline staff—cannot tell they are interacting with a prototype. This allows us to test the operational viability of a service under realistic conditions, capturing behavioral evidence rather than polite hypothetical feedback.

NOTE

My take: I have watched too many teams burn their budget building a "minimum viable product" that was neither minimum nor viable, simply because they lacked the imagination to prototype the service without writing code. The goal of prototyping is to buy information down the risk curve as cheaply as possible. If you are writing production code to test a service hypothesis, you have already failed.

The Service Prototype Ladder: From Paper to Concierge

You do not jump straight into a live pilot. Prototyping a service is a progressive exercise in increasing fidelity, moving from abstract concepts to concrete operations. I call this the Service Prototype Ladder. At each rung of the ladder, you isolate a specific risk, build the cheapest possible prototype to test it, and set a hard pass/fail gate before moving up.

Figure 14.1 The Service Prototype Ladder. Validate desirability and operational viability at the lowest fidelity before building infrastructure.

The ladder consists of five distinct rungs, each serving a specific evidentiary purpose:

1. **Concept Storyboard (Low Fidelity):** A sequence of sketched panels showing the setup, the service moment, and the resolution. This tests desirability and basic comprehension. Do customers even want this? Do they understand what the service is offering?
2. **Desktop Walkthrough (Low-Medium Fidelity):** A tabletop exercise using Lego figures, printed maps, or paper props to walk through the service blueprint step-by-step. This tests spatial routing, sequencing, and obvious logical gaps in the service flow.
3. **Service Roleplay / Bodystorming (Medium Fidelity):** Enacting the service in a physical space with team members playing the roles of customers and staff. This tests timing, ergonomics, and the emotional resonance of the interaction. Does the script feel natural? Is the physical handoff awkward?
4. **Wizard-of-Oz Simulation (Medium-High Fidelity):** A live test with real customers interacting with a seemingly functional frontstage, while hidden human "wizards" manually execute the backstage operations. This tests the interaction dynamics and the operational burden without building the actual automation.
5. **Concierge Pilot (High Fidelity):** A bounded, fully operational test where the service is delivered manually to a small cohort of real customers in the actual environment. This validates the unit economics, the service standards, and the real-world operational feasibility before scaling.

The SDAP engine enforces the discipline of this ladder through its prototyping skills. When you run `sd_method_run` for a prototyping method, the engine forces you to declare the hypothesis, the target subjects, the fidelity, and the success criteria. A prototype is an experiment, not a demonstration. If you cannot articulate what you are trying to learn and what evidence would prove you wrong, the Critic agent will veto the artifact.

prototype-plan.json

```
{
  "id": "prototype-01",
  "method": "storyboard-authoring",
  "hypothesis": "Guests will share dietary restrictions if prompted before arrival.",
  "fidelity": "low",
  "successCriteria": "80% of participants identify the dietary prompt without confusion.",
  "status": "draft"
}
```

Let's look at the first rung: the Concept Storyboard. The `storyboard-authoring` skill forces you to define the protagonist's action and emotion in every frame. A storyboard is not an architectural diagram; it is a narrative sequence. The Critic agent is instructed to reject any storyboard frame that lacks a specific action, because a frame without action is just a mood board, and you cannot test a mood.

Rung	Fidelity	Primary Risk Tested	Required Artifact
Storyboard	Low	Comprehension / Desirability	storyboard-plan
Desktop Walkthrough	Low-Medium	Sequencing / Logic	walkthrough-script
Roleplay	Medium	Ergonomics / Timing	roleplay-scenario
Wizard-of-Oz	Medium-High	Operational Burden	wizard-of-oz-plan
Concierge Pilot	High	Unit Economics	concierge-pilot-plan

Desktop Walkthroughs and Tabletop Service Roleplay

Once you have validated the basic concept via storyboarding, you must map it to time and space. This is where desktop walkthroughs and tabletop roleplay come in. These methods force you to externalize the service blueprint, pulling it off the screen and into a physical shared context where the whole team can poke holes in it.

A desktop walkthrough is exactly what it sounds like: you map out the physical environment of the service (e.g., a hospital floor plan, a restaurant layout, a transit hub) on a large table. You use avatars (Lego figures, wooden blocks, or paper cutouts) to represent the customer and the various staff members. You then physically move the avatars through the service journey, narrating each step of the blueprint aloud.

This sounds childish. It is actually one of the most ruthless diagnostic tools in the service designer's arsenal. When you force a stakeholder to physically move a piece of paper representing a "dietary requirement note" from the host stand avatar to the kitchen expo avatar, they suddenly realize that the host stand is fifty feet away from the kitchen, the host is managing a line of twelve angry guests, and there is absolutely no mechanism for that note to arrive in time. The desktop walkthrough exposes the friction of distance, time, and divided attention that a clean, two-dimensional service blueprint often hides.

```
walkthrough-script.md
```

```
# Walkthrough Script: Arrival Flow
Scene 1: Customer enters the lobby.
Action: Host greets customer.
Prop: Tablet with reservation list.
Observer Prompt: Does the host look at the screen or the customer? What if the
tablet is offline?
```

After the desktop walkthrough resolves the gross spatial and logical errors, you move to bodystorming or service roleplay. Here, the team acts out the service in a simulated physical environment. You set up chairs to represent tables, use a whiteboard as a menu, and hand out

paper props representing digital interfaces. The goal here is to test the ergonomics and the emotional tone of the script.

During roleplay, you introduce forced friction. You do not just run the happy path. You instruct the "customer" actor to arrive late, to change their mind, to ask an unexpected question. How does the "staff" actor react? Where do they look for information? Does the drafted script feel robotic, or does it allow for natural human empathy?

NOTE

My take: The most valuable insights from roleplay rarely come from the actors; they come from the observers. I always assign one team member to watch specifically for "compensating behaviors"—the little workarounds, apologies, or extra steps the staff actor takes because the designed process is awkward or broken. That compensating behavior is your failure demand in embryo.

When running these methods using the ASD platform, the Prototyper agent is responsible for authoring the scenarios and the props, while the Measurer agent ensures that the pass/fail criteria are explicitly defined. The resulting evidence—the observer notes, the timing data, the identified friction points—must be logged back into the evidence register via `sd_evidence_add`. The platform forces traceability: you cannot propose a change to the service blueprint without citing the specific observation from the walkthrough or roleplay that justifies it.

Wizard-of-Oz Testing with Agentic Backends

You have storyboarded the concept. You have walked it across a desk. You have acted it out in a room. Now you must test it with real customers. But building the backend integration, the routing logic, and the automated triggers is expensive. How do you test the operational dynamics of a software-enabled service without writing the software?

You use the Wizard-of-Oz method. In a Wizard-of-Oz prototype, the customer interacts with a seemingly fully functional frontstage interface, but the backstage operations are entirely manual. A human "wizard" sits behind the curtain, receiving the customer's inputs, manually executing the required logic, and pushing the appropriate responses back to the frontstage.

Historically, this meant someone furiously typing emails or SMS messages, trying to pretend to be an automated system. Today, with the ASD platform and LLM-driven agent stacks, Wizard-of-Oz testing has become a superpower. We can deploy lightweight, conversational agents or simple clickable frontends that route data into a shared Slack channel or a Google Sheet. The human operators (the wizards) monitor this incoming stream, make the complex routing decisions that the future algorithm will eventually make, and trigger the next step of the service.

TIP

CORE PRINCIPLE: The goal of a Wizard-of-Oz test is to measure the *operational burden*. You are testing how hard the human wizard has to work to fulfill the service promise. If the cognitive load on the wizard is too high, the future automated system will likely fail in the same edge cases.

The `wizard-of-oz` skill in the SDAP engine requires three distinct elements to pass the gate:

1. A clear definition of the frontstage interface (what the user sees).
2. A clear definition of the manual backstage process (what the wizard does).
3. A fallback script (what the wizard does when the customer does something entirely unexpected).

The Critic agent explicitly rejects any Wizard-of-Oz plan that lacks a fallback script. If a customer types "I am running late" into a chat interface, and your prototype only accounts for "Book a table" or "Cancel," the illusion breaks. The wizard needs a defined operational parameter for handling exceptions, otherwise you are just testing improvisation, not service design.

`wizard-of-oz-plan.json`

```
{
  "id": "woz-01",
  "method": "wizard-of-oz",
  "frontstage": "SMS chatbot for transit disruption alerts.",
  "backstage": "Human operator monitoring transit API and manually sending SMS replies.",
  "fallbackScript": "If user replies with unrecognized query, reply: 'I am a prototype assistant. I can only answer questions about Route 41.'",
  "fidelity": "medium-high",
  "status": "draft"
}
```

By using AI agents in the Wizard-of-Oz backend, we can simulate complex triage logic. We can use an LLM to parse a messy, free-text customer request, extract the intent, and present a structured decision to the human wizard. This allows the prototype to scale slightly higher than a purely manual test, letting us test the service with dozens or hundreds of users in a single afternoon.

Case Study: Ember Hospitality Rapid Guest Experience Experiments

Let's look at how this plays out in the field. Ember Hospitality, a composite restaurant group operating 60 sites across Europe, had a problem. Their special-occasion dining experience—birthdays, anniversaries, graduations—was fundamentally broken. Guests would leave a vague

note in the booking app ("birthday maybe cake"), and this information was routinely lost between the host stand, the kitchen, and the floor staff. The result was a high rate of missed surprises, dietary errors, and frantic backstage scrambling.

Ember's leadership initially wanted to build a complex "Occasion Management" feature directly into their booking app, complete with automated cake ordering and integration with their Point of Sale (POS) system. It was a six-month, €400,000 proposed build.

I convinced them to halt the software build and run a six-week prototype sprint using the ASD platform, progressing rigorously up the service prototype ladder.

Phase	Prototype Method	Fidelity	Key Learning
Week 1-2	Storyboard & Bodystorming	Low	Guests want strict control over surprise visibility.
Week 3	Walkthrough & Clickable Props	Medium	Staff need a single, unified brief, not scattered notes.
Week 4-5	Wizard-of-Oz Live Service	Medium-High	Universal concierge is too expensive; selective routing works.
Week 6	Concierge Pilot	High	Unit economics proven; + €6.5 attach revenue per cover.

We started with Storyboards and Bodystorming (P1). We drafted a concept where the app would ask detailed questions about the occasion and prompt staff to acknowledge the celebration publicly. When we bodystormed this with the floor staff, they hated it. They pointed out that many occasions (like a fertility anniversary or a quiet romantic dinner) demanded discretion. If every floor runner knew about the occasion, the risk of an inappropriate public comment was too high. The prototype failed the gate, preventing us from building a fundamentally flawed assumption into software.

We revised the concept and moved to a Desktop Walkthrough with clickable props (P2). We simulated the flow of information from the booking app to a simulated staff tablet. Here, we discovered that dietary requirements and deposit promises were currently owned by different systems. If we didn't unify them into a single "Celebration Brief" for the host stand, the staff would still scramble. We re-routed the handoffs in the blueprint based on this evidence.

```
asd-command-log.sh
```

```
# The real command used to simulate routing rules during the Ember pilot
opencode run --agent build "Using clean/occasion_baseline.csv, simulate routing
rules for Celebration Concierge. Route to human concierge when any of: party
size ≥ 10, multi-part dietary note, surprise coordination requested, or spend
```

estimate above 180 EUR per head. Output estimated weekly concierge volume by restaurant tier and average manual minutes per routed booking."

Next, we ran the Wizard-of-Oz live test (P3). We deployed a lightweight booking front-end to a cohort of 66 real guests across six restaurants. Behind the scenes, there was no backend database or automated routing. A hidden operator (the wizard) manually reviewed every booking, checked the surprise constraints, and manually typed out the Celebration Brief, which was pushed to the restaurant's host tablet. We tracked the effort scores of the staff and the completion rates of the guests.

The Wizard-of-Oz test revealed the critical operational constraint: a universal concierge model—where every single occasion booking was manually reviewed—would overwhelm the staff and destroy the unit economics. The manual effort was simply too high. However, we noticed that 70% of the value came from handling the 20% of high-risk bookings (groups of 10+, complex dietary needs, intricate surprises).

NOTE

My take: This is the exact moment where the Wizard-of-Oz method pays for itself. If we had built the automated software based on the initial idea, we would have deployed a universal system that choked on edge cases and alienated the staff. Because we were faking the automation, we could pivot the operating model overnight without rewriting a single database schema.

Finally, we took this insight into the Concierge Pilot (P4). We finalized the concept as the "Celebration Concierge": structured capture at booking, automated routing rules that sent only the high-risk bookings to a human concierge, and a unified Celebration Brief for the floor staff. We ran this live for 84 guests. It was a bounded, real-world test. The results were undeniable: staff scramble time dropped from 11.4 minutes to 6.2 minutes per celebratory table, special-occasion NPS jumped from 31 to 54, and attach revenue (upsells for cakes, champagne) increased by €6.5 per cover.

We delivered an empirically validated business case alongside the prototype. Final deliverables comprised a future-state service blueprint, an operational measurement plan, and a COO-approved rollout beta gate rather than speculative wireframes. By climbing the prototype ladder, we ensured that when Ember finally wrote the code, they were building a service that was guaranteed to work.

Case Study: Lighthouse Transit Concierge Test

We applied the exact same ladder logic to Lighthouse Transit, a regional public transport operator struggling with missed connections and high failure demand. As part of a massive 18-week service design sprint driven entirely through the ASD Claude Code plugin, we sought to test a disruption

recovery model. Our journey maps and telemetry showed that when riders missed a connection during peak-shoulder hours, their emotional state plummeted, and they immediately resorted to calling the contact center—driving up failure demand.

The proposed solution was a "Recovery Engine": an automated system that would detect a missed connection via GTFS-realtime data and instantly SMS the rider with a voucher or alternative routing. Building this engine required complex fare-vendor integration and predictive models. Before spending a dime on development, we ran a Wizard-of-Oz test and a Concierge Pilot.

Using the `concierge-pilot` method, we bounded the test to a single route (Route 41) during the night shift. Our hidden operators watched the AVL (Automated Vehicle Location) dashboard manually. When they saw a bus miss the tram connection window, they manually looked up the registered riders on that route (using a mocked dataset for the pilot) and manually texted them: "We see your bus is delayed and you will miss the tram. Here is a voucher for a coffee while you wait 15 mins for the next one, or reply 'TAXI' to have us dispatch a cab."

This medium-high fidelity simulation was completely manual on the backend. The operators were frantically typing SMS messages, acting as the "algorithm." We learned immediately that the cognitive load on the operators was extreme—proving that the eventual automation had to be highly robust. More importantly, we discovered a fatal flaw in our initial design: the 3-minute connection protection rule we wanted to implement contradicted existing union roster constraints. The Critic agent, analyzing the evidence register, flagged this defect before the pilot scaled. We had to rewrite the backstage lane to respect roster realities.

The pilot proved the value of proactive communication: journey abandonment dropped significantly, and NPS for the affected cohort flipped positive. We validated the recovery model's economics manually, proving to the Regional Authority that investing in the automated Recovery Engine was fiscally responsible.

Conclusion: The Rigor of the Prototype Ladder

Service prototyping is not about making things look real; it is about making the operational dynamics behave realistically. Whether you are moving Lego figures across a desk or running a sophisticated Wizard-of-Oz simulation with agentic routing, your goal is to expose the failure demand, measure the cognitive load on the staff, and validate the customer's comprehension.

The Agentic Service Designer platform enforces this rigor deterministically. It refuses to let you generate a prototype plan without a hypothesis. It demands that your test reports cite actual verbatim evidence, not just summarized vibes. It ensures that every step up the prototype ladder is justified by the data collected on the rung below.

Do not build software to discover if a service is a good idea. Build cheap, fast, manual prototypes. Fake the automation. Measure the friction. And only when the unit economics hold and the staff are confident, do you commit to the code. That is how you design services that actually survive contact with reality.

15 Transformative Services, Behavior Change & Recovery Engineering

Designing for well-being and engineering zero-defect recovery pathways

When service design transcends simple convenience, it enters the realm of human transformation and high-stakes recovery. I have spent years watching organizations build beautifully frictionless apps that utterly fail to change human behavior, or worse, systems that abandon customers the moment an edge case arises. In this chapter, I dissect how to engineer services that build human capability over time, and how to construct deterministic, AI-driven recovery playbooks when those systems inevitably fail. We are moving beyond the illusion of perfect operations into the reality of resilient, transformative service architectures.

Transformative Services: Beyond Convenience to Human Well-Being

Most service design optimizing for the digital age obsesses over convenience. We shave seconds off a checkout flow. We reduce clicks in a dashboard. But convenience is a shallow metric when you are designing for health, financial solvency, or long-term behavioral change. Enter Transformative Service Research (TSR). TSR shifts the lens from transactional satisfaction to human wellbeing. It demands that we evaluate a service based on whether it expands the real capabilities and freedoms of the people who use it.

I learned this the hard way during my time designing metabolic-risk coaching services. The industry standard was to track "engagement"—how many articles a user opened, how many steps they logged. It looked like success on a screen. But when you looked at the humans behind the telemetry, the reality was stark. Engagement did not equal capability. Users were logging data as labor for our platform without receiving value in their own context. The service was a library combined with a nagging reminder system. It did not transform their reality; it just added cognitive load to their already stressed daily ecology.

TIP

CORE PRINCIPLE: Value is co-created in the user's daily life, not inside your app. If your service requires perfect compliance to work, it is a fragile service.

To design a transformative service, you have to fundamentally change the hero of the narrative. The app is not the hero. The service is not the hero. The user is the hero, and the service is merely the mentor. Our job as service designers is to map the user's daily ecology—the messy intersection of shift work, childcare, commuting, and exhaustion—and build scaffolding that supports them right there. This means looking deeply at the underlying needs. A user doesn't want "metabolic coaching"; they want to avoid medication, or they want the energy to keep up with their children. When you anchor the service to their real, lived stakes, you stop designing for compliance and start designing for agency.

This shift requires moving from a deficit model—where the user is seen as broken and needing instructions—to an asset-based model. We must recognize the innate problem-solving capacity of communities and individuals. In TSR, we call this supporting "diffuse design." The user is already trying to solve their problem. Expert design—what we do—should provide the boundary objects, the participatory infrastructures, and the lightweight platforms that amplify their efforts.

NOTE

My take: The most dangerous lie in digital health is that "education drives behavior." It doesn't. If information alone changed behavior, nobody would smoke. We have to design for the reality of human fatigue, not the fantasy of perfect rational compliance.

Let's look at how we applied this at Sprout Wellness. The legacy metabolic-risk programme had an enrollment rate of 31%, and only 28% of those made it to week 12. Why? Because the service demanded high co-production (filling forms, logging meals) before any value was delivered. It confused labor with co-creation. We rebuilt the entire architecture around Sen's Capability Approach. Instead of measuring clicks, we measured whether the service expanded a member's real freedom to make healthier choices within the constraints of their actual lives. We measured wellbeing dimensions like physical capability and psychological resilience.

We mapped the daily ecology of our most vulnerable segments: shift workers and post-pregnancy members. We discovered that the app's rigid check-in times were actively hostile to their routines. The transformation began when we stopped asking "How do we get them to use the app?" and started asking "How does this service fit into a 2 AM warehouse break?"

Behavior Change Scaffolding: Prompts, Abilities, and Habit Loops

Transformative services rely on durable behavior change. You cannot engineer behavior change through sheer willpower; you have to scaffold it. This means explicitly mapping Behavior Change Techniques (BCTs) to specific touchpoints. But here is the trap: you cannot just throw BCTs at a user and hope they stick. You must design the habit loop—cue, routine, reward—and you must design it for failure.

In the Sprout Wellness rebuild, we used Claude Code to map the service to BCTTv1 techniques. We looked at 12 weeks of touchpoints and assigned a specific mechanism of action to each. Week 1 wasn't just "onboarding"; it was "Goal setting behavior." Week 3 wasn't "check-in"; it was "Self-monitoring of behavior." But we didn't blindly trust the LLM. We used Codex to enforce clinical boundaries, ensuring no BCT implied diagnosis or guaranteed biomarker changes without clinician review.

```
bct-touchpoint-map.json
```

```
{
  "week": 1,
  "touchpoint": "Coach kickoff and goal contract",
  "bct_label": "Goal setting behavior",
  "mechanism": "Agency and commitment",
  "safety_gate": "No diagnosis language allowed",
  "metric": "Week-1 activation"
}
```

The most radical design choice we made was planning for relapse. In most services, a lapse is treated as a user failure. The system escalates nudges, turns red, and induces shame. We explicitly rejected the "streak" mechanic. Why? Because for a vulnerable user, breaking a 40-day streak doesn't motivate them to start a new one; it triggers an identity collapse. They feel they have failed irrevocably.

Instead, we engineered the service to treat relapse as expected telemetry. When a member missed a check-in, the system didn't yell at them. The recovery script offered a smaller, fallback goal within 72 hours. We separated goals into "minimum viable habit" and "stretch habit." If the stretch habit failed because a kid got sick, the minimum viable habit was still there, protecting the user's identity as someone who is trying. Relapse recovery became a core capability of the service, not an exception path.

NOTE

My take: If your behavioral service does not have a graceful degradation path for when a user is exhausted, sick, or overwhelmed, you have not designed a service. You have designed a guilt trip.

This scaffolding extended to the coaches. We wrote SOPs-as-code for the coaching interactions. The week 4 check-in script explicitly forbade compliance language. It forced the coach to ask for context before offering advice. We built a safety branch: if the user mentioned symptoms or worrying readings, the automation paused, and a clinician was routed in. AI agents monitored the

transcripts in real-time, verifying that the human coaches were adhering to the BCT map without drifting into unlicensed medical advice. The result? Recovery within 7 days of a first lapse jumped from 22% to 57%.

The Service Recovery Paradox and the Cost of Second Failures

If transformative services are about designing for the messy reality of human life, service recovery is about designing for the messy reality of organizational failure. Services are real-time, interactive processes. They will fail. The difference between a mature organization and an immature one is how they handle the breakdown.

Christian Grönroos teaches us that complaint handling is not an administrative cost center; it is a relationship investment. The vast majority of dissatisfied customers never complain—they just leave. The ones who do complain are giving you a rare opportunity to restore trust. This brings us to the Service Recovery Paradox: a customer who experiences a failure that is resolved with exceptional speed and empathy can become more loyal than a customer who never experienced a failure at all.

However, the paradox is fragile. It only applies to minor, first-time failures. It does not survive repeated incompetence or perceived deception. And crucially, it requires justice across three dimensions:

- **Procedural Justice:** Was the process easy? Did I have to wait on hold for an hour? Did I have to repeat my story to four different agents?
- **Interactional Justice:** Was I treated with respect? Did the agent take psychological ownership of the issue, or did they hide behind "company policy"?
- **Distributive Justice:** Was the compensation fair? Did it actually offset my wasted time and financial loss?

When I led the redesign of complaint handling at Aria Telecom, we found a system bleeding loyalty. Aria had 720,000 complaints a year. The median closure time was 8.6 days. But the real horror was the repeat failure: 27% of complaints reopened within 30 days. Customers were experiencing a failure, complaining, and then experiencing a *second failure* in the recovery process itself.

We audited the 720,000 records using OpenCode to build a classifier. We found that the recovery journeys routinely violated all three dimensions of justice. Customers had to repeat their stories (procedural failure). Agents apologized but took no action (interactional failure). Compensation was wildly inconsistent, depending purely on which outsourced queue the ticket landed in (distributive failure). The economic consequence was devastating: 90-day post-complaint churn was 14.8%.

TIP

CORE PRINCIPLE: Fix first, then compensate, then prove the fix held. An apology without an action plan is just corporate gaslighting.

To fix this, we had to re-architect the service blueprint. We stopped looking at complaints as isolated tickets and started looking at them as end-to-end journeys. We defined the start of the journey at the initial moment of customer realization rather than when a ticket is logged in the CRM. This meant signposting help clearly, eliminating dead ends, and setting honest expectations upfront.

Engineering Automated Recovery Playbooks and Compensation SLAs

You cannot scale empathy if your frontline staff are handcuffed by bureaucracy. The old Aria journey forced agents to escalate anything involving money to a supervisor. This created massive procedural delays and destroyed interactional justice, as agents had to say, "I have to check with my manager."

We engineered automated recovery playbooks as SOPs-as-code. We gave frontline agents the discretionary authority to apply up to €50 compensation instantly, and up to €120 for vulnerable customers or proven billing errors. We built a compensation matrix—a deterministic rule engine—that suggested the correct band based on the root cause and customer impact.

```
compensation-matrix.yaml
```

```
rules:
  - band: C1
    trigger: "Minor inconvenience, outage < 24h"
    credit: "€10-€25"
    approval: "Agent discretion"
  - band: C2
    trigger: "Billing error, outage 1-3 days"
    credit: "€30-€60"
    approval: "Agent discretion + audit log"
  - band: C3
    trigger: "Proven financial loss, vulnerable customer harm"
    credit: "€70-€120"
    approval: "Supervisor / Vulnerable desk"
```

We used Claude Code to generate apology ladders tailored to the specific justice gaps. But we strictly enforced a copy rule: the message had to state what Aria would do next and by when. It could never imply the customer was mistaken until evidence was checked. We used Codex to lint

the generated copy, actively flagging defensive phrases like "we understand this may have caused inconvenience" when the telemetry already proved we had double-billed them.

Figure 15.1 Automated Service Recovery workflow. Proactive service recovery resolves failures before the customer is forced to file a complaint.

Crucially, we implemented a closed-loop root-cause learning system. Recovery is incomplete if it only appeases the individual customer. Every resolved complaint had to tag the root cause owner. We built a weekly Pareto review that routed the largest failure clusters directly back to the billing or network engineering teams. The complaint was no longer just a support ticket; it was high-priority telemetry for the product organization.

NOTE

My take: The most profound shift in a service organization happens when you stop measuring the contact center on "Average Handle Time" and start measuring the product team on "Failure Demand." When the engineers feel the pain of the recovery cost, the systemic defects disappear overnight.

Case Study: Sprout Wellness Health Coaching & Aria Telecom Recovery

Let's look at the concrete outcomes of these architectural shifts.

At **Sprout Wellness**, the shift to a transformative, capability-building service yielded dramatic results. By casting the user as the hero, accommodating their daily ecology, and designing explicit relapse recovery paths, we saw enrollment from eligible lists jump from 31% to 43%. More importantly, week-12 completion rates nearly doubled, moving from 28% to 49%. Relapse recovery—getting a user back on track within 7 days of a missed check-in—skyrocketed from 22% to 57%. We proved that when a service stops judging a user for failing and starts giving them the tools to restart, the user sticks around. The Member NPS increased from +11 to +34, and the employer ROI stabilized at a projected 1.8:1. This is the power of TSR applied deterministically.

At **Aria Telecom**, treating complaint handling as a designed, justice-oriented service fundamentally changed the economics of failure. The pilot of our automated recovery playbooks and empowered frontline teams was transformative. First-contact resolution (FCR) rose from an abysmal 41% to 64%. The median resolution time collapsed from 8.6 days to just 3.7 days. By enforcing the "fix first, then compensate, then prove" protocol, the 30-day repeat complaint rate plummeted from 27% to 12%. And the ultimate metric—the 90-day churn rate following a complaint—fell from 14.8% to 9.2%. We retained customers who would have otherwise become silent defectors.

These two cases illustrate the twin pillars of advanced service design: designing for the best possible human transformation, and engineering zero-defect recovery for when the system falls short. In both domains, we are not just drawing journey maps. We are building living, executing

architectures that catch users when they fall—whether they are stumbling on a health goal or suffering from a network outage. This is the agentic service designer's mandate: to build systems that are as resilient and adaptable as the humans who use them.

Service recovery, at its core, is the ultimate test of your service architecture. If your blueprint cannot dynamically re-route, compensate, and inform a customer the moment an API call fails or a physical delivery is missed, your blueprint is just a drawing. Agentic services make recovery immediate. We can monitor telemetry streams for failure, instantiate a recovery agent, execute the playbook, and notify the customer before they even realize a defect occurred. This is the transition from reactive complaint handling to proactive service resilience.

The journey from traditional service design to an AI-first, agentic operating model requires a total reimagining of what a deliverable is. We must discard the static PDF and the workshop theater. We must embrace executable SOPs, deterministic fact-checking, and continuous, autonomous remediation. When we do this, we elevate service design from a consultative luxury to the core operating system of the modern enterprise. We move from designing interfaces to engineering realities.

As we transition into exploring Service Quality Management and CX Maturity Governance in the next chapter, remember this: quality is not a static state. Quality is the continuous, dynamic equilibrium achieved when a service can adapt to human context and recover flawlessly from its own errors. The tools we have discussed here—habit scaffolding, justice-driven recovery, and agentic orchestration—are the bedrock of that equilibrium.

Deep Dive: The Mechanics of Procedural Justice

To truly understand how procedural justice alters the economics of service recovery, we must dissect the micro-interactions that comprise a complaint journey. When a customer at Aria Telecom experienced a duplicate bill, the legacy system demanded that they call a general support line, navigate a labyrinthine IVR, wait on hold, and then explain their issue to an outsourced agent who lacked access to the core billing ledger. This is a procedural failure of the highest order. It communicates to the customer that their time is worthless, and that the organization's internal silos are more important than the customer's lived experience.

Our agentic redesign dismantled this friction. We implemented a unified case object that traveled with the customer across all channels. If a customer started a complaint in the app and subsequently called the contact center, the agent's dashboard immediately populated with the full context, preempting the need for the customer to repeat themselves. Simultaneously, we deployed automated classifiers that scanned incoming text for "vulnerable customer" signals, immediately prioritizing those cases and bypassing the standard queue logic. Procedural justice is not just about speed; it is about cognitive ease and the preservation of customer dignity.

Interactional Justice: The Language of Accountability

Interactional justice is where most automated systems fail catastrophically. The instinct of risk-averse legal departments is to craft apologies that apologize for the customer's feelings rather than the organization's actions. "We are sorry you feel frustrated" is a toxic phrase that destroys trust. At Aria, we utilized Claude Code to rewrite the entire apology ladder, stripping out defensive corporate jargon and replacing it with active, accountable language.

We established a strict linguistic contract for our agents and our automated touchpoints: State the issue plainly, own the error, outline the exact steps being taken to resolve it, and provide a concrete timeline for the next update. By enforcing this through continuous integration checks on our SOPs-as-code, we ensured that every customer communication conveyed genuine psychological ownership of the failure. The result was a measurable spike in post-recovery fairness scores, jumping from 2.4 to 4.1 on a 5-point scale. Customers can forgive a technical outage; they will rarely forgive being patronized.

Distributive Justice: The Economics of Fair Restitution

Distributive justice requires that the compensation offered is proportionate to the friction experienced by the customer. A five-dollar credit for a three-day broadband outage that disrupted a customer's ability to work remotely is an insult, not a recovery. Conversely, empowering agents to offer unbounded compensation without a framework invites exploitation and destroys margins.

The solution is a deterministic compensation engine. At Aria, we built a rule matrix that evaluated the root cause (e.g., billing error, network silence), the customer impact (e.g., financial loss, time at risk), and the customer's vulnerability status. The engine then output a recommended compensation band. Crucially, the rule was simple: if the system suggested a band, the frontline agent was authorized to apply it instantly, with zero managerial oversight. This eliminated the humiliating process where agents had to put customers on hold to "ask a manager for a favor." It transformed the agent from a powerless conduit into an empowered problem-solver, fundamentally altering the power dynamic of the interaction.

Closing the Loop: From Recovery to Prevention

The final, and perhaps most critical, element of recovery engineering is the root-cause learning loop. A brilliantly executed recovery is still a failure if the same defect triggers another complaint the following day. We engineered a system where every resolved ticket automatically generated structured telemetry tagging the precise operational failure point. This data was aggregated weekly and routed to the operational owners—network engineering, billing operations, or reseller management.

This closed the loop between customer experience and operational engineering. When the billing team saw the exact financial cost of their recurring ledger defects quantified in distributed compensation credits, the prioritization of technical debt shifted immediately. We moved the organization from a defensive posture of "handling complaints" to a proactive posture of "eliminating failure demand." This is the ultimate promise of the Service Design OS: transforming qualitative human friction into actionable, systemic engineering requirements.

16 Service Quality Management & CX Maturity Governance

Institutionalizing design from project to operating system

Here is the hard truth about service design: shipping a redesigned touchpoint is easy; keeping it alive in a complex enterprise is warfare. Without a systemic governance structure, any localized improvement you launch will inevitably succumb to operational entropy. This chapter details how to transition your practice from running isolated design projects to managing a continuous, self-correcting service operating system. We will explore the theoretical foundations of perceived quality, the rigorous measurement architectures that link operations to finance, and the governance models required to institutionalize customer-centricity at scale.

Why Service Quality Degrades Without an Operating System

If you have spent any meaningful time in the trenches of enterprise service design, you have witnessed the decay curve. You assemble a brilliant cross-functional team, run an exhaustive discovery phase, map the current state, ideate, prototype, and finally launch a streamlined journey. For the first three months, metrics improve. The executive sponsor takes a victory lap. The design agency captures a glowing case study. Then, the team is disbanded and reallocated to the next digital transformation initiative.

Six months later, the service begins to drift. The compliance team injects a mandatory legal disclaimer that breaks the mobile UI, adding three extra clicks. The billing department changes a database schema to support a new product line, resulting in opaque error messages for end users on legacy plans. The offshore contact center alters a script to optimize for Average Handling Time (AHT) instead of First Contact Resolution (FCR), pushing complex queries back into the queue. Because no one owns the end-to-end journey anymore, no one catches the degradation until it shows up as an aggregate dip in the annual Net Promoter Score (NPS) report. By then, the root cause is buried under layers of organizational amnesia.

NOTE

My take: I have seen more good service designs destroyed by post-launch operational drift than by poor initial concepts. Design is not a discrete project phase; it is an ongoing management discipline. If you do not build the governance and measurement machinery to defend the service standard, your design will be dismantled piece by piece by the very organization you are trying to help.

Service quality degrades because traditional corporate structures are optimized for siloed functional efficiency, not cross-functional customer journeys. When an organization lacks a Service Quality Management (SQM) operating system, it relies on heroic individual interventions to fix systemic failures. But heroism does not scale. To build an autonomous, AI-augmented service system, we must encode quality standards into machine-readable rules, establish continuous monitoring, and enforce accountability through rigorous governance. This requires shifting our mindset from "delivering a project" to "programming an operating system."

When you transition to treating service design as an operating system, the deliverables change entirely. You stop producing PDF journey maps and start writing execution standards. You stop relying on quarterly executive readouts and start relying on continuous CI/CD pipelines that flag when a service standard is breached. The system monitors the pulse of the service autonomously, raising alarms only when human intervention is genuinely required. But to build this machine, we must first understand exactly what we are measuring.

The Grönroos Total Perceived Quality Model & The 5 GAPS

To measure and manage quality, we must define what quality actually is in a service context. Unlike physical goods, services are intangible, perishable, and co-produced in real-time interactions between the provider and the customer. You cannot inspect a service on an assembly line before it is shipped. The quality of a service is judged precisely at the "moment of truth" when it is consumed.

The foremost authority on this dynamic is Christian Grönroos, whose Total Perceived Quality model forms the bedrock of the Nordic School of service research. Grönroos posited that service quality is not an absolute metric; it is the subjective gap between what a customer expects and what they actually experience, filtered through the perceptual lens of the corporate image.

Figure 16.1 The Grönroos Total Perceived Quality Model. Customers evaluate service quality through both the technical outcome and the functional experience of delivery.

Grönroos breaks Experienced Service into two distinct, non-fungible dimensions:

- **Technical Quality (What is delivered):** This is the objective, instrumental outcome of the service process. Did the field technician fix the router? Was the billing calculation mathematically correct? Did the multi-site contract cover all intended locations? Technical quality is a prerequisite, but it is rarely a differentiator because customers expect it as a baseline. When you buy a coffee, the technical quality is the caffeine and the temperature. If it is cold, you are angry, but if it is hot, you are merely satisfied.
- **Functional Quality (How it is delivered):** This is the expressive, interactional process of receiving the service. Was the technician respectful of the customer's home? Was the bill explained in plain, accessible language? Did the onboarding process feel proactive and trustworthy? Functional quality is where loyalty is won or lost. Using the coffee example, functional quality is the barista remembering your name, making eye contact, and handing you the cup with a genuine smile.

TIP

CORE PRINCIPLE: A service can be technically flawless and functionally disastrous. If a customer receives a mathematically correct bill (Technical Quality) but has to navigate a labyrinthine IVR system and speak to three different agents to understand it (Functional Quality), the perceived quality is negative.

To complement the Nordic School's definitional rigor, we look to the American School—specifically Parasuraman, Zeithaml, and Berry's SERVQUAL model, often referred to as the 5 GAPS model. The 5 GAPS framework operationalizes the decay of service quality by identifying exactly where the breakdowns occur within the organizational machinery:

1. **The Management Perception Gap:** Management does not accurately understand what customers expect. Executives assume customers want a flashy, feature-rich mobile application, when in reality, the customers just want a transparent pricing model and an easy way to cancel their subscription.
2. **The Quality Specification Gap:** Management understands the expectations but fails to encode them into enforceable service standards. The corporate goal is "fast and friendly resolution," but there is no explicit SLA defining how many hours constitute "fast," nor any operational rule dictating who owns a cross-departmental case.
3. **The Service Delivery Gap:** Standards exist, but the frontline employees or the automated systems fail to deliver against them. This is often the result of poor tooling, inadequate training, understaffing, or conflicting internal incentives (e.g., punishing agents for long call times when they are trying to solve complex issues).
4. **The Market Communication Gap:** Marketing promises a level of service that operations fundamentally cannot fulfill. A campaign promises "Unified omni-channel support," yet the

reality is that the web chat system and the phone system run on isolated databases, forcing the customer to repeat their account details at every step.

5. **The Perceived Service Gap:** This is the ultimate discrepancy between the customer's expected service and their perceived experience. It is the cumulative result of all failures in Gaps 1 through 4.

When we build our AI-driven Service Quality Management systems, we explicitly target these gaps systematically. We use the Critic agent to close Gap 1 by demanding verbatim evidence for any claim about customer needs. We use the standards-authoring skill to close Gap 2 by writing machine-readable, testable rules. We monitor telemetry to close Gap 3, identifying instantly when an SLA is breached. We enforce blueprint constraints to close Gap 4, ensuring that no marketing promise goes live without a corresponding operational backstage.

Three-Tier KPI Trees: Linking Operations to Experience to Finance

If you want to hide operational failure, roll it up into an enterprise average. A single, top-line NPS score reported to the board quarterly tells executives absolutely nothing about why a specific customer cohort churned or why a specific region is underperforming. To manage service quality rigorously, you need a measurement architecture that maintains strict causal linkage across different altitudes of the business. We use the Three-Tier KPI Tree, a structure heavily influenced by the Service Profit Chain and Forrester's measurement disciplines.

A properly constructed KPI tree connects underlying operational inputs to experiential moments of truth, and ultimately to financial outcomes. If you cannot draw a straight line from an operational metric to a financial result, you are measuring the wrong things. The three tiers are:

Tier	Definition	Typical Metrics	Review Cadence	Primary Owner
Operational	System performance, compliance, and internal friction measures. These are the lead indicators. They tell you what is breaking right now.	Owner assignment time, API error rate, SLA adherence, first-response time, taxonomy compliance, internal handoff count.	Weekly / Daily	Operations & IT Leads
Experiential	Customer perception metrics tied to specific journeys or touchpoints. These measure the functional and technical quality as experienced by the user.	Touchpoint MOS (Moment of Satisfaction), CES (Customer Effort Score), CSAT, first-contact resolution (FCR) perception.	Bi-weekly / Monthly	Journey Owners

Financial	Commercial outcomes and macro-relationship health indicators. These are lag indicators. By the time these move, the damage is already done or the victory is secured.	NPS, Churn rate, Cost-to-Serve, Customer Lifetime Value (CLV), Avoidable failure demand cost, margin impact.	Quarterly / Monthly	Executive Council
------------------	---	--	---------------------	-------------------

In the SDAP environment, we enforce the creation of these trees using the `kpi-tree-design` method. We treat a KPI tree as a definitive artifact with strict schema contracts. Our Critic agent is instructed to unconditionally reject any KPI that is missing a tier classification or uses a placeholder like "TBD" or "unscored". You cannot manage what you cannot measure, and you cannot measure what you refuse to specify.

```
suite/packages/knowledge/data/methods/kpi-tree-design.json
```

```
{
  "id": "kpi-tree-design",
  "name": "KPI Tree Design",
  "phase": "measurement",
  "sources": [
    { "book": "20-the-service-profit-chain", "chapter": "05" },
    { "book": "01-this-is-service-design-doing", "chapter": "07" }
  ],
  "inputs": [
    { "kind": "artifact", "min": 1, "note": "a service blueprint defining the outcomes to measure" }
  ],
  "output": { "kind": "kpi-tree" },
  "steps": [
    { "role": "synthesizer", "instruction": "Derive a tree of KPIs from the blueprint's outcomes, tagging each KPI with the tier it belongs to: operational, experience, or financial." },
    { "role": "critic", "instruction": "Reject any KPI whose tier is missing or a placeholder, since an untiered KPI cannot be rolled up to the outcome it serves.", "gate": true }
  ],
  "acceptance": [
    { "rule": "field-non-empty", "field": "tier" },
    { "rule": "forbid-field-values", "field": "tier", "deny": ["tbd", "unscored"] }
  ],
  { "rule": "min-nodes", "min": 4 }
],
  "evidenceRules": { "traceability": "required" },
}
```



```
"owner": "practice"  
}
```

The causal logic of the KPI tree is non-negotiable: If the Operational tier (e.g., owner assigned within 24 hours) fails, the Experiential tier (e.g., CES on case resolution) will suffer. If the Experiential tier suffers repeatedly, the Financial tier (e.g., account retention and lifetime value) bleeds. By mapping these linkages explicitly, we stop treating CX as a soft science relying on subjective feelings, and start treating it as industrial engineering with predictable inputs and outputs. When an executive asks why churn is up (Financial), you trace the tree down to see that CSAT on the renewal journey dropped (Experiential) because the API facilitating automatic contract generation timed out 15% of the time last month (Operational).

Forrester's 6 CX Maturity Disciplines & Federated Governance

Measurement alone is just observation; it does not fix anything. To enact change, you need governance. Forrester's seminal work *Outside In* (Manning & Bodine) posits that Customer Experience is not a project; it is an enterprise capability consisting of six reinforcing disciplines. A failure in any one discipline caps the maturity of the entire organization. This is known as the limiting-discipline rule.

1. **Strategy:** A shared vision of the intended experience that aligns with the corporate brand and guides prioritization. It answers the question: What kind of experience are we uniquely trying to deliver?
2. **Customer Understanding:** Systematic, continuous practices for uncovering customer needs, behaviors, and pain points (e.g., evidence-gated personas, transcript mining). It ensures the strategy is grounded in reality.
3. **Design:** The rigorous, repeatable processes for defining and prototyping experiences before they are built. This is where blueprints and journey maps are operationalized.
4. **Measurement:** The tiered KPI architecture we just discussed, providing visibility into quality at every level of the service stack.
5. **Governance:** The formal structures, forums, and decision rights required to resolve cross-functional conflicts, enforce standards, and hold leaders accountable.
6. **Culture:** The shared habits, hiring practices, training regimes, and incentive structures that make customer-centricity the default behavior of the organization.

In large, complex organizations—especially those with multiple distinct Business Units (BUs) operating on their own P&Ls—governance is almost universally the limiting discipline. BUs naturally optimize for their own margins, creating fractured experiences for customers who interact across

silo boundaries. You cannot solve this by deploying a massive, centralized CX police force; that simply breeds resentment, shadow IT, and passive resistance. Instead, we architect a **Federated Governance Model**.

Figure 16.2 The Federated Governance Model. Effective CX governance requires a small central standard-setting body, partnered with named journey owners embedded within the business units who hold the authority to resolve cross-silo defects.

The federated model separates the definition of standards from the delivery of the service. A small Central CX Office owns the measurement architecture, the standards library, and the review gates. However, the actual delivery and budget remain within the BUs. Crucially, every cross-BU journey is assigned a **Named Journey Owner**. This individual has the explicit authority to cross organizational boundaries, arbitrate technical decisions, and hold local operational managers accountable to the central standards.

NOTE

My take: The most politically dangerous part of standing up a governance model is the illusion of power. If you create a "CX Council" that only has the authority to make recommendations, you have created a talking shop, not a governing body. Governance without funding authority and PR-blocking review gates is completely useless. Your governance charter must dictate that no customer-facing release goes live unless it clears the standard. If you cannot block a release, you are just an advisor.

Case Study: Kestrel Energy & Titan Manufacturing

To demonstrate how these theoretical frameworks operate under live fire, we will examine two major engagements where we transitioned clients from ad hoc project delivery into mature, AI-augmented operating systems.

Kestrel Energy: Orchestrating the Quality Loop

Kestrel Energy, a major EU utility with 2.4 million household accounts and 180,000 SMEs, had a massive operational leak. Out of 760,000 annual service contacts, approximately 14% (106,400) were classified as avoidable failure demand. These contacts stemmed mostly from first-bill confusion following smart-meter installations, estimated vs. actual bill shock, and broken connection appointments. Kestrel was hemorrhaging €617k annually just to field calls that should never have happened. They had redesigned the touchpoints in Phase 1, but the quality kept degrading month over month. They needed a robust Service Quality Management (SQM) system.

We applied Grönroos's model explicitly to redefine how Kestrel tracked success. Kestrel was previously measuring success via average handling time (AHT) and basic CSAT. We blew that legacy scorecard up. We instrumented a Moment of Satisfaction (MOS) model for every touchpoint,

demanding separate scores for Technical Quality and Functional Quality. The formula favored functional quality slightly, recognizing that clarity, tone, and trust were the actual drivers of retention:

```
formula / Moment of Satisfaction (MOS)
```

```
MOS = round(((technical * 0.45) + (functional * 0.55) - 1) * 25)
```

We used our AI agents extensively to operationalize this. We pointed OpenCode at their disparate data lakes to join CRM logs, survey data, complaints, and billing operations into a single, unified view. The CLI command was aggressive in its parsing constraints, forcing the model to categorize failure rigorously:

```
terminal
```

```
asd run --agent build "Read contact_logs_12m.parquet, complaints.csv,
survey_events.csv, billing_ops.csv.
Create one episode table keyed by contact_id_hash.
For each row output: touchpoint, channel, avoidable_flag, mos_technical_score,
mos_functional_score, resolution_days, survey_score, root_cause_owner,
operating_unit.
Rules: avoidable means the contact would not be needed if Kestrel had met its
published standard; ambiguous rows must be tagged review_required. Export monthly
aggregates and a root-cause Pareto for billing and connections."
```

Claude Code was then deployed to author the actual standards library. It generated version-controlled YAML files for every core touchpoint (e.g., `first-bill-after-smart-meter.yml`). We then unleashed Codex as an adversarial critic to review the library. Codex was instructed to attack any standard that lacked a computable metric or promised something operations could not actually deliver. This forced Kestrel to stop writing aspirational marketing fluff (e.g., "We aim to delight our customers") and start writing hard, enforceable SLAs (e.g., "First-contact agent confirms issue, freezes collection risk, and gives a due-by time in the first interaction").

The results were transformative. By shifting the monthly review from a generic complaint autopsy into a structured root-cause analysis governed by the 3-Tier KPI tree, Kestrel dropped its avoidable failure demand from 14.0% to 8.6%. The billing-query MOS rose from 61 to 78, and crucially, frontline employee Net Promoter Score (eNPS) skyrocketed from +8 to +23. This proved the Service Profit Chain thesis: equipping employees with working systems and clear standards drives immense internal satisfaction, which flows directly into customer value.

Titan Manufacturing: B2B CX Maturity Governance

Titan Manufacturing presented a different beast altogether. A global industrial manufacturer with €2.18B in revenue across five highly autonomous Business Units (Packaging Systems, Flow Control, Industrial Drives, Process Heat, and Precision Components), Titan was torturing its multi-BU clients. A large food processing customer buying from the Packaging BU and the Flow Control BU faced multiple portal logins, separate service contracts, disjointed invoicing, and a complete lack of accountability when a problem spanned both domains. Only 42% of cross-BU cases received a named owner within one business day, leading to immense frustration and 31,200 annual avoidable cross-BU contacts.

We initiated a rigorous six-discipline maturity assessment. We conducted 52 stakeholder interviews and audited 340 operational artifacts. Our AI agents parsed the transcripts and extracted structured evidence against the Forrester disciplines. The assessment revealed that Titan was actually decent at Customer Understanding, but Measurement and Governance were dismal (scoring 1.6 out of 5). Using the limiting-discipline rule, Titan's enterprise maturity was capped at the Explore/Formalize boundary.

We designed a federated governance model to break the deadlock. We established an Executive CX and Service Council sponsored by the COO, not the IT department. We created a Journey Owner Forum and appointed named owners for cross-BU journeys, like the unified portal access experience. To prove the model, we tackled the portal login journey first. Customers were authenticating separately for different BU portals. We mapped a blueprint for a unified Titan workspace that provided single sign-on, site-level entitlements, and a cross-BU owner tile.

OpenCode was directed to build the telemetry pipeline to monitor this new governance structure:

```
terminal
```

```
asd run --agent build "Create a journey telemetry pipeline for Titan.  
Inputs: portal_identity.csv, crm_cases.csv, erp_contracts.csv, fsm_events.csv,  
dealer_escalations.csv.  
Outputs: journey_event_model, owner_assignment_sla, login_success_rate,  
renewal_effort_score, dealer_direct_cycle_time.  
Do not invent fields. Mark any missing join key explicitly."
```

We enforced the governance through GitHub pull requests. No digital team could merge a change affecting the login flow without linking it to the journey standard, naming the metric impact, and securing approval from the central Standards Board.

WARNING

CAUTION: When implementing federated governance, you will inevitably encounter aggressive resistance from BU leaders who view centralized standards as a direct threat to their autonomy. Frame governance as a shared service that eliminates duplicate costs and resolves boundary disputes, rather than an adversarial inspection regime.

The nine-month outcomes proved the model's validity emphatically. Cross-BU cases assigned a named owner within one business day jumped from 42% to 87%. Portal login success in the consolidation pilot improved from 78% to 94%. Most importantly, the Relationship NPS for Titan's top 500 accounts climbed from +9 to +16, unlocking millions in retained aftermarket revenue and securing the funding for the next 24 months of the transformation roadmap. Titan had successfully evolved from running isolated CX projects to operating a mature, self-governing service ecosystem.

17 SOPs-as-Code: The AI Execution Playbook

Institutionalizing service design into executable, versioned prompt-and-schema architectures

For decades, operational rigor was measured by the thickness of a corporate Standard Operating Procedure binder. It was a lie we all agreed to ignore. We wrote the process on a PDF, stored it on an intranet, and pretended that the front line was executing it. In reality, operators survived by inventing undocumented workarounds to overcome the gap between our pristine process models and the messy operational truth. The rise of autonomous AI agents demands an end to this theater. When machines run processes, they do not interpret intention—they execute what is written. If you give an agent a prose document, it will invent plausible hallucinations. If you give it a structured schema, it will execute with precision. In this chapter, I will show you how to obliterate the corporate SOP binder. We will convert your operational methods into executable, versioned prompt-and-schema architectures, allowing you to test service consistency, run automated compliance checks, and execute wave-gated rollouts with continuous integration. This is how you make service design run as reliable software.

The Failure of the Corporate SOP Binder

The traditional Standard Operating Procedure (SOP) is an artifact of despair. It is written by compliance teams who have not worked the floor in a decade, read by nobody, and weaponized only during post-incident reviews to prove that a frontline worker violated policy. The fundamental failure of the corporate SOP binder is that it is disconnected from the telemetry of the service itself. It describes an idealized flow of work while remaining completely blind to the actual queue states, exception events, and cycle times unfolding in production.

When service design methods are treated as prose-only instructions, they inherit this exact failure mode. If I write a method for extracting insights from a customer interview and I leave it as a set of bullet points, one designer will extract rigorous, evidence-backed tension statements, while another will summarize it into vague, generic tropes. The method lives partly in the document and mostly in the tacit judgment of the facilitator.

TIP

CORE PRINCIPLE: Prose-only methods create interpretive drift. Human judgment is hidden, and execution depends entirely on the facilitator's habits. An SOP must be an executable contract, not a suggestion.

When we introduce AI agents into this mix, the prose problem becomes catastrophic. LLMs are engineered to be helpful. If you ask an agent to "create a journey map from these transcripts," and your SOP is just a paragraph of prose, the agent will confidently generate a journey map. It will invent emotional scores, fabricate touchpoints to satisfy the template, and smooth over the contradictions in the evidence. It will produce something that looks incredibly polished and is structurally unsafe.

To fix this, we must stop treating SOPs as documents and start treating them as software code. We need an execution playbook where the instructions, the required inputs, the exact shape of the output, the acceptance criteria, and the review gates are encoded in a machine-readable format. When an SOP is code, an agent cannot free-style past the evidence. It must satisfy the schema.

The SOP-as-Code Lifecycle: Author → Test → Version → Execute → Audit

The SOP-as-code paradigm requires treating your process catalog exactly like a software codebase. We do not draft an SOP and email it to the team. We author it in YAML, generate JSON Schemas for its outputs, write unit tests against those schemas, version control it in Git, and execute it via CI/CD pipelines.

The lifecycle consists of five distinct phases. First, we author the SOP, defining its anatomy. Next, we test it against regression fixtures to ensure deterministic behavior. We then version it, requiring pull request reviews for any changes. Once merged, it is executed by orchestrating agents against live data. Finally, we audit the telemetry of its executions to track cycle times and failure rates.

Figure 17.1 SOP-as-Code pipeline: Natural Language Standard → Typed Schema + Prompt → Unit Test Suite → Git Versioning → Agent Execution. When operating procedures are code, service consistency can be tested and deployed with continuous integration.

NOTE

My take: The most significant cultural shift for a service design team is learning to write acceptance tests for their own methods. The first time a senior designer has their journey map rejected by a CI pipeline because they forgot to link an emotional score to an evidence hash, they usually curse the system. By the third time, they realize the system just saved them from presenting a hallucination to the board.

The anatomy of an executable SOP requires strict boundaries. It must contain eight non-negotiable components. If any of these are missing, it is a draft, not an SOP.

Component	Purpose	Machine Validation
Context & Principles	The operational bounds and ethical framing of the task.	Passed into the LLM system prompt as fixed invariants.
Trigger Condition	The exact event or state that initiates the process.	Event-stream listener configuration.
Required Inputs	The files, data streams, or artifacts needed to run.	Pre-execution check; halts run if dependencies are missing.
Step Instructions	The procedural actions, explicitly assigned to actors (agent or human).	Orchestration graph routing.
Output Schema	The exact data structure of the required deliverable.	JSON Schema validation.
Acceptance Criteria	The business and logic rules the output must satisfy.	Automated unit tests and Critic agent review.
Evidence Rules	The traceability requirements linking output to source data.	Content-addressed SHA-256 hash checks.
Review Gate	The named human or agent roles required to approve the output.	GitHub CODEOWNERS and PR approval requirements.

Let's look at a concrete example. When we conduct service blueprinting, we use an SOP-as-code to ensure that every single node on the blueprint traces back to a verified observation. Here is an excerpt of how that SOP is authored in YAML.

```
sop/SOP-BP-CORE.yaml
```

```
sop_id: SOP-BP-CORE
name: BlueprintBuild from validated journey spine
version: 1.2.0
phase: define
purpose: Build one service blueprint whose rows are evidence-gated and
operationally reviewable.
trigger_when:
  - validated_journey_spine_exists
inputs:
  - name: journey_spine
    required: true
    shape: markdown rows with stable column ids
  - name: evidence_register
    required: true
    shape: jsonl evidence items with source refs
steps:
  - id: load_spine
```



```

instruction: read customer actions and treat each as a column anchor
  - id: build_lane_rows
    actor: agent
    instruction: propose one row per lane per relevant column with evidence ids
output:
  format: mixed
  schema: schemas/blueprint_row.schema.json
acceptance_criteria:
  - every status OK row has at least one evidence_id
  - every fail point has a numeric frequency or status NEEDS_QUANT
  - no row contains resident identifying data
evidence_rules:
  quote_verification: if quote field present it must resolve verbatim
  practice_rule: documented policy cannot be sole support for status OK
review_gate:
  creator: mapper_agent
  reviewer: critic_and_domain_panel
  approver: service_owner

```

This YAML is not just documentation; it is the configuration file for the orchestrator. When the `asd` CLI runs this SOP, it uses these exact keys to route work, parse outputs against the schema, and invoke the Critic agent if the `evidence\_rules` are violated.

Schema Specifications for Service Handoffs and Exception Rules

The heart of an executable SOP is its output schema. If you allow an agent to output unstructured markdown, you cannot programmatically verify its work. You cannot write a script to check if every pain point has an owner, or if every timeline event has a duration. You are forced to rely on human reading, which defeats the purpose of autonomous execution at scale.

We enforce rigor by requiring agents to output JSON that strictly adheres to a predefined JSON Schema. This schema acts as a structural contract for service handoffs. If the agent's output fails the schema validation, the execution loop is aborted, and the agent is forced to retry with the validation errors provided in its context.

TIP

CORE PRINCIPLE: The schema is the boundary object. It forces the LLM to commit to specific values, classifications, and linkages, exposing its reasoning to deterministic verification tools.

Consider the `BlueprintRowCollection` schema used in conjunction with `SOP-BP-CORE`. It dictates exactly what a valid blueprint node looks like.

schemas/blueprint_row.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "BlueprintRowCollection",
  "type": "array",
  "items": {
    "type": "object",
    "required": ["node_id", "lane", "column_id", "label", "evidence_ids",
"status"],
    "properties": {
      "node_id": {"type": "string", "pattern": "^[A-Z0-9_-]+$"},
      "lane": {
        "type": "string",
        "enum": ["physical_evidence", "customer_actions", "frontstage",
"backstage", "support"]
      },
      "column_id": {"type": "string"},
      "label": {"type": "string", "minLength": 3},
      "evidence_ids": {"type": "array", "items": {"type": "string"}},
      "status": {"type": "string", "enum": ["OK", "GAP", "NEEDS_QUANT"]},
      "fail_tag": {"type": ["string", "null"]},
      "duration_min": {"type": ["number", "null"], "minimum": 0},
      "frequency_per_month": {"type": ["number", "null"], "minimum": 0}
    },
    "allOf": [
      {
        "if": {"properties": {"status": {"const": "OK"}}},
        "then": {"properties": {"evidence_ids": {"minItems": 1}}}
      ]
    ]
  }
}

```

Notice the `allOf` condition at the bottom. This is the critical enforcement mechanism. It states, deterministically, that if a blueprint node's status is `OK`, it **MUST** contain an array of `evidence\_ids` with at least one item. A designer cannot just assert that a backstage process works fine without citing the exact log or interview transcript proving it. An agent cannot hallucinate a smooth handoff. If they try, the JSON Schema validator throws an error and blocks the merge.

We apply this same schema-driven rigor to exception rules. When an operational process breaks—for instance, a temperature excursion in a cold-chain logistics network—the handling of that exception must not be left to the improvisational skills of a local shift manager. The exception rule itself must be encoded as a schema, detailing the severity, the routing logic, and the maximum allowable resolution time before escalation.

NOTE

My take: I have sat in too many operational post-mortems where the root cause of a catastrophic service failure was "the SOP was unclear on who owned the exception." When exception rules are typed schemas, ownership is a mandatory string field. You literally cannot deploy the process if the owner is undefined. It forces the organization to solve political ambiguities in the repo, rather than on the warehouse floor at 2 AM.

Wave-Gated Rollouts and Change Risk Management

Encoding your SOPs as executable schemas solves the problem of defining the process. But deploying that process into a live, messy operational environment carries immense risk. If you push a flawed change to a network of twelve warehouses simultaneously, you don't just create a local disruption; you create a systemic cascading failure that will impact thousands of customer promises within hours.

To manage this, we encode the rollout strategy itself as a structured method. We utilize the concept of Wave-Gated Rollouts. This dictates that any operational change must be sequenced into discrete waves, scoped to an independent site, region, or segment. Crucially, each wave cannot proceed until specific, measurable exit criteria are met and signed off by a named owner.

We enforce this through the `wave-gated-rollout.json` method file, which acts as the ruleset for generating implementation roadmaps.

methods/wave-gated-rollout.json

```
{
  "id": "wave-gated-rollout",
  "name": "Wave-Gated Rollout",
  "phase": "delivery",
  "inputs": [
    { "kind": "artifact", "min": 1, "note": "an implementation roadmap or backlog to sequence into waves" }
  ],
  "output": { "kind": "roadmap" },
  "steps": [
    { "role": "synthesizer", "instruction": "Group the backlog into rollout waves, each scoped to a site, region or segment that can go live independently." },
    { "role": "critic", "instruction": "Reject any wave with no exit criteria owner, since an ungated wave cannot be stopped before it spreads a defect." },
    "gate": true }
  ],
  "acceptance": [
```

```
{ "rule": "field-non-empty", "field": "wave" },
  { "rule": "forbid-field-values", "field": "owner", "deny": ["tbd",
"unassigned"] }
],
  "reviewGate": "critic"
}
```

The `critic` step is the absolute safeguard. The orchestrating AI will construct the rollout waves, but the Critic agent will deterministically evaluate the output against the `acceptance` rules. If the `owner` field contains "tbd" or "unassigned", the Critic vetoes the artifact. An ungated wave is a disaster waiting to happen; by forcing the assignment of a human owner for every gate, we ensure that risk management is actively managed, not passively hoped for.

Similarly, the actual components of the change—communication, training, and system cutovers—must be authored with the same rigorous assignment of responsibility. We use `change-pack-authoring.json` to ensure every discrete action of a change has an owner and a non-evasive status.

methods/change-pack-authoring.json

```
{
  "id": "change-pack-authoring",
  "name": "Change Pack Authoring",
  "phase": "delivery",
  "inputs": [
    { "kind": "artifact", "min": 1, "note": "a future-state blueprint or backlog
describing what is changing" }
  ],
  "output": { "kind": "change-plan" },
  "steps": [
    { "role": "synthesizer", "instruction": "Break the change into discrete
actions covering communication, training and system cutover, each with an owner
and status." },
    { "role": "critic", "instruction": "Reject any action with no owner, since an
unowned change action will not happen.", "gate": true }
  ],
  "acceptance": [
    { "rule": "min-nodes", "min": 3 },
    { "rule": "field-non-empty", "field": "owner" },
    { "rule": "forbid-field-values", "field": "status", "deny": ["tbd"] }
  ]
}
```

This is change risk management executed at machine speed. The human designers and operational leaders define the parameters of the change, but the automated pipelines and Critic agents enforce the structural integrity of the rollout plan. It prevents the insidious organizational

habit of launching a change with undefined accountability.

Case Study: Halcyon Logistics Order-to-Cash Transformation

To see the power of SOPs-as-code in a physical, high-stakes environment, we must examine the transformation of Halcyon Logistics. Halcyon is a European third-party logistics provider (3PL) operating 12 warehouses and processing 1.84 million shipments annually. Following a successful pilot that improved their customer SLA attainment to 93.1%, the board faced a grim reality: the improvement was manufactured entirely by the heroic, manual efforts of account managers and dispatchers compensating for a fractured operational backbone.

Halcyon had 17 local variants of their outbound shipment and returns process. Order release, dock appointments, and proof-of-delivery (POD) closure differed wildly across sites. They possessed static PDF SOPs, but these documents had zero connection to the WMS or TMS event data. The result was rampant local optimization that destroyed end-to-end flow. The median outbound cycle time sat at a sluggish 11.6 hours, and first-pass yield was only 91.2%.

The mandate was to redesign the order-to-cash backbone so the service promise was generated by the system, not by human heroics. We achieved this by throwing out the PDF binders and replacing them with a versioned process catalog in GitHub.

WARNING

CAUTION: Do not attempt to digitize a broken process. If your current state is defined by undocumented workarounds and invisible queues, encoding it into schemas will only crystallize the dysfunction. You must first map the operational truth and design the future state blueprint before writing the SOP-as-code.

We defined the future-state blueprint and then encoded the critical operational controls as machine-readable standard templates. Every repeatable process became a versioned object in the `halcyon-process-library` repository.

Here is an excerpt from the `process.outbound-dispatch-release.md` standard, encoded in YAML format to serve as both human documentation and machine configuration:

```
halcyon-process-library/process.outbound-dispatch-release.md

process_id: P-OUT-06
name: outbound dispatch release
journey_scope: order to cash
version: 1.2.0
owner: order_to_cash_process_owner
trigger: order_status = packed AND dock_slot_assigned = true
```

```

- pick_complete_scan
- pack_manifest
- trailer_id
- temperature_status_if_applicable
outputs:
- dispatch_confirm_event
- visible_milestone_to_customer
service_standard:
  cycle_time_target: dispatch release within 18 min of dock ready
  first_pass_yield_target: 98 percent
  queue_threshold: no more than 35 orders waiting > 45 min per site
steps:
- id: gate_1
  actor: wms
  action: verify all handling units scanned
  telemetry_event: release_scan_check
  fail_safe: block release if any unit missing
- id: gate_3
  actor: qa_bot
  action: auto hold cold_chain loads if temperature out_of_band_minutes > 5
  telemetry_event: temp_hold
  fail_safe: no manual override without qa signoff

```

This code was the operational reality. By linking these documented steps directly to telemetry hooks in the WMS and TMS, we established explicit "poka-yoke" controls. `gate\_1` became a hard scan-to-load release rule; `gate\_3` became an automated hold on any cold-chain load experiencing a temperature excursion. These were no longer suggestions in a binder; they were digital barriers.

To execute the change without collapsing the network, we utilized the wave-gated rollout methodology. The pilot was constrained to three sites—Eindhoven, Venlo, and Łódź—representing a painful mix of retail peaks and industrial complexity. We built a mocked event-stream layer using OpenCode, allowing us to test the future-state controls against historical WMS data before flipping a single switch in production.

NOTE

My take: The most contentious moment in the Halcyon engagement wasn't defining the new process; it was enforcing the review gates. When a site manager tried to unilaterally bypass a scan step to clear a midday backlog, the automated queue monitor flagged the variance, and the resulting PR to change the standard required the signature of the global Order-to-Cash Process Owner. For the first time, local variation was visible and subject to global governance.

The results of treating SOPs as executable, versioned code were profound. Over an eight-week live pilot, the structural enforcement of these rules eradicated the silent queues and rework loops. The metrics tell the story of a system brought under rigorous control.

Operational Metric	Before SOPs-as-Code	After Pilot Execution	Impact
Median Outbound Cycle Time	11.6 hours	8.4 hours	Dramatically faster throughput.
First-Pass Yield	91.2%	97.1%	Massive reduction in rework and relabel loops.
Failure Demand (WISMO)	17.1 per 1,000 shipments	10.2 per 1,000 shipments	Proactive controls eliminated blind spots.
Labor Cost per Shipped Unit	€4.82	€4.31	10.6% reduction in operational cost.
Returns Disposition Time	6.8 days	3.2 days	Blind cages replaced by strict FIFO clocks.

The transformation was not merely technical; it required a profound organizational shift. When operational controls are hard-coded into machine-executable standards, the political ambiguities that traditionally grease the wheels of warehouse operations are laid bare. In the past, if a wave release was delayed because of an upstream failure in order cut-off times, the site manager would absorb the delay, ask their team to work through lunch, and the failure would remain invisible to the broader network. With SOPs-as-code, the ``process.outbound-dispatch-release.md`` standard explicitly defines the maximum allowable queue time and automatically triggers a severity-1 alert when breached. The failure is broadcast immediately, and more importantly, the root cause is traced back to the exact system state that caused it.

This level of transparency initially sparked fierce resistance. Middle managers felt they were being micromanaged by an algorithmic overseer. The critical breakthrough came from redefining the role of the Critic agent and the overarching governance structure. The Critic agent was not deployed to punish operators; it was deployed to enforce the integrity of the process definitions themselves. When an operator submitted a pull request to alter a queue threshold because they claimed a specific account required longer processing times, the Critic agent evaluated the request against the historical telemetry data. If the data did not support the claim, the Critic rejected the PR, demanding evidence. This shifted the burden of proof from human supervisors to the data, depoliticizing process changes.

The wave-gated rollout methodology also proved essential for building trust. By strictly isolating the initial pilot to Eindhoven, Venlo, and Łódź, and enforcing hard exit criteria before expanding, the broader network was able to witness the benefits without enduring the initial friction of the cutover. The exit criteria were meticulously defined in ``wave-gated-rollout.json`` schemas, ensuring that any

localized defects were caught and remediated. Only when the pilot sites demonstrated a sustained 97.1% first-pass yield and a reduction in median cycle time to 8.4 hours did the global Order-to-Cash Process Owner authorize the next wave of deployments.

NOTE

My take: The most significant victory of the Halcyon project wasn't the 10.6% reduction in labor cost, though the board certainly loved that. It was the fact that for the first time in the company's history, the operations team and the digital product team were speaking the exact same language. The SOP was the code, and the code was the SOP. They were finally building the business together, in the same repository.

Ultimately, the Halcyon case study proves that the transition from static service design deliverables to living, agent-driven operational architectures is commercially imperative. The organizations that thrive in the next decade will be those that recognize their operating procedures are their most critical software assets. They will stop writing prose that nobody reads and start writing schemas that machines can execute. They will institutionalize their service design into executable, versioned prompt-and-schema architectures, ensuring their customer promises become structurally guaranteed rather than aspirational slogans.

The era of the prose SOP is over. The future of service delivery belongs to teams who can author their operational intent as code, test it rigorously, and orchestrate its execution with autonomous precision. When you turn your methods into schemas and your procedures into pipelines, you stop hoping for consistency and start engineering it. The playbook is open. The tools are ready. The only remaining question is whether you are prepared to build services that actually execute exactly as you designed them.

18 Frontline Enablement & Employee Experience (EX)

Co-designing assistive agentic tooling with frontline staff

The hardest truth in service design is that you cannot mandate warmth from an exhausted workforce. When we deploy agentic systems, the instinct is to automate the customer interaction directly. But the most transformative applications of AI in service design happen backstage, providing superhuman memory and context to the human cast. In this chapter, I will unpack how to design internal service quality, build AI co-pilots that reduce cognitive fatigue rather than micromanaging it, and navigate the ethical boundaries of union collaboration. We will look at how Calder Community Bank saved 17 administrative minutes per customer interaction, and how Sable & Slate turned fragmented luxury hospitality into a unified "butler's memory" that dropped check-in times to four minutes.

Frontline Workers Are the Service: Internal Service Quality

The Service Profit Chain begins on the inside of the organization. Heskett, Sasser, and Schlesinger mapped this out in 1997, and the math has not changed: internal service quality drives employee satisfaction, which drives external value, which creates customer loyalty. Yet, in the rush to digitize, organizations routinely ignore the internal tooling environment. We treat customer apps as products and employee tools as an IT afterthought.

I have shadowed enough frontline workers to know that the "Cycle of Failure" is alive and well. A customer walks into a bank branch, already anxious about a loan application. The adviser, trying to project competence, is actually fighting a losing battle against thirteen different legacy systems. They are swivel-chairing between a CRM, a policy repository, a KYC tracker, and an ancient terminal emulator. Every time they switch screens, they drop a thread of the customer's narrative. This is not a training problem. It is a service design failure.

TIP

CORE PRINCIPLE: The Satisfaction Mirror dictates that employee satisfaction and customer satisfaction directly reflect one another. A frustrated employee cannot deliver empathetic service.

If we want to fix the customer experience, we must start by designing the employee experience. This means treating the frontline worker as the primary user of the service operating system. It means mapping their workarounds, quantifying their administrative burden, and actively designing systems that absorb complexity rather than passing it down to the lowest-paid person in the room.

Consider the internal service quality diagnostic. Before writing a single line of agentic code, you must measure the baseline:

Diagnostic Category	Audit Question	Common Symptom
Tool & System Adequacy	Do frontline systems provide fast, reliable context?	12+ tool switches per interaction.
Job Design & Autonomy	Do employees have authority to resolve issues?	Escalation required for trivial appeasements.
Backstage Support	Do IT/HR/Credit respond rapidly to the frontline?	Status blindness; "I'll have to check and call you back."

Designing AI Co-Pilots That Reduce Cognitive Fatigue

When we introduce AI to the frontline, the design pattern matters immensely. If we build a system that tells the employee exactly what to say, we degrade them into meat-routers. We induce cognitive fatigue by forcing them to constantly mediate between a prescriptive machine and a fluid human conversation.

The alternative is the *assistive co-pilot*. The co-pilot does not script the interaction; it surfaces context. It handles the heavy lifting of information retrieval, summarization, and policy verification, leaving the human to handle empathy, judgment, and connection.

NOTE

My take: I have seen too many "AI agent" rollouts fail because they tried to make the employee obsolete. The goal is not to replace the human; it is to make the human look like a genius. If your AI is generating literal scripts for the frontline to read, you are designing a dystopia. Let the machine fetch the policy; let the human read the room.

Let's look at Calder Community Bank. In our shadowing sessions, we found that branch advisers were spending 41 administrative minutes per customer interaction. They were making 6.8 internal calls or chase messages a day. The pain wasn't that they didn't know how to do their jobs; the pain was that they were acting as human API bridges between disconnected departments.

We designed a federated adviser desktop. It wasn't a massive core-banking replacement. It was a single pane of glass that aggregated customer context, open tasks, and document receipt states. Crucially, we introduced an approved-sources knowledge assistant.

```
knowledge-assistant-prompt.md
```

```
SYSTEM ROLE: CALDER APPROVED KNOWLEDGE ASSISTANT
```

```
Allowed sources only: policy pack, approved procedures, current branch FAQs.
```

```
When asked a question:
```

1. Retrieve only approved sources in the current data zone.
2. Answer in plain branch language.
3. Cite source ID, owner, and review date.
4. If answer is absent or conflicting, say: NOT APPROVED IN SOURCE PACK.
5. Suggest escalation route if needed.

Notice rule 4. The AI is explicitly forbidden from hallucinating or inferring. In a regulated environment, graceful incompleteness is far safer than confident fabrication. By strictly bounding the AI's discretion, we gave the advisers a tool they could trust, reducing administrative time from 41 to 24 minutes and lifting adviser eNPS from -18 to +11.

Context Handover: Giving Staff Superhuman Memory Without Screen Tyranny

Figure 18.1 Frontline Context Handover flow. Technology must empower the human cast rather than forcing customers and staff to stare at screens.

The true luxury in any service is not opulent surroundings; it is being remembered. But memory in a scaled enterprise is difficult. At Sable & Slate, a luxury boutique hotel group charging €680 a night, they had digitized their way to impersonality. The app was slick, but the staff were blind. Guests were re-telling their preferences to the front desk, the concierge, and the restaurant staff.

We introduced the concept of the "Butler's Memory." This was a staff-facing, guest-invisible context brief. Before a guest arrived, the agentic backend synthesized their consented history, stated preferences, and any open requests into a 30-second read.

The architectural flow for this context handover requires strict latency and relevance gating. You cannot dump a five-page CRM dossier on a front-desk agent who has ten seconds before the guest walks through the door.

```
context-synthesis.yaml
```

```
name: ButlersMemorySynthesis
trigger: onGuestArrivalWindow (-2 hours)
inputs:
  - guest_profile_db
  - previous_stay_transcripts
  - active_requests
```

```
- max_reading_time: 30s
- format: "[minimum] name + room + open requests + boundaries."
- rule: NO TRAIT INFERENCES. Facts only.
output: dispatch_to_staff_terminal
```

When the guest approaches, the ambassador reads the brief. They don't look at the screen while talking; they absorb the context and engage human-to-human. This express arrival dropped check-in times from 14 minutes down to 4 minutes. First-touch resolution jumped to 61%. By giving the staff superhuman memory, we restored the human reference point that luxury demands.

Union Collaboration and Ethical Automation Boundaries

You cannot deploy agentic telemetry into a unionized workforce without transparent, negotiated boundaries. The moment employees suspect that AI is being used for keystroke surveillance or individual performance ranking, trust evaporates, and the service quality plummets.

At Sable & Slate, we worked directly with the staff union to co-design the system. We established ironclad rules for the Critic agent and the overarching platform:

- **Aggregation Only:** Telemetry is used to identify systemic friction (e.g., "Step 4 takes 3 minutes on average"), never to rank individual associates.
- **Opt-In Consent:** For guests, context memory is strictly opt-in and revocable. For staff, the usage of assistive tools is a supported right, not a surveilled mandate.
- **No Algorithmic Discipline:** An AI cannot flag an employee for review.

NOTE

My take: If you try to sneak telemetry past a works council, you deserve the strike that will inevitably follow. Transparency is not just ethical; it is good engineering. The staff know where the system is broken better than any dashboard does. Treat them as co-designers, and they will help you build tools that actually work.

This collaboration extended to the operational rules. The union flagged that the night audit shift (3 AM) operates with a skeleton crew and cannot process a heavy contextual brief. We built a two-tier handover: a minimum brief for high-stress or low-staff scenarios, and a complete brief for standard operations. This wasn't an AI hallucination; it was human operational reality encoded into the system.

Case Study Deep-Dive: Calder Bank & Sable & Slate

Let's contrast these two deployments to see how agentic frontline enablement adapts to different operational realities.

Calder Community Bank: Here, the constraint was legacy architecture. We couldn't rip out the 2009 core system. The frontline enablement strategy was a *federated overlay*. The AI acted as a translation layer, pulling from disparate systems and presenting a unified case timeline. The measurable win was time: clawing back 17 minutes of admin time per customer, which allowed advisers to actually advise. The internal Service Profit Chain dashboard proved that this internal tooling investment directly drove a 12-point increase in repeat-customer NPS.

Sable & Slate: Here, the constraint was the brand promise of luxury and intimacy. The technology had to be entirely invisible to the guest. The agentic system operated purely backstage, executing the evidence-gated-blueprint method to ensure every touchpoint cited real operational constraints. The Critic agent actively vetoed any feature that attempted to infer a guest's emotional state ("guest seems anxious"), enforcing the rule that memory stores facts, not psychological dossiers.

```
critic-veto-log.json
```

```
{
  "rowKind": "defect",
  "severity": "high",
  "rule": "no-trait-inference",
  "message": "Auto-filled emotional inferences have a dossier feel and zero supporting rows.",
  "fix": "Memory stores facts, requests, and boundaries only; never trait judgments."
}
```

In both cases, the AI was not the star of the show. It was the stagehand, moving the props into place so the human actors could deliver a flawless performance. That is the essence of frontline enablement in the agentic era.

When we treat frontline staff as the primary beneficiaries of our service architecture, we unlock a level of quality that no customer-facing chatbot could ever achieve. The tools we build must respect their judgment, protect their attention, and amplify their empathy.

The Mechanics of the Internal Service Profit Chain

To truly understand why frontline enablement is non-negotiable, we have to look at the mechanics of Heskett's Service Profit Chain through an agentic lens. The chain postulates a direct, mathematically verifiable link between internal service quality and corporate growth. But how do

we measure 'internal quality' when the environment is saturated with AI tools?

We measure it through the reduction of failure demand on the employee. Failure demand is the work an employee must do because a system, a process, or another department failed to do its job correctly the first time. When an adviser at Calder Bank spends fifteen minutes chasing a credit approval via email because the CRM does not expose the true status of an application, that is internal failure demand. It drains morale, it extends customer wait times, and it destroys the illusion of competence.

Agentic systems excel at eradicating internal failure demand. By deploying an autonomous monitoring agent that constantly reconciles the CRM with the credit queue, we can surface the true status of an application to the adviser's dashboard before they even have to ask. This shifts the employee's cognitive load from *investigating* the system to *interpreting* the system for the customer.

TIP

CORE PRINCIPLE: Treat the employee's attention as a scarce, perishable resource. Every click, every search query, and every context switch degrades their ability to connect with the human sitting across from them.

This is where the concept of **Bounded Discretion** becomes critical. In traditional service design, we often talk about empowering employees. But empowerment without boundaries is just abandonment. If you tell a frontline worker "do whatever it takes to make the customer happy" but give them no clear policy guardrails or budget authority, you are setting them up for anxiety and failure.

Agentic tooling provides bounded discretion dynamically. When a guest at Sable & Slate experiences a service failure (e.g., a flooded bathroom), the Butler's Memory system doesn't just flag the issue; it proactively retrieves the approved service recovery matrix. It presents the front desk agent with pre-authorized compensation options based on the guest's tier and the severity of the failure. The agent still makes the final call, but they make it with the absolute certainty that they are operating within approved parameters. They are empowered, not abandoned.

Architecting the Agentic Desktop: A Technical Blueprint

Building a federated agentic desktop requires a precise separation of concerns. You cannot simply pipe a Large Language Model into your core banking system and hope for the best. The architecture must be layered, resilient, and aggressively audited.

Here is the architectural pattern we use for frontline AI co-pilots:

1. **The Systems of Record (SoR):** The underlying legacy databases (Core Banking, PMS, CRM). These remain the single source of truth for all transactional data. The agentic layer treats these as read-heavy, write-cautious APIs.
2. **The Context Aggregator:** A deterministic middleware layer that pulls data from the SoR, normalizes it, and constructs a unified state object for the current interaction.
3. **The Agentic Reasoner:** This is where the LLM operates. It takes the unified state object and applies specific skill manifests (like the `frontLine-enablement` or `knowledge-retrieval` skills) to synthesize the data into human-readable insights. It does not make transactional database writes directly.
4. **The Presentation Layer:** The UI that the frontline worker actually sees. It is clean, sparse, and organized around the immediate 'Job to Be Done'.

`agentic-overlay-architecture.mermaid`

```
graph TD
    A[Systems of Record] -->|Raw Data APIs| B(Context Aggregator)
    B -->|Normalized State| C{Agentic Reasoner}
    D[Approved Policy Store] -->|RAG Retrieval| C
    C -->|Synthesized Brief| E[Presentation Layer]
    E -->|Human Action| A
```

This architecture inherently protects the integrity of the business. Because the Agentic Reasoner is sandboxed between the Context Aggregator and the Presentation Layer, it cannot accidentally execute a rogue transaction. Its output is always mediated by the human operator, who reviews the synthesized brief and takes the final action in the Presentation Layer.

NOTE

My take: The most dangerous phrase in enterprise AI is "end-to-end automation." When you are dealing with complex human problems—a bereaved spouse closing an account, an anxious traveler missing a connection—end-to-end automation is a liability. You want human-in-the-loop architecture, and you want that loop to be as tight and informed as possible.

Continuous Diagnostics and the Review-Veto Loop

How do we ensure that these assistive tools don't drift into irrelevance or hallucination over time? We use the exact same rigorous evaluation loops on our internal tools as we do on our external products. In the Agentic Service Designer ecosystem, this is governed by the Critic agent.

Whenever a new workflow, policy script, or UI component is proposed for the frontline desktop, it must pass through the review gate. The Critic agent evaluates the proposal against the `frontLine-enablement` method manifest. This manifest includes strict acceptance criteria:

- Does this change reduce or increase the number of required clicks?
- Is the ownership of this new standard clearly assigned? (Any standard with "tbd" or "unassigned" ownership is automatically vetoed.)
- Does the output cite verifiable evidence from the shadowing transcripts or telemetry logs?

If a product manager attempts to push a new cross-selling prompt onto the branch adviser's screen without removing an existing administrative burden, the Critic will flag it as a violation of the cognitive load limits. The veto log persists as an artifact, forcing a conversation about priorities. The platform literally defends the frontline worker's attention span against corporate feature bloat.

The Economics of Frontline Enablement

Let us return to the financial reality. Executives often balk at the cost of bespoke internal tooling. They view employee experience as a "nice to have," a soft metric managed by HR with ping-pong tables and free snacks.

But the math is inescapable. At Calder Community Bank, annualized voluntary turnover among branch advisers was 29%. Every time an experienced adviser quit, the bank lost 16 weeks of productivity while a new hire ramped up, incurred massive recruitment costs, and suffered a spike in customer complaints due to novice errors. The true cost of turnover was staggering.

By deploying the federated desktop and the approved-knowledge assistant, we fundamentally changed the nature of the job. Advisers no longer felt like punching bags for broken systems. They felt competent, supported, and capable. In the pilot branches, eNPS swung from -18 to +11, and the annualized turnover trend dropped from 29% to below 20%. The hard savings in recruitment and lost productivity alone paid for the entire 14-week engagement.

And that doesn't even account for the revenue side. First-contact quality rose to 79%. Repeat-customer NPS jumped 12 points. When customers interacted with calm, confident advisers who actually had the answers at their fingertips, they bought more products and stayed longer.

This is the ultimate promise of agentic service design. We are not just building software; we are re-engineering the socio-technical fabric of the organization. We are using artificial intelligence to restore human dignity to the frontline, and in doing so, we are unlocking the premier economic catalyst in the service economy: a supported workforce that genuinely cares.

The tools we give our people are the truest reflection of our corporate values. If we give them fragmented, slow, punitive systems, we are telling them their time is worthless. If we give them intelligent, contextual, assistive co-pilots, we are telling them their judgment is invaluable. Design accordingly.

Overcoming the Fallacy of the Single System

One of the most persistent illusions in enterprise IT is the belief that all frontline problems will be solved by migrating to a single, monolithic platform. "Once we move everything to Salesforce," or "Once the new SAP rollout is finished," the narrative goes, "the frontline will finally have an effortless experience."

As service designers, we know this is a costly fantasy. Large enterprises acquire companies, launch new product lines, and accumulate regulatory requirements faster than any single platform can integrate them. The reality is always a patchwork. Frontline enablement focuses on designing the *useful illusion* of unity across fragmented tools rather than waiting for an idealized single platform.

The federated adviser desktop we built at Calder Bank is a prime example of this useful illusion. Underneath the hood, the bank still ran a 2009 core system, a separate KYC tracker, a siloed complaint management queue, and a legacy document vault. If we had waited for an IT transformation program to consolidate those backends, we would have been waiting for five years.

Instead, we used agentic orchestration to decouple the employee experience from the backend complexity. The agentic layer acts as a highly skilled concierge for the employee. When the adviser opens a customer profile, the agentic middleware concurrently queries the CRM, the core banking system, and the document vault, synthesizing the results into a single, coherent case timeline. The employee experiences a unified system, even though the enterprise architecture remains fragmented.

TIP

CORE PRINCIPLE: Shield the frontline from the org chart. Employees should not have to navigate internal silos to serve the customer standing in front of them.

Knowledge Retrieval vs. Knowledge Comprehension

A significant portion of frontline friction comes from policy navigation. In banking, insurance, and healthcare, the rules change constantly. Expecting a human to memorize thousands of pages of compliance guidelines is absurd, which is why employees rely on sticky notes, informal chat channels, and "hero" colleagues who happen to know the answer.

Traditional knowledge bases fail because they optimize for retrieval, not comprehension. A search for "SME onboarding requirements" might return fourteen different PDF documents. The employee still has to read, interpret, and cross-reference those documents while the customer waits.

Agentic knowledge assistants fundamentally change this dynamic. They don't just retrieve documents; they comprehend the query and synthesize a direct answer from the approved corpus. But this introduces a massive risk: hallucination. If an AI hallucinates a policy answer in a regulated environment, the liability is severe.

This is why the SOPs for the knowledge assistant at Calder were so draconian. The RAG (Retrieval-Augmented Generation) pipeline was strictly limited to sources marked explicitly as 'approved' by the knowledge manager. The agent was forced to output its confidence level and the exact source citations. If the agent could not find a definitive answer in the approved corpus, it was hard-coded to reply: "NOT APPROVED IN SOURCE PACK," and automatically route the query to a human subject matter expert.

This deliberate limitation—this planned incompleteness—is what made the tool successful. The advisers learned they could trust the AI because it never lied to them. It never tried to guess. It either gave them the exact, cited policy, or it admitted ignorance. In service design, predictable failure is always preferable to unpredictable success.

Measuring the Unmeasurable: The Impact of Confidence

How do you measure confidence? When a new hire at Calder Bank reduced their time-to-competence from 16 weeks to 9 weeks, what exactly changed? It wasn't that they typed faster. It was that they stopped second-guessing themselves.

In our shadowing sessions, we observed a phenomenon we called "performative certainty." A frontline worker, unsure of a policy, would confidently tell a customer one thing, then frantically message a supervisor in the background, praying they were right. If they were wrong, they had to call the customer back and apologize, destroying trust.

With the agentic co-pilot providing real-time, verified context and policy answers, performative certainty was replaced by actual certainty. The advisers could look the customer in the eye and make honest promises. "I can see your documents have been received by our credit team. The SLA for review is 48 hours. I will personally monitor the queue and call you on Thursday at 10 AM."

This shift from defensive posturing to proactive service is the most profound outcome of frontline enablement. It completely alters the emotional tenor of the interaction. The customer relaxes because they sense they are in competent hands. The employee relaxes because they aren't terrified of making a mistake. The transaction becomes a relationship.

The Role of the Branch Manager in an Agentic Era

When you automate the administrative drudgery and provide instant knowledge retrieval, what happens to middle management? Historically, branch managers spent their days acting as traffic controllers and policy encyclopedias. They approved trivial overrides and answered the same procedural questions repeatedly.

With the agentic system absorbing those tasks, the branch manager's role must be redesigned. They must transition from administrative supervisors to behavioral coaches.

As part of the Calder deployment, we introduced a weekly coaching cadence. The manager's dashboard did not show keystrokes or time-in-motion metrics. Instead, it aggregated the systemic friction the team was experiencing. It highlighted which case types were causing the most delays and which policy areas were triggering the most "NOT APPROVED IN SOURCE PACK" escalations.

The managers used this data to run targeted, 20-minute scenario labs. They roleplayed complex interactions, focusing entirely on empathy, tone, and de-escalation techniques—the deeply human skills that the AI cannot replicate. By elevating the manager's role from firefighting to coaching, we ensured that the technology adoption translated into sustained capability growth.

Conclusion: The Ethical Imperative of Frontline Design

We are at a critical juncture in the evolution of service design. Modern tooling—LLMs, autonomous agents, dynamic RAG pipelines—can fundamentally reshape the enterprise. But power is agnostic. It can be used to squeeze every last drop of efficiency out of a surveilled and commoditized workforce, or it can be used to elevate them.

The Agentic Service Designer chooses the latter. We recognize that the frontline worker is the beating heart of the service ecosystem. We refuse to design systems that treat them as failing algorithms. We design systems that treat them as human beings who deserve dignity, context, and support.

When we get this right, the results border on the magical. Check-in queues vanish. Administrative backlogs clear. Employee turnover plummets. And most importantly, the people delivering the service finally have the time and the mental space to actually care about the people receiving it.

That is not just good business. That is good design.

19 The 15 Principles of Good Public Services

Translating Lou Downe's Good Services canon into machine-auditable heuristics

Public services are not optional. If you need planning permission, a blue badge, or a council tax reduction, you cannot shop around. Because citizens are captive, the service must protect dignity, clarity, and procedural justice. In this chapter, I will show you how we translate Lou Downe's canonical 15 Principles of Good Services from aspirational goals into deterministic, machine-auditable heuristics using the Service Design Agent Platform (SDAP). You will learn how to turn abstract quality into broken build pipelines, forcing government to ship services that actually work.

The Stakes of Public Service Design

I have spent years inside government departments and local authorities watching brilliant policy fail at the point of delivery. Policy is an abstract intent; the service is the lived reality. When a resident struggles to understand a PDF checklist for a kitchen extension, they do not blame the "statutory framework." They blame the council. The failure pattern in public services is almost universally identical: organizations optimize for their statutory steps, not the whole outcome. We build systems that mirror our internal org charts, forcing the resident to play systems integrator across housing, planning, and revenue departments.

Public-service design changes the success criteria. It is not about conversion rates or maximizing engagement. It is about minimizing cognitive and administrative burden. Tim Brown's argument about a new social contract is particularly relevant here: we have to design with residents, case officers, and objectors, not merely digitize the internal process. But how do we enforce that? How do we prevent a "digital transformation" from simply putting a cleaner CSS skin over the same broken administrative seams?

TIP

CORE PRINCIPLE: You cannot improve what you cannot measure, and in service design, you cannot measure what you do not have evidence for. The 15 Principles of Good Services must become an observable standard, not a motivational poster.

This is where the SDAP steps in. By making handoffs artifacts instead of chat, and by using the Critic agent to veto any claim lacking content-addressed evidence, we turn design principles into CI/CD gates. When a service fails to explain a decision in plain language, the build breaks. When an assisted-digital path is treated as an afterthought, the prototype is rejected.

The 15 Principles of Good Services Decoded

Lou Downe's book *Good Services* gave the industry a vocabulary. Our job is to give it teeth. Let us decode the 15 principles into measurable, observable tests that an agentic system can audit.

Figure 19.1 The 15 Principles. A good service is one that enables a user to achieve their goal with the minimum cognitive and administrative burden.

- 1. Easy to find.** A service must use the natural vocabulary of the user. If a resident searches for "kitchen extension rules," they should not hit a dead end because the council calls it "Householder Planning Consent Part 3." We test this by measuring search success against natural language logs.
- 2. Explains its purpose.** Within 10 seconds of landing on the start page, a user must be able to state who the service is for and what it does. If they cannot, the service fails.
- 3. Sets expectations.** The service must show time, cost, required documents, and consultation stages before the user clicks "Start." We measure this by checking if users can recall the prerequisites without creating an account.
- 4. Completes the outcome.** The resident's goal is not "submit a form." Their goal is "get permission to build." The service must cover submission, validation, consultation, decision, and next steps.
- 5. Works in a way that is familiar.** Reuse standard patterns. Do not invent a new date picker for the waste department when the planning department already has a tested one.
- 6. Requires no prior knowledge.** Strip out the internal jargon. If you must use a statutory term, define it inline with examples. We audit this using plain-language linters and reading-age scores.
- 7. Agnostic of organizational structures.** The resident should experience one timeline, regardless of whether their case is bouncing between planners, statutory consultees, or external partners.
- 8. Requires the minimum possible steps.** Never ask a user for information the government already holds. Implement document pre-checks to reduce invalid first submissions.
- 9. Consistent throughout.** The tone, terminology, and SLA language must be identical across the web portal, formal PDF letters, and contact centre phone scripts.

NOTE

My take: Principle 9 is where most local authorities bleed trust. You build a beautiful, empathetic web front-end, but when the backend casework system generates a statutory refusal letter, it sounds like it was written by a Victorian bailiff. If you do not own the backend letters, you have not designed the service.

10. Has no dead ends. Every invalid state, timeout, objection, refusal, or conditional path must offer a clear next step and human help.

11. Usable by everyone, equally. Universal accessibility is non-negotiable. The assisted-digital route (phone or in-person) must use the same core service and provide the same level of dignity.

12. Responds to change quickly. A user must be able to amend, reopen, and update their case through their account without starting from scratch.

13. Explains why a decision was made. A decision explainer must pair the legal notice with plain-language reasons and an explicit appeal route.

14. Makes it easy to get human assistance. Human support must be visible on every unhappy path, and the phone adviser must be able to see the exact draft and timeline the resident is looking at.

15. Asks for the right knowledge only. Ask for facts and intent, not obscure planning classifications. We enforce this through strict data-burden audits.

Automating Service Standards Audits (good-services-audit-15)

We operationalize these principles through the `good-services-audit-15` skill. This is a measurement phase method that takes a shipped map artifact and post-launch evidence-set as inputs, and produces a scored insight-set.

```
good-services-audit-15.json
```

```
{
  "id": "good-services-audit-15",
  "name": "Good Services 15-Principle Audit",
  "phase": "measurement",
  "output": { "kind": "insight-set" },
  "steps": [
    { "role": "researcher", "instruction": "Score the live service against each of the fifteen Good Services principles, citing evidence for every score below target." },
    { "role": "critic", "instruction": "Reject any principle with a below-target score and no cited evidence, and require all fifteen principles to be scored, not
```



```
a sample.", "gate": true }
],
"acceptance": [
  { "rule": "min-nodes", "min": 15 },
  { "rule": "field-non-empty", "field": "principle" },
  { "rule": "field-non-empty", "field": "evidence" }
],
"reviewGate": "critic"
}
```

Notice the mechanics here. The Researcher agent does the initial scoring, but the Critic agent sits on the write path as a hard gate. If the service scores a 0 or 1 on "Completes the outcome" (Principle 4) or "Minimum steps" (Principle 8), the Critic demands content-addressed evidence—a quote from a user interview or a telemetry log proving the friction. You cannot just guess. The platform mechanically forces evidence resolution.

Designing for Universal Accessibility, Dignity, and Trust

Trust in government is easily lost and incredibly expensive to regain. When we design for the public sector, we are designing for users in states of distress, confusion, and high cognitive load.

Consider Ron, a 72-year-old retired bus driver who wants to replace a conservatory. He is digitally shy. He prefers phone calls and printed letters. If we design a service that forces him onto a web portal he cannot navigate, we have not transformed the service; we have just shifted the failure demand from the front desk to the complaints department.

This is why Principle 11 (Usable by everyone, equally) and Principle 14 (Easy to get human assistance) are load-bearing pillars of the architecture. The assisted-digital route cannot be a shadow process running in a separate spreadsheet. It must write into the exact same case record as the digital route. When Ron calls the contact centre, the adviser should see his saved draft and be able to complete it with him, without him having to repeat his name, address, and intent.

NOTE

My take: The biggest lie in public sector digital transformation is "channel shift." You cannot mandate a channel shift for a captive audience without leaving the most vulnerable behind. A truly good service is channel-agnostic. It lets the user start on the web, continue on the phone, and finish with a printed letter, while maintaining a single, unbroken timeline backstage.

Case Study: Fernwood Council's Planning & Citizen Account Transformation

To see how this works in practice, let us look at Fernwood Council. Fernwood handles roughly 1,260 planning applications a year. The pre-redesign baseline was bleak: 34% of householder applications were invalid at first submission, residents checked case progress an average of 7.8 times per case, and formal decision letters satisfied legal process but failed plain-language comprehension.

We ran the engagement AI-first using Claude Code, the SDAP plugin, OpenCode, and Codex. We treated the 15 principles as a measurable standard. We did not just redesign the screens; we redesigned the service owner model.

Metric	Baseline	After 16-Week Beta
Mean decision days	50.5	35.7
Mean progress checks per case	7.8	2.7
Completion within service commitment	58%	83%
Digital take-up	61%	79%
Good Services Audit Score	28/75	54/75

The turning point in the Fernwood engagement was defining the standards register. Every principle was translated into an observable test. Principle 13 (Explains decisions) became a four-part explanation template tested with refused applicants. The Critic agent enforced these tests in CI. We built a GitHub action that ran automated accessibility, copy-pattern, and standards checks on every PR.

```
service-standard-gate.yml
```

```
on: [pull_request]
jobs:
  planning-standard-gate:
    steps:
      - uses: actions/checkout@v4
      - run: node scripts/check_principles_register.mjs standards/register.md
      - run: node scripts/check_mermaid.mjs README.md
      - run: pnpm axe prototype --rules wcag2aa,wcag22aa
      - run: node scripts/check_plain_language.mjs prototype/content/
      - run: node scripts/check_unhappy_paths.mjs prototype/spec/failure-
states.yml
```

We prototyped the entire end-to-end service. Not just the happy path. We prototyped the start pages, eligibility guidance, document checklists, upload flows, status updates, neighbour-notification emails, objection acknowledgements, refusal letters, and the case officer message template. We ran scenario-based tests for three routes: a standard householder extension, a conservation-area objection, and an assisted-digital application by phone.

The result was a unified citizen account. A resident logging in could check their planning application, report a missed bin, and pay their council tax—all using the same timeline pattern, the same identity layer, and the same standard of clarity. The planning service was no longer an opaque administrative ritual; it was a coherent public service.

The honest boundary here is crucial. The AI agents did not solve the politics of union consultation or planning committee resistance. That remained my job. What the agents did was hold the line on evidence. When an officer suggested hiding objection details until the decision stage to make the UI "simpler," the Critic agent flagged it as a breach of procedural fairness and Principle 10. The platform mechanically forced us to confront our own compromises.

NOTE

My take: The magic of the Agentic Service Designer is not that it generates perfect services. It is that it makes it mathematically impossible to hide a bad service behind good visual design. If your service fails the 15 principles, the engine will produce the receipts.

When you build public services, you are building the interface between the state and the citizen. It is a profound responsibility. By integrating Lou Downe's 15 Principles into the SDAP rigour engine, we ensure that every interaction—whether an immediate digital approval or a painful statutory refusal—is handled with transparency, dignity, and respect for the user's time. That is the new social contract in action.

Deep Dive: Evaluating the 15 Principles in the Wild

Let us look closely at how the agentic stack evaluates these principles in practice. The theoretical alignment is straightforward, but the operational enforcement is where the SDAP proves its worth.

Principle 1: Easy to find. In the Fernwood baseline, 10 out of 16 applicants searched for terms like "extension" or "loft conversion," not "planning application." The Researcher agent mapped these verbatim search queries to the drop-off rates on the council's homepage. The solution was not SEO optimization; it was renaming the service entry points to match user intent. The Critic gate enforced that the new service card in the citizen account was accessible via natural vocabulary.

Principle 2: Explains its purpose. The "Check before you start" panel is a masterclass in this. Before a user creates an account, they see a single panel explaining exactly what the service does, who it is for, and what it cannot do. The observable test is the 10-second comprehension check. If user research transcripts show confusion, the `good-services-audit-15` skill forces a score deduction.

Principle 3: Sets expectations. Public services often hide the cost and timeline until page 14 of a form. This is hostile. Fernwood's redesigned start page exposes the timeline, the fees, and the required drawings immediately. By setting expectations early, we saw mean progress checks drop from 7.8 to 2.7. Users only call to ask "where is my application?" when the service fails to tell them.

Principle 4: Completes the outcome. The service lifecycle does not end when the user clicks submit. It ends when they understand the decision and take the next step. The mapping agents (`@mapper`) enforce this by extending the journey map through to the "After" phase. If a blueprint ends at "Submission Received," the Critic vetoes it for failing to complete the outcome.

Principle 5: Works in a way that is familiar. We do not need innovation in form fields. We need predictability. By standardizing the citizen account shell across planning, repairs, and waste, Fernwood residents learned the UI once. The engine's consistency checks flag custom components that deviate from the established design system.

Principle 6: Requires no prior knowledge. Planning jargon is exclusionary. Terms like "Article 4 Direction" or "Curtilage" alienate users. We ran the `transcript-mining` skill over call centre logs and identified the exact phrases that triggered failure demand. We then enforced plain-language replacements in the content registry. If a statutory term is unavoidable, it is defined inline, with examples.

Principle 7: Agnostic of organizational structures. The user does not care that the planning officer and the conservation officer report to different directors. They just want a decision. Our ecosystem value mapping exposed the internal silos. The redesigned service presents a single unified timeline to the resident, masking the internal routing and handoffs happening in the backstage lanes of the blueprint.

Principle 8: Requires the minimum possible steps. Every extra field is a point of failure. We implemented a document pre-check flow that catches missing drawings before formal submission. This intervention alone drove the invalid-first-submit rate down drastically. The `minimum-steps` rule in the audit matrix deducts points for any data requested that the council already holds in its GIS or tax systems.

Principle 9: Consistent throughout. The omnichannel experience is usually a myth. A resident gets a polite web interface and a draconian PDF in the mail. We centralized the content registry. The web copy, the email notifications, and the call centre scripts all pull from the same source of truth. Consistency is not just branding; it is operational reliability.

Principle 10: Has no dead ends. Unhappy paths are where public services truly fail. If an application is invalid, the user must be told exactly how to fix it, right there. If a session times out, the draft must be saved. The Prototyper agent is instructed to build and test failure states explicitly. A prototype without unhappy paths is rejected at the Beta readiness gate.

Principle 11: Usable by everyone, equally. We do not just run automated WCAG linters. We require inclusive walkthroughs with screen-reader users and low-literacy cohorts. The assisted-digital route is treated as a core feature, not an edge case. If the phone adviser cannot see the user's screen state, the service is not equal.

Principle 12: Responds to change quickly. Life happens. A resident might need to amend a drawing or update their contact details. The legacy system required them to withdraw and reapply. The new citizen account allows in-place amendments without restarting the timeline, drastically reducing the cycle time for minor changes.

Principle 13: Explains why a decision was made. This is the hardest principle to enforce because it requires behavioral change from caseworkers. We implemented a transparent decision message template. It forces the officer to state the outcome in plain language, cite the exact policy, explain what it means, and outline the appeal route. The Critic agent audits samples of these letters to ensure compliance.

Principle 14: Makes it easy to get human assistance. The "Contact Us" link cannot be buried in the footer. It must be contextual. If a user spends more than 3 minutes on a validation error, the UI proactively surfaces the contact centre number and a unique reference code. This bridges the digital-to-physical gap reliably.

Principle 15: Asks for the right knowledge only. Do not ask for a resident's life history to process a tree works application. The SDAP platform audits the data payload of every form against the required backend fields. Extraneous data collection is flagged as a privacy and usability violation.

The Future of Agentic Public Services

The transformation of Fernwood Council is just the beginning. As we deploy these AI-first methodologies across broader public sectors—from healthcare logistics to transit systems like Lighthouse Transit—the value of deterministic, evidence-backed service design becomes undeniable. We are moving from a world where service quality is a subjective debate to a world where it is a mathematically verifiable property of the system.

We are not automating the designer. We are automating the rigour. The human designer is freed to handle the politics, the empathy, and the strategic reframing, while the agentic stack ensures that no user is ever left stranded in a dead-end PDF because a project ran out of time.

Public services are the bedrock of civic trust. By enforcing the 15 Principles of Good Services through the uncompromising lens of the Agentic Service Designer, we ensure that this trust is earned, maintained, and rigorously defended, one commit at a time.

20 The Future of Service Design: The Agentic Practice

From static deliverables to living autonomous platforms

The next frontier for service design moves past faster artifact production into a fundamental shift in what the practice delivers. In the first wave, service designers made invisible systems visible. Now, we make live systems governable: versioned, evidence-linked, ethically reviewable, and continuously improvable. Here, we define the AI-native practice of 2027: a capability stack blending service craft with prompt anatomy, evidence literacy, infrastructure judgment, and institutional ethics.

The Death of Workshop Theater and Static Deliverables

For too long, the discipline of service design has lingered in workshop theater. Teams gather stakeholders in brightly lit rooms, plaster walls with sticky notes, and distill complex human realities into static PDF journey maps. These artifacts look persuasive. Yet they decay the moment delivery teams inherit them. Insights grow stale, untethered from the operational reality of the evolving service in the wild.

The AI-native practice fundamentally rejects this model. We shift from episodic project interventions to continuous service layers. Under this paradigm, journeys, blueprints, standards, and standard operating procedures (SOPs) are living, versioned artifacts in Git repositories, updated continuously by agentic workflows reading live telemetry.

NOTE

My take: The most dangerous habit in modern service design is handing over a static blueprint and walking away. Unwired to live operational telemetry, a design is pure fiction. Our responsibility extends beyond designing the service; we must engineer the system that governs it.

When an agentic swarm powers our practice, the mechanical drag of transcript coding, synthesis transcription, and blueprint maintenance vanishes. Designers are not replaced. Instead, practitioners step into systems orchestration. Dropping the cost of rigor preserves direct traceability from raw field evidence to operational KPIs without losing context.

Market Positioning: The Rise of the Agentic Experience Architect

Market hiring trends already signal this bifurcation. On one side, legacy enterprise and public-sector postings still emphasize traditional human-centered facilitation. On the other, a high-leverage tier has emerged: "Service Designer AI," "Agentic Experience Architect," and "Lead AI Product Designer." These positions fuse senior design craft with deep AI enablement, requiring expertise in generative system principles, agentic workflows, and quantitative evaluation frameworks.

Agentic service blueprinting serves as the primary bridge skill in this landscape. Modeling LLM tool calls, confidence thresholds, autonomous exception handling, and human-in-the-loop escalations alongside classic frontstage and backstage actions distinguishes AI-native practitioners from legacy peers. You must master the autonomy spectrum: knowing when an agent suggests, when it executes, and when it defers to human judgment.

TIP

CORE PRINCIPLE: Design for the autonomy spectrum. Avoid blind automation. Define explicit thresholds where agent confidence scores trigger human pre-approval or full escalation handovers. Graceful degradation belongs directly inside the service architecture.

Continuous Service Architecture and the SDAP Engine

The Service Design Agent Platform (SDAP) establishes the reference architecture for this operating model. It deploys a 10-role agent topology: Orchestrator, Researcher, Librarian, Synthesizer, Mapper, Ideator, Prototyper, Measurer, Presenter, and the Critic. Rather than replacing the Double Diamond, this swarm opens it to streaming evidence.

Figure 20.1 The Continuous Service Architecture. Detailed view of the evidence-linked, agentic feedback loop replacing static handoffs.

Every factual claim inside this architecture traces directly to an immutable evidence hash. The Critic agent enforces this integrity through a deterministic fact-checking pipeline. Teams hand off structured artifacts rather than conversational chat logs. Furthermore, the architecture separates LLM data synthesis from deterministic SVG and HTML rendering. CLI tools and MCP servers keep our infrastructure repeatable, inspectable, and robust.

```
schema-validation.sh
```

```
asd artifact-write --target living-blueprint.json --enforce-schema
asd factcheck --target living-blueprint.json --strict
```

This is service design as code. We establish shared schemas, versioned standards, and automated quality gates. A nightly pipeline verifies blueprint freshness, raising an alert whenever a critical node lacks an evidence refresh within 24 hours. The service operates as an active, governable system.

Human-in-the-Loop Ethical Sovereignty

Extensive automation demands rigorous responsibility. Allowing unconstrained agents to generate maps, scripts, and concepts without explicit fallback rules or ethical checkpoints produces plausible hallucinations and coercive service patterns. Human boundaries must hold firm.

Crucial decisions demand human sovereignty. Service framing, vulnerability assessments, dignity trade-offs, high-stakes escalation language, and structural value redistributions cannot be delegated to unsupervised models. Organizations must mandate ethics review triggers. Every automated outcome requires a plain-language explanation and a clear appeal channel for users.

NOTE

My take: The true test of an AI-native designer lies in disciplined judgment about which decisions must remain socially accountable, rather than raw tool count. We automate the mechanical, never the moral.

Adversarial review keeps our systems honest. While the Critic agent flags evidentiary gaps, human practitioners audit for dignity and fairness. Whenever a journey touches vulnerability, debt, healthcare, or legal eligibility, our strict "stay-human" checklist governs the release.

The Manifesto of the Agentic Service Designer

Operating effectively in this landscape requires clear guiding commitments. Here is the manifesto for the Agentic Service Designer:

Principle	What it means in practice	Operational Test
1. Evidence before elegance	No artifact outranks its evidence chain.	Every major claim links to quotes, telemetry, or policy source.
2. Services stay versioned	Journeys, blueprints, and standards live in Git.	Change history is visible and reviewable.
3. Human accountability must be named	Every live service has a service owner and gate signers.	No anonymous approvals.
4. Automate the mechanical, not the moral	AI drafts and sorts; humans decide stakes and fairness.	High-risk changes blocked without human sign-off.

5. Design for contestability	Users and staff can challenge automated outcomes.	Appeals path exists and is monitored.
------------------------------	---	---------------------------------------

NOTE

My take: Without institutionalizing these principles, our discipline risks declining into superficial decoration over opaque automated systems. We must step forward and govern the platforms we shape.

The transition to an AI-native practice extends far beyond acceleration. It restores the chain of custody between raw human experience and operational service delivery. It builds services that learn, adapt, and remain accountable. The future of service design is agentic, continuous, and uncompromisingly rigorous. It is time to leave the workshop behind and build the operating system.

A The SDAP Method & Artifact Catalog

Complete reference catalog of the 74 canonical skills and 38 artifact templates

This appendix serves as the definitive structural specification for the Service Design Agent Platform (SDAP). It catalogs the 74 specialized method skills and 38 strongly typed artifact templates governing autonomous agent execution, validation schemas, and review gates.

The 12 Machine-Gated Flagship Skills

The core engine relies on 12 machine-checked methods that enforce strict mathematical, referential, and qualitative validation rules before artifacts can be marked as final:

Skill Identifier	Primary Role	Input Schema	Output Artifact Kind	Machine-Checked Acceptance Rule	Review Gate
transcript-mining	Researcher	audio-transcript.json	evidence-register	Every quote must verify against raw transcript character offsets with SHA-256 hash.	Critic
contextual-inquiry	Researcher	field-observation-notes.md	field-observation-log	Deny-list subjective inferences; require physical setting and timestamped action.	Critic
behavioural-personas	Synthesizer	evidence-register	persona-set	Deny-list demographic-only segments; require at least 3 distinct behavioral friction citations.	Critic
mental-model-alignment	Mapper	persona-set	mental-model-map	Towers must represent user cognitive tasks, not system functional specifications.	Critic

ecosystem-value-mapping	Mapper	stakeholder-registry	ecosystem-map	Every actor node must possess at least one reciprocal tangible or intangible value edge.	Critic
journey-from-transcripts	Mapper	evidence-register	customer-journey-map	Emotion scores must be computed as weighted sentiment vectors from cited quotes.	Critic
service-blueprinting	Mapper	customer-journey-map	service-blueprint	5 canonical lanes mandatory; line of visibility strictly enforced; deny "TBD" and "Unknown".	Critic
evidence-gated-blueprint	Mapper	service-blueprint	agentic-blueprint	6th lane (Agent Decisions) requires explicit confidence thresholds and fallback branches.	Critic
hmw-formulation	Ideator	customer-journey-map	hmw-statement-set	Every HMW statement must trace directly to an emotional valley node in the journey.	Critic
nuf-triple-lens-filter	Ideator	service-concept-set	nuf-evaluation-matrix	Concepts scored across New (1-10), Useful (1-10), Feasible (1-10); pass threshold ≥ 21.0.	Critic
kpi-tree-design	Measurer	service-blueprint	kpi-tree-specification	Must bridge operational metrics through experience metrics to business financial outcomes.	Critic

good-services-audit-15	Critic	service-blueprint	service-audit-report	Machine audit across Lou Downe's 15 Principles; minimum passing score 60/75.	Orchestrator
------------------------	--------	-------------------	----------------------	--	--------------

The 38 Artifact Catalog Schemas

Every handoff between agents in the SDAP operating model produces a typed JSON document adhering to the canonical artifact schemas. The 38 artifact kinds span the 5 lifecycle phases of service design:

Artifact Kind (Slug)	Lifecycle Phase	Producing Agent	Consuming Agent(s)	Renderer Target
project-charter	Framing	Orchestrator	All roles	Markdown / HTML
evidence-register	Discovery	Researcher	Synthesizer, Critic	Tabular HTML / JSON
field-observation-log	Discovery	Researcher	Synthesizer	Markdown
stakeholder-registry	Discovery	Researcher	Mapper	HTML Table
affinity-cluster-map	Synthesis	Synthesizer	Mapper, Ideator	Interactive Graph
persona-set	Synthesis	Synthesizer	Mapper, Ideator	Card Grid SVG
insight-deck	Synthesis	Synthesizer	Presenter, Human	Slide Deck HTML
mental-model-map	Mapping	Mapper	Ideator, Prototyper	Indi Young Tower SVG
ecosystem-map	Mapping	Mapper	Ideator, Prototyper	Network Topology SVG
customer-journey-map	Mapping	Mapper	Ideator, Prototyper	Multi-Lane Journey SVG
service-blueprint	Mapping	Mapper	Prototyper, Measurer	5-Lane Blueprint SVG
agentic-blueprint	Mapping	Mapper	Prototyper, Engineers	6-Lane State Machine SVG
orchestration-canvas	Mapping	Mapper	Prototyper	Matrix HTML / SVG
story-map	Mapping	Mapper	Prototyper	Narrative Arc SVG
hmw-statement-set	Ideation	Ideator	Ideator, Prototyper	Card Deck HTML
morphological-matrix	Ideation	Ideator	Ideator	Permutation Table HTML

service-concept-set	Ideation	Ideator	Critic, Prototyper	Concept Sheet HTML
nuf-evaluation-matrix	Ideation	Ideator / Critic	Orchestrator, Human	Scoring Matrix Table
prototype-plan	Prototyping	Prototyper	Human, Testers	Structured Checklist
storyboard-script	Prototyping	Prototyper	Presenter	Visual Comic SVG
roleplay-scenario	Prototyping	Prototyper	Human Cast	Facilitation Script
prototype-test-report	Prototyping	Prototyper	Measurer, Critic	Test Matrix Table
kpi-tree-specification	Measurement	Measurer	Presenter, Leadership	Hierarchical Tree SVG
cx-maturity-assessment	Measurement	Measurer	Presenter	Radar Chart SVG
sop-codebase-spec	Operations	Ideator / Engineers	Frontline Systems	YAML Schema Pack
change-impact-pack	Operations	Measurer	HR / Change Teams	Impact Matrix HTML
wave-rollout-schedule	Operations	Orchestrator	Leadership, Ops	Gantt Timeline SVG
service-recovery-playbook	Operations	Mapper	Frontline Agents	Decision Tree SVG
frontline-co-pilot-card	Operations	Prototyper	Frontline Staff	UI Component Widget
review-veto-log	Governance	Critic	Producing Agent	Defect Audit Table
phase-gate-decision	Governance	Orchestrator / Human	All roles	Signed Milestone Badge

B CLI & MCP Tool Specification

Complete parameter references for the 52 ASD tools, 16 CLI commands, and stdio MCP server

This appendix details the exact command-line syntax, parameter flags, environment configurations, and JSON-RPC interfaces for the `asd` CLI runtime and the `asd-mcp stdio` server.

The 16 Core Engine CLI Commands

The `@asd/runtime` CLI provides 16 primary commands supporting both kebab-case and snake_case aliases:

CLI Command	Engine Alias	Required Flags	Description
<code>asd project-create</code>	<code>sd_project_create</code>	<code>--name</code> , <code>--client</code> , <code>--tier</code>	Initializes a new SDAP workspace repository with canonical directory structure and role permissions.
<code>asd evidence-add</code>	<code>sd_evidence_add</code>	<code>--project</code> , <code>--source</code> , <code>--kind</code> , <code>--body</code>	Appends a content-addressed SHA-256 evidence row. Deny-lists invalid kinds (e.g., observation).
<code>asd evidence-list</code>	<code>sd_evidence_list</code>	<code>--project</code>	Queries the evidence register with optional tag, stage, or speaker filters.
<code>asd artifact-write</code>	<code>sd_artifact_write</code>	<code>--project</code> , <code>--kind</code> , <code>--file</code>	Validates artifact JSON against target schema and commits it to the repository store as draft.
<code>asd artifact-read</code>	<code>sd_artifact_read</code>	<code>--project</code> , <code>--kind</code> , <code>--id</code>	Retrieves the parsed JSON payload and associated provenance metadata for a specific artifact.
<code>asd method-run</code>	<code>sd_method_run</code>	<code>--skill</code> , <code>--input</code> , <code>--params</code>	Executes an SDAP method skill within the isolated runtime sandbox.
<code>asd review</code>	<code>sd_review</code>	<code>--project</code> , <code>--artifact</code> , <code>--verdict</code>	Records an adversarial Critic verdict. Veto demotes artifact to draft and generates a veto log.

asd factcheck	sd_factcheck	--project	Replays cryptographic hash checks, schema validation, and citation integrity across the entire project.
asd render	sd_render	--artifact, --format, --out	Compiles a structured artifact into deterministic SVG, PDF, or HTML using @asd/render .
asd gate-advance	sd_gate_advance	--project, --gate, --token	Advances a phase milestone following sovereign human sign-off.

The asd-mcp Server Stdio Architecture

The `asd-mcp` server exposes the SDAP engine to external coding assistants (Claude Code, Cursor, Windsurf) over standard input/output (stdio). It implements Model Context Protocol (MCP) version 2024-11-05:

mcp-config.json

```
{
  "mcpServers": {
    "asd": {
      "command": "npx",
      "args": ["-y", "@asd/runtime", "mcp"],
      "env": {
        "ASD_WORKSPACE": "${ASD_WORKSPACE:~/.asd/workspace}",
        "ASD_SCALE_TIER": "2",
        "ASD_LOG_LEVEL": "info"
      }
    }
  }
}
```

The server exposes 215 paginated resources rooted at `asd://` :

- `asd://skills/{skill_id}` : The full YAML manifest, input schemas, and execution rules for each method.
- `asd://templates/{kind}` : The JSON Schema Draft-07 specification and starter template for every artifact.
- `asd://knowledge/{book_id}/{chapter}` : Markdown summaries and theoretical citations from the 20-book library.

Cursor pagination is managed via the `nextCursor` token with a maximum page size of 50 resources per request.

C The 20-Book Canonical Cross-Reference

Theoretical concordance mapping the 20 foundational service design books (159 chapters) to agentic methods

This concordance establishes the academic and industry provenance of the Agentic Service Designer methodology. Every skill, prompt template, and heuristic in SDAP traces directly to the canonical literature.

Book Title	Authors	Canonical Chapters	Associated SDAP Method Skills	Primary Artifact Kinds
<i>This Is Service Design Doing</i>	Marc Stickdorn et al.	Ch 01-10 (Processes, Research, Ideation, Prototyping)	transcript-mining , contextual-inquiry , storyboard-authoring , wizard-of-oz	evidence-register , prototype-plan , prototype-test-report
<i>Service-Dominant Logic</i>	Stephen L. Vargo & Robert F. Lusch	Ch 01-08 (Operant Resources, Value Co-Creation, Service Ecosystems)	ecosystem-value-mapping , stakeholder-mapping	ecosystem-map , stakeholder-registry
<i>Good Services</i>	Lou Downe	Ch 01-15 (15 Principles of Service Design)	good-services-audit-15	service-audit-report
<i>Mapping Experiences</i>	Jim Kalbach	Ch 01-11 (Mental Models, Journey Maps, Spatial Diagrams)	mental-model-alignment , journey-from-transcripts , service-blueprinting	mental-model-map , customer-journey-map , service-blueprint
<i>Orchestrating Experiences</i>	Chris Risdon & Patrick Quatromoni	Ch 01-09 (Channels, Ecosystems, Orchestration Frameworks)	orchestration-canvas	orchestration-canvas
<i>The User's Journey</i>	Donna Lichaw	Ch 01-07 (Narrative Arcs, Service Climax, Story Mapping)	story-mapping	story-map

<i>101 Design Methods</i>	Vijay Kumar	Modes 1-7 (Sense Intent, Know People, Frame Insights, Explore Concepts)	hmw-formulation , morphological-concept-generation , nuf-triple-lens-filter	hmw-statement-set , service-concept-set , nuf-evaluation-matrix
<i>The Service Profit Chain</i>	James L. Heskett, W. Earl Sasser, Leonard A. Schlesinger	Ch 01-08 (Internal Service Quality, Frontline Enablement, Customer Loyalty)	frontline-enablement , kpi-tree-design	frontline-co-pilot-card , kpi-tree-specification
<i>Service Management and Marketing</i>	Christian Grönroos	Ch 01-12 (Total Perceived Quality, 5 GAPS, Service Recovery)	service-recovery-playbook , kpi-tree-design	service-recovery-playbook , kpi-tree-specification
<i>Outside In</i>	Harley Manning & Kerry Bodine	Ch 01-09 (The 6 CX Maturity Disciplines)	cx-maturity-audit	cx-maturity-assessment
<i>The Experience Economy</i>	B. Joseph Pine II & James H. Gilmore	Ch 01-08 (The Four Realms of Experience Staging)	experience-staging-design	service-concept-set
<i>Design Unbound</i>	Ann M. Pendleton-Jullian & John Seely Brown	Volumes 1 & 2 (Designing for Emergence in Complex Systems)	continuous-service-architecture	agentic-blueprint

APPLIED AI FIELD GUIDES

Disclaimer

This book was written with the assistance of AI tools.
All tool references, JSON schemas, code examples,
and CLI commands reflect platform capabilities as of
August 2026.

The Agentic Service Designer

Version 1.0.0 - August 2026

Written by Sam Mozaffari

License: CC-BY-NC-ND-4.0

